# VittaCare

Documentação completa – Blocos 1 a 9

# Sumário geral

Bloco 1 – Concepção e escopo ..... 3

Bloco 2 – Requisitos ..... 40

Bloco 3 – Arquitetura ..... 58

Bloco 4 – Diagramas UML ..... 81

Bloco 5 – Banco de dados ..... 102

Bloco 7 – Planejamento e estimativas ..... 129

Bloco 8 – Documentação dev ..... 141

Bloco 9 – Qualidade e processo ..... 165

Anexo A – Telas do sistema ..... 174

Os números indicam a página do PDF. O Bloco 6 não faz parte desta entrega.

# VittaCare

Concepção e Escopo – Bloco 1

# Sumário

1. DOCUMENTO DE VISÃO ..... 3
1.1 Identificação do projeto ..... 3
1.2 Visão geral ..... 3
1.3 Problema identificado ..... 3
1.4 Solução proposta ..... 4
1.5 Objetivo geral ..... 4
1.6 Objetivos específicos ..... 4
1.7 Público-alvo ..... 4
1.8 Proposta de valor ..... 5
1.9 Escopo do sistema ..... 5
1.10 Restrições e premissas ..... 6
1.11 Indicadores de sucesso propostos ..... 7
2. PERSONAS ..... 8
2.1 Persona 1 — Paciente ..... 8
2.2 Persona 2 — Profissional de saúde ..... 8
2.3 Persona 3 — Funcionária administrativa ..... 9
2.4 Persona 4 — Administrador ..... 10
3. MAPA DE STAKEHOLDERS ..... 11
3.1 Análise dos stakeholders ..... 12
3.2 Estratégia de relacionamento ..... 13
4. ESTUDO DE VIABILIDADE ..... 13
4.1 Viabilidade técnica ..... 13
4.1.1 Recursos técnicos necessários ..... 14
4.1.1.1 Justificativa técnica: ..... 14
4.1.2 Profissionais necessários ..... 17
4.1.2.1Como ficaria a composição para o orçamento? ..... 19
4.1.3 Conclusão análise técnica e profissional ..... 19
4.2 Viabilidade operacional ..... 20
4.2.1Benefícios operacionais esperados ..... 20
4.2.2 Possíveis dificuldades ..... 21
4.3 Viabilidade financeira ..... 21
4.3.1 Quanto custariam os recursos técnicos? ..... 21
4.3.2 Resumo dos custos da infraestrutura ..... 25
4.3.2 Quanto custariam os profissionais? ..... 26
4.3.3 Cálculo utilizado e definição da mensalidade ..... 27

4.3.4 CÁLCULO DO PONTO DE EQUILÍBRIO ..... 29
4.4 Viabilidade legal e regulatória ..... 30
4.4.1 Conclusão Viabilidade Legal ..... 31
4.5 Conclusão geral da viabilidade ..... 31
5. MATRIZ DE RISCOS INICIAIS ..... 33
5.1 Risco 1 — Vazamento de dados pessoais e sensíveis ..... 33
5.2 Risco 2 — Conflitos e duplicidade de agendamentos ..... 33
5.3 Risco 3 — Indisponibilidade da plataforma ..... 34
5.4 Risco 4 — Resistência dos usuários à adoção do sistema ..... 34
5.5 Risco 5 — Aumento inesperado dos custos de infraestrutura ..... 35
5.6 Resumo da matriz de riscos ..... 35
5.7 Considerações finais ..... 35

# 1. DOCUMENTO DE VISÃO

## 1.1 Identificação do projeto

***VittaCare***

**Mais organização para quem cuida. Mais facilidade para quem precisa de cuidado.**

**Categoria:** Sistema de gestão de saúde
**Domínio:** Saúde.

## 1.2 Visão geral

O VittaCare é uma plataforma digital desenvolvida para auxiliar na gestão de consultas e atendimentos em clínicas e ambulatórios. O sistema busca centralizar os processos de agendamento, gerenciamento de horários, organização das agendas profissionais e acompanhamento dos atendimentos.

A plataforma será utilizada por pacientes, médicos, recepcionistas e administradores, oferecendo funcionalidades específicas de acordo com as permissões de cada perfil.

## 1.3 Problema identificado

Clínicas e ambulatórios podem enfrentar dificuldades na organização de seus atendimentos devido à utilização de processos manuais, informações descentralizadas e ausência de ferramentas integradas para gerenciamento de agendas.

Entre os principais problemas identificados estão:

- Conflitos de horários e agendamentos duplicados.
- Dificuldade para visualizar horários disponíveis.
- Processos demorados de marcação e cancelamento de consultas.
- Falta de centralização das informações administrativas.
- Dificuldade no acompanhamento de faltas e cancelamentos.
- Sobrecarga dos funcionários responsáveis pela recepção.

Essas dificuldades podem comprometer a eficiência operacional das instituições e prejudicar a experiência dos pacientes.

## 1.4 Solução proposta

O VittaCare propõe uma plataforma que centraliza o gerenciamento de consultas e facilita a comunicação entre pacientes e profissionais de saúde.

Por meio do sistema, os pacientes poderão consultar profissionais, visualizar horários disponíveis, realizar agendamentos e acompanhar suas consultas. Os profissionais poderão organizar suas agendas e consultar os atendimentos programados. Enquanto, os funcionários administrativos terão ferramentas para gerenciar cadastros, agendamentos e informações operacionais.

## 1.5 Objetivo geral

Desenvolver a concepção de um sistema digital para otimizar a gestão de consultas e atendimentos ambulatoriais, proporcionando maior organização administrativa, facilidade de acesso aos serviços e eficiência no gerenciamento das agendas profissionais.

## 1.6 Objetivos específicos

- Centralizar as informações de agendamento.
- Facilitar a marcação, remarcação e cancelamento de consultas.
- Permitir a visualização da disponibilidade dos profissionais.
- Reduzir conflitos de horários.
- Organizar o fluxo administrativo dos atendimentos.
- Permitir o acompanhamento do histórico de agendamentos.
- Implementar controle de acesso conforme os diferentes perfis de usuários.

## 1.7 Público-alvo

O sistema contará com quatro perfis principais de usuários:

- **Paciente:** poderá consultar especialidades e profissionais, visualizar horários disponíveis, realizar e acompanhar agendamentos.
- **Profissional de saúde:** poderá gerenciar sua disponibilidade, visualizar sua agenda e acompanhar os atendimentos programados.

- **Recepcionista:** poderá cadastrar e atualizar pacientes, realizar agendamentos em nome deles e gerenciar as operações cotidianas da recepção.
- **Administrador:** será responsável pelo gerenciamento das contas de usuários, profissionais, especialidades e permissões de acesso, além do acompanhamento administrativo da plataforma.

A separação entre recepcionista e administrador permite estabelecer diferentes níveis de responsabilidade e acesso às funcionalidades do sistema.

No modelo comercial proposto, clínicas e ambulatórios serão os clientes contratantes da plataforma, mediante assinatura periódica. Pacientes, profissionais de saúde e funcionários administrativos serão usuários da solução, com permissões e funcionalidades adequadas aos seus respectivos perfis.

## 1.8 Proposta de valor

O VittaCare oferece uma solução centralizada para gerenciamento de consultas, reduzindo a dependência de processos manuais e facilitando a organização das atividades administrativas e profissionais.

Seu principal valor está em integrar diferentes perfis de usuários em uma única plataforma, proporcionando maior controle sobre os agendamentos e melhor organização dos serviços de saúde.

A plataforma será disponibilizada no modelo SaaS, por meio de um plano único de assinatura que dará acesso a todas as funcionalidades previstas no escopo do sistema. Dessa forma, todas as clínicas contratantes terão acesso ao mesmo conjunto de recursos, independentemente de seu porte.

## 1.9 Escopo do sistema

### Funcionalidades incluídas

- Cadastro e gerenciamento de pacientes.
- Cadastro e gerenciamento de profissionais.
- Autenticação e controle de acesso.
- Recuperação de senha.
- Consulta de especialidades.
- Pesquisa de profissionais.

- Visualização de horários disponíveis.
- Agendamento de consultas.
- Cancelamento e remarcação de consultas.
- Histórico de agendamentos.
- Notificações e lembretes.
- Gerenciamento de disponibilidade dos profissionais.
- Registro de comparecimento e ausência.
- Agendamento realizado pela recepção.
- Relatórios administrativos.
- Gerenciamento de permissões.

Todas as funcionalidades descritas nesta seção compõem o escopo funcional disponibilizado pelo plano único de assinatura do VittaCare. Funcionalidades classificadas como complementares poderão ser disponibilizadas em versões posteriores, sem a criação de categorias comerciais de assinatura.

### Funcionalidades fora do escopo

- Diagnóstico médico automatizado.
- Prescrição eletrônica de medicamentos.
- Realização de consultas por videoconferência.
- Integração com equipamentos médicos.
- Inteligência artificial para análise clínica.
- Gestão financeira completa da clínica.
- Processamento de pagamentos.
- Gerenciamento completo de prontuários eletrônicos.

A delimitação dessas funcionalidades permite concentrar o desenvolvimento conceitual nos processos de agendamento e organização administrativa.

## 1.10 Restrições e premissas

- O sistema deverá funcionar por meio de uma interface web.
- Os usuários precisarão de conexão com a internet.

- Cada perfil terá permissões específicas.
- As informações pessoais deverão ser protegidas conforme a LGPD.
- O sistema deverá possuir mecanismos de autenticação e controle de acesso.
- O funcionamento dependerá da disponibilidade da infraestrutura contratada.
- O sistema não substituirá o julgamento ou a atuação de profissionais de saúde.
- O VittaCare será comercializado no modelo SaaS, mediante assinatura periódica.
- A plataforma terá um único plano comercial, com acesso a todas as funcionalidades incluídas no escopo.
- As clínicas e ambulatórios serão os clientes contratantes, enquanto pacientes, profissionais e funcionários administrativos serão usuários da plataforma.
- A quantidade inicial estimada é de 20 clínicas clientes.

## 1.11 Indicadores de sucesso propostos

- Redução de conflitos de agendamento.
- Redução do tempo necessário para realizar uma consulta na agenda.
- Disponibilidade de visualização de horários atualizados.
- Registro correto dos agendamentos.
- Facilidade de utilização pelos diferentes perfis.
- Proteção e controle de acesso às informações.

Os indicadores deverão receber metas quantitativas durante a elaboração dos requisitos não funcionais.

# 2. PERSONAS

## 2.1 Persona 1 — Paciente

**Nome:** Mariana Oliveira
**Idade:** 29 anos
**Profissão:** Professora

**Contexto:**

Mariana possui uma rotina de trabalho ocupada e precisa conciliar seus compromissos profissionais com consultas médicas. Atualmente, encontra dificuldades para entrar em contato com clínicas durante o horário comercial e verificar a disponibilidade de profissionais.

**Objetivos:**

- Agendar consultas sem precisar realizar ligações;
- Visualizar horários disponíveis;
- Receber lembretes de consultas;
- Cancelar ou remarcar atendimentos com facilidade;
- Consultar seu histórico de agendamentos.

**Dificuldades:**

- Demora no atendimento telefônico;
- Falta de informações sobre horários disponíveis;
- Esquecimento de consultas agendadas;
- Dificuldade para realizar alterações nos agendamentos.

**Como o VittaCare ajuda:**

Permite que Mariana consulte a disponibilidade dos profissionais e realize seus agendamentos de maneira digital, além de acompanhar suas consultas e receber lembretes.

---

## 2.2 Persona 2 — Profissional de saúde

**Nome:** Ricardo Mendes
**Idade:** 38 anos
**Profissão:** Médico clínico geral

**Contexto:**

Ricardo atende em uma clínica particular e depende da organização da recepção para controlar seus horários. Alterações frequentes na agenda e falta de atualização das informações podem provocar conflitos e comprometer sua rotina profissional.

**Objetivos:**

- Visualizar sua agenda diária e semanal;
- Gerenciar horários disponíveis;
- Acompanhar consultas agendadas;
- Consultar informações necessárias para a organização dos atendimentos;
- Registrar comparecimentos e ausências.

**Dificuldades:**

- Conflitos de horários;
- Falta de atualização da agenda;
- Dificuldade para acompanhar cancelamentos;
- Dependência de comunicação manual com a recepção.

**Como o VittaCare ajuda:**

Disponibiliza uma agenda centralizada, permitindo visualizar os atendimentos programados e gerenciar a disponibilidade profissional.

---

## 2.3 Persona 3 — Funcionária administrativa

**Nome:** Fernanda Costa
**Idade:** 34 anos
**Profissão:** Recepcionista de clínica médica

**Contexto:**

Fernanda trabalha na recepção de uma clínica que recebe diversos pacientes diariamente. Ela precisa organizar consultas, verificar disponibilidade dos médicos, atualizar cadastros e lidar com cancelamentos e remarcações.

**Objetivos:**

- Gerenciar os agendamentos dos pacientes.

- Consultar rapidamente os horários disponíveis.
- Cadastrar e atualizar informações.
- Organizar as agendas dos profissionais.
- Acompanhar cancelamentos e ausências.

**Dificuldades:**

- Grande volume de solicitações.
- Informações descentralizadas.
- Possibilidade de agendamentos duplicados.
- Retrabalho na atualização de horários.
- Dificuldade para localizar informações rapidamente.

**Como o VittaCare ajuda:**

Centraliza as operações administrativas, permitindo que Fernanda gerencie os agendamentos e cadastros em uma única plataforma.

## 2.4 Persona 4 — Administrador

**Nome:** Roberto Almeida
**Idade:** 45 anos
**Profissão:** Administrador de clínica médica

**Contexto:**

Roberto é responsável pela administração de uma clínica médica e precisa acompanhar o funcionamento dos serviços, gerenciar os profissionais cadastrados e garantir que os funcionários tenham acesso apenas às funcionalidades necessárias para suas atividades.

**Objetivos:**

- Gerenciar as contas dos usuários da plataforma.
- Cadastrar, atualizar e desativar profissionais de saúde.
- Gerenciar especialidades médicas.
- Definir permissões de acesso para os diferentes perfis.
- Acompanhar indicadores administrativos da clínica.

**Dificuldades:**

- Dificuldade para controlar os acessos dos funcionários.

- Falta de centralização das informações administrativas.
- Necessidade de acompanhar as atividades da clínica.
- Risco de acesso não autorizado a informações restritas.

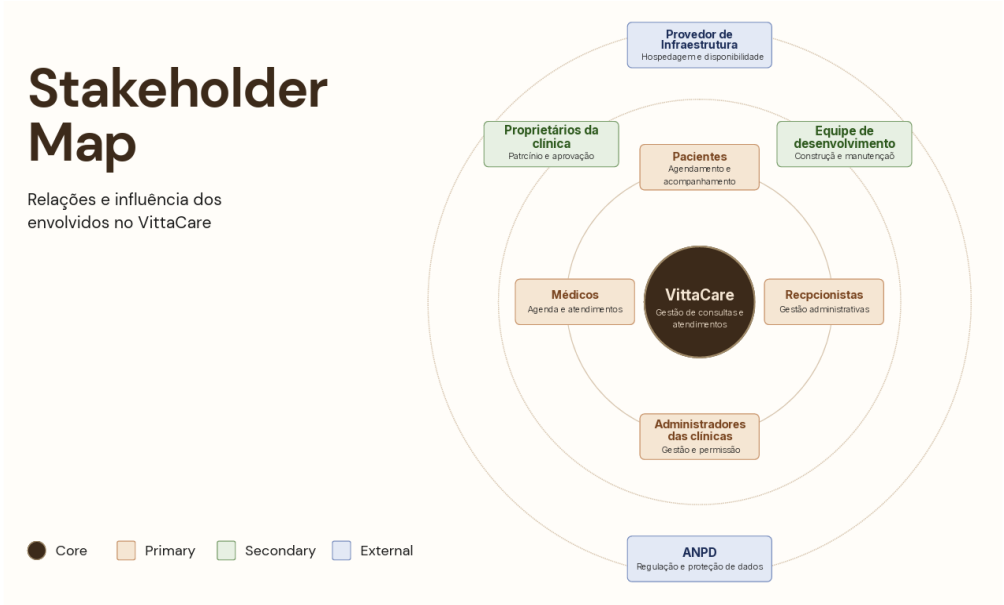
**Como o VittaCare ajuda:**

Disponibiliza ferramentas administrativas para gerenciamento de usuários, profissionais, especialidades e permissões, além de permitir o acompanhamento das informações operacionais da clínica.

## 3. MAPA DE STAKEHOLDERS

| Stakeholder | Interesse no projeto | Influência | Participação |
|---|---|---|---|
| **Pacientes** | Facilidade para agendar e acompanhar consultas | Média | Usuários finais |
| **Médicos** | Organização da agenda e dos atendimentos | Alta | Usuários finais |
| **Recepcionistas** | Facilidade no gerenciamento administrativo | Alta | Usuários finais |
| **Administradores da clínica** | Controle e eficiência operacional | Alta | Gestão e tomada de decisões |
| **Proprietários da clínica** | Retorno financeiro e melhoria dos serviços | Alta | Patrocínio e aprovação do projeto |
| **Equipe de desenvolvimento** | Construção e manutenção da plataforma | Alta | Desenvolvimento e suporte |

| | | | |
|---|---|---|---|
| **Provedor de infraestrutura** | Disponibilidade dos serviços contratados | Média | Hospedagem e infraestrutura |
| **Autoridade Nacional de Proteção de Dados (ANPD)** | Cumprimento das normas de proteção de dados pessoais | Alta | Referência regulatória e fiscalização |




## 3.1 Análise dos stakeholders

### *Alta influência e alto interesse*

**Administradores e proprietários das clínicas:** são fundamentais para a aprovação, implantação e continuidade do sistema. Suas necessidades devem ser consideradas durante todo o planejamento.

### *Alta influência e interesse operacional*

**Médicos e recepcionistas:** participam diretamente dos processos que o sistema pretende otimizar. Suas necessidades devem orientar as funcionalidades e a organização das interfaces.

### *Interesse alto e influência média*

**Pacientes:** são os principais beneficiários da facilidade de agendamento e acompanhamento das consultas. Sua experiência deve orientar a usabilidade da plataforma.

*Influência técnica*

**Equipe de desenvolvimento e provedor de infraestrutura:** são responsáveis por aspectos técnicos, manutenção, segurança e disponibilidade da plataforma.

*Influência regulatória*

**ANPD:** suas normas e orientações devem ser consideradas para que o tratamento de dados pessoais esteja em conformidade com a legislação aplicável.

## 3.2 Estratégia de relacionamento

- **Administradores e proprietários:** manter comunicação frequente e apresentar resultados.
- **Médicos e recepcionistas:** envolver no levantamento e validação dos requisitos.
- **Pacientes:** considerar suas necessidades de acessibilidade e facilidade de utilização.
- **Equipe técnica:** manter documentação atualizada e comunicação contínua.
- **Provedor de infraestrutura:** acompanhar disponibilidade, custos e condições dos serviços.
- **ANPD:** acompanhar as obrigações legais e orientações relacionadas à proteção de dados.

# 4. ESTUDO DE VIABILIDADE

## 4.1 Viabilidade técnica

O desenvolvimento do VittaCare é tecnicamente viável, pois suas funcionalidades podem ser implementadas utilizando tecnologias consolidadas para desenvolvimento de aplicações web.

O sistema não exige, em sua proposta inicial, recursos avançados de processamento, inteligência artificial ou integração com equipamentos médicos.

A arquitetura poderá ser estruturada inicialmente como um monólito modular, permitindo organizar as funcionalidades em módulos independentes dentro de uma única aplicação.

### 4.1.1 Recursos técnicos necessários

| Recurso | Tecnologia | Finalidade |
|---|---|---|
| ***Servidor da aplicação*** | Amazon EC2 | Executar o backend e a aplicação |
| ***Banco de dados*** | Amazon RDS (PostgreSQL) | Armazenar informações de pacientes, profissionais e agendamentos |
| ***Armazenamento*** | Amazon S3 | Armazenar arquivos, caso necessário |
| ***Monitoramento*** | Amazon CloudWatch | Monitorar disponibilidade, erros e desempenho |
| ***Backup*** | AWS Backup / RDS Backup | Garantir recuperação dos dados |
| ***Segurança de rede*** | AWS VPC e Security Groups | Restringir e controlar o acesso à infraestrutura |
| ***Versionamento*** | Git e GitHub | Controlar versões do código |
| ***Containerização*** | Docker | Padronizar os ambientes de desenvolvimento e execução |
| ***Testes*** | Jest / ferramentas equivalentes | Verificar o funcionamento das funcionalidades |

#### 4.1.1.1 Justificativa técnica:

##### *1. Amazon EC2 — Hospedagem do backend*

O Amazon EC2 foi escolhido por permitir executar o backend do VittaCare em uma máquina virtual configurável, oferecendo controle sobre o processamento, memória e sistema operacional utilizados. Isso permite dimensionar os recursos conforme a quantidade de usuários e agendamentos realizados.

Como alternativa, poderia ser utilizado o AWS Lambda, porém o EC2 oferece maior controle sobre o ambiente de execução e é adequado para hospedar continuamente a aplicação monolítica modular proposta.

### *2. Amazon RDS (PostgreSQL) — Banco de dados*

O Amazon RDS foi escolhido por oferecer um serviço gerenciado de banco de dados, reduzindo a necessidade de administrar manualmente tarefas como manutenção, atualizações e backups.

O PostgreSQL é adequado porque o VittaCare trabalha com informações relacionadas entre si, como pacientes, profissionais, especialidades e agendamentos. Seu modelo relacional permite utilizar chaves estrangeiras e restrições de integridade para manter a consistência dos dados e evitar conflitos nos horários.

Como alternativa, poderia ser utilizado o MongoDB, mas seu modelo orientado a documentos não oferece a mesma estrutura relacional que atende às necessidades identificadas no projeto.

### *3. Amazon S3 — Armazenamento de arquivos*

O Amazon S3 foi escolhido por permitir armazenar arquivos de maneira independente do servidor principal da aplicação. Isso evita utilizar o espaço de armazenamento da máquina EC2 para arquivos e facilita a expansão da capacidade conforme a necessidade.

Como alternativa, os arquivos poderiam ser armazenados diretamente no servidor, porém isso aumentaria a dependência do armazenamento local e dificultaria a expansão e a manutenção da aplicação.

Como o escopo inicial não prevê gerenciamento completo de prontuários ou armazenamento de documentos médicos, sua utilização será limitada aos arquivos que eventualmente forem necessários.

### *4. Amazon CloudWatch — Monitoramento*

O Amazon CloudWatch foi escolhido por permitir acompanhar o funcionamento da infraestrutura, coletando métricas, registros e informações sobre possíveis falhas.

Isso é importante porque o VittaCare depende da disponibilidade do sistema para que pacientes e funcionários consigam consultar e atualizar agendamentos. O monitoramento permite identificar problemas e auxiliar na manutenção preventiva.

Como alternativa, poderiam ser utilizadas ferramentas externas, como Prometheus e Grafana, porém o CloudWatch possui integração direta com os serviços da AWS, simplificando a configuração inicial.

### *5. AWS Backup / RDS Backup — Backup*

Os serviços de backup foram escolhidos para proteger as informações armazenadas no banco de dados e permitir sua recuperação em situações de falha, exclusão acidental ou problemas na infraestrutura.

Essa medida é especialmente importante porque o VittaCare armazena informações pessoais e registros de agendamento que precisam manter sua integridade e disponibilidade.

Como alternativa, poderiam ser desenvolvidos mecanismos próprios de backup, porém isso exigiria maior esforço de implementação e manutenção. A utilização dos recursos gerenciados da AWS simplifica esse processo.

### *6. AWS VPC e Security Groups — Segurança de rede*

A AWS VPC foi escolhida para permitir a criação de uma rede virtual isolada, na qual os componentes da infraestrutura podem ser organizados e ter seu acesso controlado.

Os Security Groups complementam essa proteção, permitindo definir quais conexões podem acessar os servidores e o banco de dados.

Essa configuração reduz a exposição dos componentes da aplicação a acessos não autorizados e contribui para a proteção das informações dos usuários.

Como alternativa, seria possível utilizar uma infraestrutura sem isolamento adequado entre os serviços, mas isso aumentaria a superfície de exposição e dificultaria o controle de acesso.

### *7. Git e GitHub — Versionamento*

O Git foi escolhido para controlar as versões do código-fonte, permitindo acompanhar alterações, recuperar versões anteriores e desenvolver funcionalidades sem comprometer diretamente o código principal.

O GitHub será utilizado como plataforma para armazenar os repositórios e facilitar a colaboração entre os integrantes da equipe.

Como alternativa, poderia ser utilizado outro sistema de versionamento, como SVN, porém o Git oferece um modelo distribuído que facilita o trabalho simultâneo e o gerenciamento de diferentes branches.

### *8. Docker — Containerização*

O Docker foi escolhido para padronizar o ambiente de execução da aplicação, permitindo que os componentes funcionem com suas dependências configuradas de maneira consistente.

Isso reduz problemas relacionados às diferenças entre os ambientes de desenvolvimento, testes e produção, facilitando a implantação e a manutenção do VittaCare.

Como alternativa, a aplicação poderia ser executada diretamente no sistema operacional do servidor, porém isso aumentaria a possibilidade de conflitos entre dependências e diferenças de configuração.

*9. JUnit e Jest — Testes automatizados*

O JUnit 5, com Mockito, foi escolhido para testar o back-end Java/Spring Boot, pois já acompanha o starter de testes do Spring Boot e permite validar regras de negócio, endpoints, autorização e transações. O Jest, com React Testing Library, foi escolhido para testar os componentes do front-end React, pois executa testes de componentes com simulação de DOM integrada.

No VittaCare, isso é importante para validar operações como cadastro, autenticação, agendamento, cancelamento e remarcação de consultas, reduzindo a possibilidade de erros durante as atualizações.

Como alternativa, poderiam ser utilizadas outras ferramentas, como TestNG no back-end ou Mocha no front-end, porém elas exigiriam dependências e configurações adicionais sem benefício para o escopo do projeto.

### 4.1.2 Profissionais necessários

O desenvolvimento do VittaCare exige profissionais especializados em diferentes etapas da construção e manutenção do sistema.

*Equipe Front-end*

| Profissional | Responsabilidade |
|---|---|
| **Desenvolvedor Front-end** | Desenvolver as interfaces web dos pacientes, médicos, recepcionistas e administradores. |
| **UI/UX Designer** | Planejar a experiência de utilização, acessibilidade e organização visual das interfaces. |

O VittaCare possui diferentes perfis de usuários, cada um com necessidades específicas. Uma equipe dedicada ao front-end permite desenvolver interfaces adequadas a cada perfil, priorizando facilidade de utilização e acessibilidade.

*Equipe Back-end*

| Profissional | Responsabilidade |
|---|---|
| **Desenvolvedor Back-end** | Implementar as funcionalidades e regras de negócio previamente especificadas, desenvolver os mecanismos de autenticação e realizar a integração entre o Front-end, o banco de dados e os demais componentes do sistema. |
| **Desenvolvedor Back-end** | Auxiliar na implementação das funcionalidades, integração dos componentes e comunicação entre as diferentes partes da aplicação. |

A equipe de Back-end será responsável por transformar as especificações técnicas e funcionais previamente documentadas em código executável. Suas atividades envolverão a implementação das funcionalidades, autenticação dos usuários, desenvolvimento das APIs e integração com o Front-end, banco de dados e demais serviços definidos na arquitetura.

As regras de negócio, tecnologias, padrões de desenvolvimento e estrutura da aplicação serão estabelecidos durante a etapa de planejamento e documentação, cabendo aos desenvolvedores seguir essas definições durante a implementação.

*Equipe de Banco de Dados*

| Profissional | Responsabilidade |
|---|---|
| **DBA / Engenheiro de Dados** | Implementar e administrar a estrutura do banco de dados conforme a modelagem, os relacionamentos, os tipos de dados e as restrições previamente definidos no projeto. |

A equipe de Banco de Dados será responsável por transformar a modelagem conceitual e lógica em uma estrutura funcional, seguindo as especificações estabelecidas durante o planejamento. Também será responsável pela implementação das tabelas, relacionamentos, índices, restrições de integridade, mecanismos de backup e configurações de segurança do banco de dados.

*Equipe de apoio*

| Profissional | Responsabilidade |
|---|---|
| **Arquiteto de Software** | Definir a arquitetura, padrões e integração entre as equipes. |

| | |
|---|---|
| **Engenheiro DevOps** | Gerenciar infraestrutura AWS, containers e implantação. |
| **Analista de QA** | Realizar testes funcionais, integração, desempenho e segurança. |
| **Analista de Sistemas / PO** | Levantar requisitos, organizar prioridades e validar entregas. |

#### 4.1.2.1Como ficaria a composição para o orçamento?

| PROFISSIONAL | QUANTIDADE |
|---|---|
| **ANALISTA DE SISTEMAS / PO** | 1 |
| **ARQUITETO DE SOFTWARE** | 1 |
| **DESENVOLVEDOR FRONT-END** | 1 |
| **DESENVOLVEDOR BACK-END** | 2 |
| **DBA / ENGENHEIRO DE DADOS** | 1 |
| **ENGENHEIRO DEVOPS** | 1 |
| **ANALISTA DE QA** | 1 |
| **UI/UX DESIGNER** | 1 |
| **TOTAL** | **9 profissionais** |

### 4.1.3 Conclusão análise técnica e profissional

O VittaCare apresenta viabilidade técnica, considerando a utilização de tecnologias consolidadas e uma arquitetura monolítica modular, adequada à complexidade e ao escopo inicial do sistema.

Para garantir que o desenvolvimento possa ser iniciado sem depender de novas definições estruturais, será necessário concluir previamente o planejamento técnico do projeto. Isso inclui a documentação dos requisitos funcionais e não funcionais, definição da arquitetura, modelagem do banco de dados, estrutura dos módulos, especificação das APIs, configuração da infraestrutura e definição dos padrões de segurança e testes.

Em relação aos profissionais, o desenvolvimento será dividido principalmente entre três áreas:

- **Front-end:** responsável pela implementação das interfaces e experiências de utilização dos diferentes perfis de usuários.
- **Back-end:** responsável autenticação e integração com o banco de dados.
- **Banco de Dados:** responsável pela modelagem, estruturação e definição das regras de integridade e armazenamento das informações.

Também será necessária a atuação de profissionais responsáveis pela arquitetura de software, análise de requisitos, qualidade e infraestrutura, principalmente durante a etapa de planejamento e preparação técnica.

Dessa forma, quando as equipes de Front-end e Back-end iniciarem suas atividades, deverão receber toda a documentação necessária, incluindo protótipos das interfaces, contratos das APIs, modelo do banco de dados, regras de negócio, arquitetura definida, ambiente de desenvolvimento e backlog organizado.

Conclui-se que o projeto apresenta viabilidade técnica e profissional, desde que a etapa de planejamento e documentação seja concluída antes do início da implementação. Essa organização permite reduzir retrabalho, evitar decisões técnicas conflitantes e possibilitar que os desenvolvedores iniciem suas atividades com responsabilidades e orientações claramente estabelecidas.

## 4.2 Viabilidade operacional

O VittaCare poderá ser utilizado por clínicas e ambulatórios que desejam digitalizar seus processos de agendamento e organização de consultas.

Sua implantação exigirá adaptação dos funcionários aos novos procedimentos e treinamento básico para utilização da plataforma.

### 4.2.1 Benefícios operacionais esperados

- Centralização dos agendamentos.
- Redução de tarefas administrativas manuais.
- Maior organização das agendas profissionais.
- Facilidade para localizar informações.
- Redução de conflitos de horários.
- Melhor acompanhamento dos atendimentos.

### 4.2.2 Possíveis dificuldades

- Resistência de funcionários à mudança de processos.
- Necessidade de treinamento.
- Dependência de conexão com a internet.
- Necessidade de atualização e manutenção da plataforma.

## 4.3 Viabilidade financeira

A análise de viabilidade financeira do VittaCare considera os custos necessários para o desenvolvimento, hospedagem, manutenção e operação da plataforma, assim como a receita recorrente proveniente das assinaturas contratadas pelas clínicas.

O modelo de negócio adotado será o SaaS (Software as a Service), no qual as clínicas contratam o acesso à plataforma mediante o pagamento de uma assinatura periódica. Será disponibilizado um plano único, que dará acesso a todas as funcionalidades previstas no escopo do sistema, sem diferenciação de recursos entre categorias de assinatura.

Para a projeção inicial, considera-se uma carteira de 20 clínicas clientes. A receita será calculada com base na quantidade de assinaturas ativas e no preço mensal estabelecido para o plano.

### 4.3.1 Quanto custariam os recursos técnicos?

Para estimar os custos da infraestrutura do VittaCare, foi utilizada a **AWS Pricing Calculator**, considerando a região América do Sul (São Paulo). A escolha dos serviços foi baseada na arquitetura proposta para o sistema, que utiliza uma aplicação web organizada como monólito modular, banco de dados PostgreSQL e serviços complementares de armazenamento, monitoramento, backup e distribuição de conteúdo.

Como o VittaCare ainda não está em produção, não existem dados reais de utilização. Portanto, foram adotadas premissas de dimensionamento para representar um cenário inicial de funcionamento. Os valores apresentados são estimativas mensais e podem sofrer alterações conforme o crescimento da quantidade de usuários, o volume de agendamentos e a utilização dos recursos.

#### *4.3.1.1 Amazon EC2 — Hospedagem da aplicação*

O Amazon EC2 foi escolhido para hospedar e executar o backend do VittaCare. Esse serviço disponibiliza máquinas virtuais configuráveis, permitindo definir recursos de processamento, memória e armazenamento de acordo com as necessidades da aplicação.

A estimativa considera duas instâncias de computação, com configuração inicial de 2 vCPUs e 2 GB de memória cada, utilizando o modelo de cobrança sob demanda e funcionamento contínuo durante 730 horas mensais.

A utilização de duas instâncias foi considerada para permitir a distribuição da aplicação entre servidores e contribuir para a disponibilidade do sistema. Essa configuração também possibilita ampliar a capacidade computacional conforme o crescimento da utilização da plataforma.

O EC2 foi escolhido em vez de alternativas como AWS Lambda porque a arquitetura proposta é baseada em um monólito modular, que poderá permanecer em execução em um ambiente controlado, oferecendo maior autonomia sobre sua configuração.

**Custo mensal estimado: US$ 39,13.**

#### *4.3.1.2 Application Load Balancer (ALB) — Distribuição de requisições*

O Application Load Balancer foi considerado para distribuir as requisições recebidas pelo VittaCare entre as duas instâncias Amazon EC2 responsáveis pela execução da aplicação. Essa distribuição permite compartilhar a carga de processamento entre os servidores e contribui para a disponibilidade do sistema.

O balanceador realiza verificações de integridade (*health checks*) nas instâncias configuradas, permitindo identificar possíveis falhas e deixar de encaminhar novas requisições para destinos considerados não saudáveis. Dessa forma, caso uma instância apresente problemas, o tráfego poderá continuar sendo direcionado para a outra instância, desde que ela permaneça operacional.

A estimativa foi configurada considerando um Application Load Balancer em funcionamento contínuo durante 730 horas mensais, na região América do Sul (São Paulo). Para o dimensionamento inicial, foram consideradas as seguintes premissas: 1.000 usuários simultâneos, aproximadamente 100 requisições por segundo e tráfego estimado de 7,2 GB por hora.

O custo considera a cobrança por hora de funcionamento do balanceador e o consumo de unidades de capacidade (LCUs), podendo variar conforme o volume efetivo de conexões e dados processados.

**Custo mensal estimado: US$ 82,64.**

#### *4.3.1.3 Amazon RDS — Banco de dados PostgreSQL*

O Amazon RDS foi selecionado para armazenar e gerenciar os dados do VittaCare, incluindo informações de pacientes, profissionais de saúde, recepcionistas, administradores, especialidades e agendamentos.

Foi escolhido o PostgreSQL por ser um sistema gerenciador de banco de dados relacional, adequado à estrutura de informações relacionadas do projeto. Sua utilização permite estabelecer relacionamentos, chaves estrangeiras e restrições de integridade, contribuindo para evitar inconsistências e conflitos nos registros de horários.

A configuração estimada considera uma instância da família db.t4g.small, com funcionamento contínuo durante 730 horas mensais, armazenamento de 50 GB em SSD gp3 e implantação Multi-AZ.

A configuração Multi-AZ foi considerada devido ao requisito não funcional RNF006, que estabelece disponibilidade mensal mínima de 99,9%. A redundância entre zonas de disponibilidade contribui para a continuidade do serviço em situações de falha na infraestrutura principal.

O armazenamento foi dimensionado inicialmente em 50 GB, considerando que a versão atual do VittaCare não prevê gerenciamento completo de prontuários eletrônicos ou armazenamento extensivo de documentos médicos.

**Custo mensal estimado: US$ 121,91.**

#### *4.3.1.4 Amazon S3 — Armazenamento de arquivos*

O Amazon S3 foi escolhido para armazenar arquivos de maneira independente do servidor principal da aplicação. Sua utilização evita que arquivos ocupem diretamente o armazenamento das instâncias EC2 e permite ampliar a capacidade de armazenamento conforme a necessidade.

A estimativa considera 5 GB de armazenamento na classe S3 Standard, destinada a arquivos que precisam permanecer disponíveis para acesso frequente.

No escopo inicial do VittaCare, o armazenamento será limitado aos arquivos eventualmente necessários para o funcionamento da plataforma, uma vez que o gerenciamento completo de prontuários e documentos médicos não faz parte da versão proposta.

**Custo mensal estimado: US$ 0,20.**

#### *4.3.1.5 Amazon CloudWatch — Monitoramento*

O Amazon CloudWatch foi selecionado para monitorar a infraestrutura e acompanhar informações relacionadas ao funcionamento dos serviços utilizados pelo VittaCare.

O serviço permite coletar métricas, registrar eventos e acompanhar possíveis falhas ou comportamentos anormais. Dessa forma, auxilia na identificação de problemas de desempenho e disponibilidade, contribuindo para a manutenção preventiva da plataforma.

A estimativa foi configurada considerando métricas e solicitações de monitoramento, incluindo uma quantidade inicial de métricas, consultas e outras chamadas de API. Esses valores representam uma hipótese de utilização e poderão ser ajustados conforme a necessidade de observabilidade do sistema.

A escolha do CloudWatch também se justifica por sua integração direta com os demais serviços da AWS, simplificando o monitoramento da infraestrutura.

**Custo mensal estimado: US$ 6,03.**

#### *4.3.1.6 AWS Backup — Backup e recuperação de dados*

O AWS Backup foi considerado para centralizar e gerenciar políticas de proteção dos recursos utilizados pelo VittaCare.

A realização de cópias de segurança é importante porque o sistema armazenará dados pessoais e registros de agendamentos que precisam manter sua integridade e disponibilidade. Os backups permitem recuperar informações em situações de falha, exclusão acidental ou problemas na infraestrutura.

A estimativa inicial considera um volume de 50 GB de dados protegidos, com retenção diária de sete dias e sem utilização de armazenamento a frio. Essa configuração foi adotada como referência para uma política inicial de recuperação de dados.

É importante destacar que o Amazon RDS também disponibiliza mecanismos próprios de backup automatizado. Portanto, a utilização simultânea do AWS Backup e dos backups automatizados do RDS deverá ser avaliada para evitar sobreposição de recursos e custos desnecessários.

**Custo mensal estimado: US$ 0,33.**

#### *4.3.1.7 Amazon CloudFront — Distribuição de conteúdo*

O Amazon CloudFront foi selecionado para realizar a distribuição de conteúdos estáticos da plataforma, como imagens e arquivos armazenados no Amazon S3.

Por meio de uma rede de distribuição de conteúdo (CDN), o serviço permite disponibilizar arquivos aos usuários a partir de pontos de presença distribuídos geograficamente, contribuindo para reduzir a latência e melhorar o tempo de carregamento dos conteúdos.

Para a estimativa inicial, foram consideradas 500.000 solicitações HTTPS mensais. A transferência de dados de saída para a Internet foi informada como zero na calculadora, pois a estimativa de tráfego adotada permanece dentro da franquia gratuita indicada pela ferramenta.

O custo mensal apresentado foi de US$ 0,00, considerando as franquias gratuitas aplicáveis à configuração informada. Esse valor não significa que o serviço seja necessariamente gratuito em qualquer cenário: caso o volume de solicitações ou transferência ultrapasse os limites considerados, poderão existir cobranças adicionais.

**Custo mensal estimado: US$ 0,00.**

### 4.3.2 Resumo dos custos da infraestrutura

A tabela a seguir apresenta os custos mensais estimados para os serviços configurados na AWS Pricing Calculator.

| SERVIÇO | FINALIDADE | CUSTO MENSAL |
|---|---|---|
| **AMAZON EC2** | Hospedagem da aplicação | US$ 39,13 |
| **APPLICATION LOAD BALANCER** | Distribuição de requisições | US$ 82,64 |
| **AMAZON RDS** | Gerenciamento do banco de dados PostgreSQL | US$ 121,91 |
| **AMAZON S3** | Armazenamento de arquivos | US$ 0,20 |
| **AMAZON CLOUDWATCH** | Monitoramento da infraestrutura | US$ 6,03 |
| **AWS BACKUP** | Proteção e recuperação de dados | US$ 0,33 |
| **AMAZON CLOUDFRONT** | Distribuição de conteúdo | US$ 0,00 |
| **TOTAL ESTIMADO** | | **US$ 250,24** |

Considerando uma cotação de referência de R$ 5,5071 por dólar, em 06/11/2026, o custo mensal estimado da infraestrutura corresponde a aproximadamente **R$ 1.378,10**.

Os valores representam uma estimativa inicial, podendo variar conforme o consumo real, a região contratada, o volume de acessos e a necessidade de ampliação da infraestrutura. Também poderão existir custos adicionais relacionados à transferência de dados, endereços IP públicos e serviços complementares.

Os valores desta seção representam a equipe permanente da empresa, responsável por desenvolvimento, operação, manutenção e evolução do VittaCare, com provisão simplificada de encargos para os profissionais CLT. O custo de construção do MVP, com dedicação parcial de alguns perfis e período de

16 semanas, é detalhado no Bloco 7, que utiliza valores de referência da faixa salarial publicada no Glassdoor, sem encargos. Os dois blocos respondem a perguntas diferentes: o Bloco 1 sustenta a viabilidade financeira da operação mensal; o Bloco 7 estima o investimento para entregar a primeira versão.

### 4.3.2 Quanto custariam os profissionais?

A equipe de nove profissionais será mantida após o lançamento do VittaCare, compondo a estrutura permanente da empresa responsável pelo desenvolvimento, operação, manutenção e evolução da plataforma. Dessa forma, os custos com esses profissionais serão considerados despesas recorrentes da operação, e não apenas custos temporários da fase de desenvolvimento.

A estimativa mensal da equipe, considerando os valores diretos de remuneração e uma provisão simplificada para encargos e obrigações trabalhistas dos profissionais CLT, corresponde a R$ 81.641,92. Esse valor será utilizado como referência inicial para a projeção financeira, podendo sofrer ajustes conforme o regime tributário e os encargos efetivamente aplicáveis.

A composição financeira utiliza uma contratação mista, com profissionais em regime CLT e prestadores de serviços PJ. Os valores abaixo são estimativas mensais de remuneração direta, utilizadas para planejamento do projeto.

| Profissional | Quantidade | Modalidade | Valor mensal unitário | Total mensal |
|---|---|---|---|---|
| **Analista de Sistemas / PO** | 1 | CLT | R$ 9.000,00 | R$ 9.000,00 |
| **Arquiteto de Software** | 1 | PJ | R$ 9.000,00 | R$ 9.000,00 |
| **Desenvolvedor Front-end** | 1 | CLT | R$ 7.200,00 | R$ 7.200,00 |
| **Desenvolvedor Back-end** | 2 | CLT | R$ 7.800,00 | R$ 15.600,00 |
| **DBA / Engenheiro de Dados** | 1 | PJ | R$ 7.000,00 | R$ 7.000,00 |
| **Engenheiro DevOps** | 1 | CLT | R$ 8.750,00 | R$ 8.750,00 |
| **Analista de QA** | 1 | CLT | R$ 6.250,00 | R$ 6.250,00 |
| **UI/UX Designer** | 1 | PJ | R$ 6.000,00 | R$ 6.000,00 |

| | | | | |
|---|---|---|---|---|
| **Total** | **9** | | | **R$ 68.800,00** |

Os profissionais CLT representam R$ 46.800,00 em salários brutos mensais, enquanto os contratos PJ representam R$ 22.000,00 em pagamentos mensais.

Para estimar o custo total da equipe, foi aplicada provisoriamente uma provisão de 27,44% sobre os salários CLT, destinada a representar encargos e obrigações trabalhistas. Com essa estimativa, o custo mensal da equipe fica em R$ 81.641,92, sendo R$ 59.641,92 referentes aos profissionais CLT com provisão e R$ 22.000,00 referentes aos contratos PJ.

Essa provisão é uma aproximação para o estudo de viabilidade e deverá ser refinada conforme o regime tributário, os encargos efetivamente incidentes e a modalidade de contratação. Os valores também não representam uma cotação salarial definitiva.

Os valores desta seção representam a equipe permanente da empresa, responsável por desenvolvimento, operação, manutenção e evolução do VittaCare, com provisão simplificada de encargos para os profissionais CLT. O custo de construção do MVP, com dedicação parcial de alguns perfis e período de 16 semanas, é detalhado no Bloco 7, que utiliza valores de referência da faixa salarial publicada no Glassdoor, sem encargos. Os dois blocos respondem a perguntas diferentes: o Bloco 1 sustenta a viabilidade financeira da operação mensal; o Bloco 7 estima o investimento para entregar a primeira versão.

### 4.3.3 Cálculo utilizado e definição da mensalidade

O VittaCare adotará o modelo de assinatura SaaS, no qual as clínicas e ambulatórios contratam o acesso à plataforma mediante pagamento mensal.

Foi definida inicialmente a previsão de **20 clínicas contratantes**, todas com acesso a um único plano de assinatura. O plano disponibilizará as funcionalidades contempladas no escopo do projeto, respeitando as permissões de acesso de cada perfil de usuário.

Com base nos custos mensais estimados, foi realizado o seguinte cálculo:

| DESCRIÇÃO | VALOR MENSAL |
|---|---|
| **EQUIPE PROFISSIONAL** | R$ 81.641,92 |
| **INFRAESTRUTURA AWS** | R$ 1.378,10 |
| **CUSTO MENSAL ESTIMADO** | **R$ 83.020,02** |

Considerando a previsão inicial de 20 clínicas contratantes, foi realizado o seguinte cálculo:

R$ 83.020,02 ÷ 20 clínicas = **R$ 4.151,00 por clínica**.

Esse resultado representa o custo médio mensal estimado por cliente para cobrir os custos considerados na projeção financeira. Entretanto, não corresponde ao preço de venda recomendado, pois ainda não contempla integralmente tributos sobre o faturamento, despesas administrativas, eventuais serviços adicionais e margem de resultado da empresa.

Diante disso, foi mantida a proposta de mensalidade inicial de **R$ 6.000,00 por clínica**.

Com 20 clínicas contratantes, a projeção financeira seria:

| INDICADOR | VALOR |
|---|---|
| **MENSALIDADE POR CLÍNICA** | R$ 6.000,00 |
| **QUANTIDADE PREVISTA DE CLÍNICAS** | 20 |
| **RECEITA MENSAL ESTIMADA** | R$ 120.000,00 |
| **CUSTOS MENSAIS ESTIMADOS** | R$ 83.020,02 |
| **DIFERENÇA MENSAL ESTIMADA** | **R$ 36.979,98** |

A diferença apresentada representa o saldo antes da dedução de tributos sobre a receita e de outras despesas ainda não contempladas no levantamento. Portanto, não deve ser interpretada como lucro líquido.

#### *4.3.3.1 Justificativa executiva da mensalidade*

A mensalidade de R$ 6.000,00 foi proposta considerando a necessidade de manter uma estrutura profissional permanente e uma infraestrutura tecnológica capaz de sustentar a operação do VittaCare. Diferentemente de um projeto desenvolvido apenas para entrega pontual, a plataforma SaaS exige continuidade operacional, incluindo manutenção, segurança, monitoramento, suporte e evolução dos serviços.

A adoção de um plano único também favorece a simplicidade comercial e operacional, pois todas as clínicas contratantes terão acesso ao mesmo conjunto de funcionalidades, sem a necessidade de administrar diferentes categorias de assinatura. Essa estratégia facilita a comunicação do produto, a padronização do atendimento e a previsibilidade da receita recorrente.

A previsão de 20 clínicas contratantes permite estabelecer uma primeira referência de faturamento e avaliar a capacidade de sustentação da operação. A mensalidade proposta fica acima do custo médio estimado por cliente, criando uma margem para absorver despesas que ainda precisam ser detalhadas e contribuir para a sustentabilidade financeira da empresa.

Ressalta-se que o valor de R$ 6.000,00 constitui uma **proposta comercial preliminar**, definida a partir da estrutura de custos e da quantidade inicial de clientes projetada. Sua validação definitiva dependerá de pesquisa de mercado, análise da disposição de pagamento das clínicas e revisão dos custos operacionais e tributários.

Dessa forma, a proposta busca equilibrar a capacidade de pagamento dos clientes, a manutenção da qualidade do serviço e a sustentabilidade financeira do VittaCare.

### 4.3.4 CÁLCULO DO PONTO DE EQUILÍBRIO

**Ponto de equilíbrio financeiro**

O ponto de equilíbrio representa a quantidade mínima de clínicas contratantes necessária para que a receita mensal cubra os custos considerados na operação do VittaCare.

Considerando o custo mensal estimado de R$ 83.020,02 e a mensalidade de R$ 6.000,00 por clínica, obtém-se o seguinte cálculo:

R$ 83.020,02 ÷ R$ 6.000,00 = 13,84 clínicas.

Como não é possível contratar uma fração de clínica, o resultado deve ser arredondado para cima. Portanto, o VittaCare precisará de **14 clínicas contratantes para atingir o ponto de equilíbrio financeiro**.

Com 14 clínicas, a receita mensal estimada será de R$ 84.000,00, enquanto os custos considerados serão de R$ 83.020,02, resultando em uma diferença positiva de R$ 979,98.

Esse resultado representa apenas a cobertura dos custos incluídos na estimativa, não correspondendo ao lucro líquido da empresa, uma vez que ainda existem tributos e outras despesas que poderão impactar o resultado financeiro.

## 4.4 Viabilidade legal e regulatória

*Lei nº 13.709/2018 – LGPD*

Proteção de dados pessoais e dados sensíveis de saúde dos pacientes e profissionais.

*Lei nº 12.965/2014 – Marco Civil da Internet*

Estabelece direitos e deveres relacionados ao uso da internet, privacidade e proteção de dados.

Decreto nº 8.771/2016

Regulamenta aspectos do Marco Civil da Internet, incluindo proteção e segurança dos registros e dados pessoais.

*Lei nº 8.078/1990 – Código de Defesa do Consumidor*

Pode ser aplicável às relações de consumo envolvendo a contratação e utilização dos serviços da plataforma.

*Lei nº 13.787/2018*

Estabelece regras para digitalização e utilização de sistemas informatizados para guarda e manuseio de prontuários médicos. Sua aplicação direta dependerá da inclusão dessas funcionalidades no sistema. Como o VittaCare não prevê gerenciamento completo de prontuários eletrônicos no escopo inicial, não devemos afirmar que todas as suas disposições se aplicam diretamente às funcionalidades atuais. Ela deverá ser considerada caso o escopo seja ampliado.

Lei nº 13.146/2015 – Lei Brasileira de Inclusão

Estabelece requisitos de acessibilidade, relevantes para as interfaces utilizadas por pessoas com deficiência.

Além das legislações, o VittaCare deverá observar algumas medidas:

*Proteção de dados e privacidade*

- Implementação de controle de acesso por perfil.
- Criptografia das informações armazenadas e transmitidas.
- Política de privacidade.
- Definição de finalidade para o tratamento dos dados.
- Procedimentos para comunicação de incidentes de segurança.
- Definição das responsabilidades entre clínicas e fornecedores.

*Segurança e rastreabilidade*

- Registro de operações relevantes.

- Controle de acessos e alterações.
- Políticas de backup e recuperação.
- Monitoramento de atividades suspeitas.

*Acessibilidade*

- Interfaces compatíveis com tecnologias assistivas.
- Contraste adequado entre textos e elementos.
- Navegação por teclado.
- Identificação clara dos elementos da interface.

### 4.4.1 Conclusão Viabilidade Legal

Em relação à viabilidade legal, o VittaCare apresenta condições para ser desenvolvido e comercializado, desde que sejam observadas as normas aplicáveis à proteção de dados pessoais e sensíveis, especialmente a Lei Geral de Proteção de Dados Pessoais (LGPD).

Como a plataforma realizará o tratamento de informações relacionadas à saúde, será necessário implementar mecanismos adequados de controle de acesso, autenticação, proteção das informações, registro de operações e definição das finalidades de tratamento dos dados.

Também deverão ser observadas as demais normas aplicáveis aos serviços digitais e às relações de consumo, conforme a natureza das operações realizadas pela plataforma.

Considerando que a versão inicial do VittaCare não contempla o gerenciamento completo de prontuários eletrônicos, a aplicação direta das disposições específicas sobre digitalização e armazenamento de prontuários deverá ser avaliada caso esse escopo seja ampliado futuramente.

Dessa forma, conclui-se que o projeto apresenta **viabilidade legal preliminar**, condicionada à implementação das medidas de segurança, privacidade e conformidade normativa necessárias antes de sua disponibilização comercial.

## 4.5 Conclusão geral da viabilidade

Com base nas análises realizadas, conclui-se que o VittaCare apresenta **viabilidade técnica, operacional e financeira preliminar**, desde que sejam observadas as condições necessárias para sua implantação e continuidade.

Do ponto de vista técnico, o projeto utiliza tecnologias consolidadas e uma arquitetura monolítica modular, considerada adequada ao escopo inicial da plataforma. A equipe multidisciplinar definida contempla as competências necessárias para o desenvolvimento, implantação e manutenção do sistema. Entretanto, será necessário concluir o planejamento técnico, incluindo a documentação da arquitetura, modelagem do banco de dados, definição das APIs, implementação dos requisitos de segurança e planejamento dos testes.

Em relação à viabilidade operacional, a plataforma apresenta potencial para melhorar a organização dos processos de clínicas e ambulatórios, centralizando agendamentos, facilitando o gerenciamento das agendas profissionais e reduzindo atividades administrativas manuais. Sua adoção dependerá da preparação dos usuários, da realização de treinamentos e da adaptação dos processos internos das instituições contratantes.

Quanto à viabilidade financeira, foi proposto o modelo SaaS, com um único plano de assinatura no valor mensal de **R$ 6.000,00 por clínica**. Considerando a previsão inicial de 20 clínicas contratantes, estima-se uma receita mensal de R$ 120.000,00, diante de um custo mensal preliminar de R$ 83.020,02, contemplando a equipe profissional e a infraestrutura tecnológica estimadas. Essa projeção indica potencial de sustentabilidade financeira, embora ainda dependa da validação dos custos tributários, administrativos e operacionais, além da aceitação do preço pelo mercado.

Também deverão ser consideradas as exigências legais e regulatórias relacionadas à proteção de dados pessoais e sensíveis, especialmente por se tratar de uma plataforma que lida com informações relacionadas à saúde. A implementação de mecanismos de segurança, controle de acesso, privacidade e rastreabilidade será fundamental para a operação adequada do sistema.

Dessa forma, o desenvolvimento do VittaCare é considerado **viável em caráter preliminar**, apresentando uma proposta técnica compatível com seu escopo, benefícios operacionais relevantes para o público-alvo e um modelo de negócio com potencial de geração de receita recorrente. A viabilidade definitiva dependerá da conclusão do planejamento, da validação das estimativas financeiras, da adequação às exigências legais e da conquista de uma quantidade de clientes suficiente para sustentar a operação.

# 5. MATRIZ DE RISCOS INICIAIS

## 5.1 Risco 1 — Vazamento de dados pessoais e sensíveis

**Categoria:** Segurança e privacidade.

**Descrição:** Possibilidade de acesso não autorizado ou exposição de informações pessoais dos pacientes e profissionais devido a falhas de segurança, permissões inadequadas ou vulnerabilidades na aplicação.

**Probabilidade:** Média.

**Impacto:** Alto.

**Plano de mitigação:**

- Implementar autenticação segura.
- Adotar controle de acesso baseado em papéis (RBAC).
- Utilizar criptografia na transmissão e no armazenamento de informações.
- Registrar operações relevantes.
- Realizar testes de segurança.
- Aplicar os princípios de proteção de dados desde a concepção do sistema.

## 5.2 Risco 2 — Conflitos e duplicidade de agendamentos

**Categoria:** Técnico e operacional.

**Descrição:** Possibilidade de dois ou mais pacientes conseguirem reservar o mesmo horário de um profissional devido a falhas na validação de disponibilidade ou concorrência de requisições.

**Probabilidade:** Média.

**Impacto:** Alto.

**Plano de mitigação:**

- Implementar validação de disponibilidade no backend.
- Utilizar transações no banco de dados.
- Criar restrições de integridade para impedir conflitos.
- Realizar testes de concorrência.
- Atualizar a disponibilidade após confirmações e cancelamentos.

## 5.3 Risco 3 — Indisponibilidade da plataforma

**Categoria:** Técnico.

**Descrição:** Interrupção do funcionamento do sistema devido a falhas na infraestrutura, problemas de conexão, indisponibilidade do banco de dados ou erros na aplicação.

**Probabilidade:** Média.

**Impacto:** Alto.

**Plano de mitigação:**

- Contratar serviços de infraestrutura confiáveis.
- Implementar monitoramento.
- Realizar backups periódicos.
- Definir procedimentos de recuperação.
- Configurar alertas para falhas críticas.
- Documentar procedimentos de contingência.

## 5.4 Risco 4 — Resistência dos usuários à adoção do sistema

**Categoria:** Operacional e negócio.

**Descrição:** Possibilidade de pacientes, médicos e funcionários apresentarem dificuldades ou resistência à utilização da plataforma, mantendo processos manuais anteriores.

**Probabilidade:** Média.

**Impacto:** Médio.

**Plano de mitigação:**

- Desenvolver interfaces intuitivas.
- Realizar treinamento dos funcionários.
- Disponibilizar documentação de utilização.
- Realizar testes de usabilidade.
- Implantar o sistema gradualmente.
- Coletar feedback dos usuários.

## 5.5 Risco 5 — Aumento inesperado dos custos de infraestrutura

**Categoria:** Financeiro.

**Descrição:** Possibilidade de os custos de hospedagem e manutenção ultrapassarem o orçamento previsto devido ao aumento da utilização, crescimento do armazenamento ou contratação de recursos adicionais.

**Probabilidade:** Média.

**Impacto:** Médio.

**Plano de mitigação:**

- Estimar os custos antes da implantação.
- Monitorar o consumo dos serviços contratados.
- Configurar alertas de orçamento.
- Dimensionar os recursos conforme a demanda.
- Avaliar alternativas de infraestrutura.
- Realizar revisões periódicas dos custos.

## 5.6 Resumo da matriz de riscos

| ID | Risco | Probabilidade | Impacto |
|---|---|---|---|
| **R01** | Vazamento de dados | Média | Alto |
| **R02** | Conflitos de agendamento | Média | Alto |
| **R03** | Indisponibilidade do sistema | Média | Alto |
| **R04** | Resistência dos usuários | Média | Médio |
| **R05** | Aumento dos custos | Média | Médio |

## 5.7 Considerações finais

Os riscos identificados representam situações que podem comprometer a segurança, a operação e a continuidade do VittaCare.

A identificação antecipada desses riscos permite estabelecer medidas preventivas e estratégias de mitigação, reduzindo a possibilidade de impactos negativos durante o desenvolvimento e a utilização do sistema.

# VittaCare

Requisitos – Bloco 2

# Sumário

1. REQUISITOS DO SISTEMA ..... 3
1.1 Introdução ..... 3
1.2 Requisitos Funcionais (RF) ..... 3
1.2.1 Método de priorização MoSCoW ..... 3
PRIORIDADE CLASSIFICAÇÃO DESCRIÇÃO ..... 4
1.2.2 Especificação dos requisitos funcionais ..... 4
1.3 Funcionalidades fora da versão atual ..... 7
1.3.1 Requisitos Não Funcionais (RNF) ..... 7
2.Histórias do usuário ..... 9
Justificativa das métricas dos requisitos não funcionais ..... 9
2.1 Histórias de Usuário — Paciente ..... 10
2.2 Histórias de Usuário — Profissional de Saúde ..... 13
2.3 Histórias de Usuário — Recepcionista ..... 14
2.4 Histórias de Usuário — Administrador ..... 15
2.5 Resumo das histórias de usuário ..... 17
2.6 Considerações finais ..... 18

# 1. REQUISITOS DO SISTEMA

## 1.1 Introdução

Esta etapa tem como objetivo definir os requisitos funcionais e não funcionais do VittaCare, estabelecendo as funcionalidades que deverão ser disponibilizadas pela plataforma e as condições necessárias para seu funcionamento.

Os requisitos foram elaborados com base no Documento de Visão e Escopo do projeto, considerando os quatro perfis principais de usuários: pacientes, profissionais de saúde, recepcionistas e administradores.

A especificação dos requisitos permitirá orientar as próximas etapas do projeto, servindo como referência para a definição da arquitetura de software, elaboração dos diagramas UML, modelagem do banco de dados e planejamento do desenvolvimento.

O VittaCare será disponibilizado comercialmente no modelo SaaS, por meio de um plano único de assinatura destinado a clínicas e ambulatórios. Os usuários da plataforma terão acesso às funcionalidades conforme seus perfis e permissões.

**Os requisitos estão organizados em três categorias:**

1. **Requisitos Funcionais (RF):** descrevem as funcionalidades e operações que o sistema deverá oferecer.
2. **Requisitos Não Funcionais (RNF):** estabelecem critérios de qualidade, desempenho, segurança, disponibilidade e usabilidade.
3. **Histórias de Usuário (US):** descrevem as necessidades dos usuários sob suas respectivas perspectivas, acompanhadas de critérios de aceitação e estimativas de esforço.

---

## 1.2 Requisitos Funcionais (RF)

Os requisitos funcionais descrevem as operações que o VittaCare deverá executar, especificando os serviços e funcionalidades disponibilizados aos seus usuários.

Cada requisito recebe um código de identificação, uma descrição, uma prioridade definida pela técnica MoSCoW e uma justificativa para a classificação atribuída.

### 1.2.1 Método de priorização MoSCoW

Para estabelecer a importância de cada funcionalidade, será utilizada a técnica MoSCoW, que divide os requisitos em quatro níveis de prioridade:

| PRIORIDADE | CLASSIFICAÇÃO | DESCRIÇÃO |
|---|---|---|
| **MUST HAVE** | Obrigatório | Funcionalidade indispensável para o funcionamento do sistema. |
| **SHOULD HAVE** | Importante | Funcionalidade relevante, cuja ausência não impede as operações essenciais. |
| **COULD HAVE** | Desejável | Funcionalidade complementar, que poderá ser implementada posteriormente. |
| **WON'T HAVE** | Fora da versão atual | Funcionalidade que não será desenvolvida na versão atual do projeto. |

### 1.2.2 Especificação dos requisitos funcionais

*Autenticação e gerenciamento de usuários*

| CÓDIGO | DESCRIÇÃO | PRIORIDADE |
|---|---|---|
| **RF001** | O sistema deve permitir que pacientes, profissionais de saúde, recepcionistas e administradores realizem autenticação por meio de e-mail e senha. | Must |
| **RF002** | O sistema deve permitir que os usuários recuperem suas senhas por meio de um mecanismo de redefinição. | Must |
| **RF003** | O sistema deve permitir que administradores cadastrem, ativem e desativem contas de usuários. | Must |
| **RF004** | O sistema deve permitir que usuários autorizados visualizem e atualizem suas informações cadastrais. | Should |

*Cadastro e gerenciamento de informações*

| CÓDIGO | DESCRIÇÃO | PRIORIDADE |
|---|---|---|
| **RF005** | O sistema deve permitir o cadastro de pacientes, tanto por meio de autorregistro na plataforma quanto por funcionários autorizados durante o atendimento presencial. | Must |
| **RF006** | O sistema deve permitir que funcionários autorizados consultem e atualizem os cadastros dos pacientes. | Must |
| **RF007** | O sistema deve permitir o cadastro e gerenciamento de profissionais de saúde. | Must |
| **RF008** | O sistema deve permitir que administradores cadastrem, atualizem e desativem especialidades médicas. | Must |
| **RF009** | O sistema deve permitir a associação de profissionais de saúde às suas respectivas especialidades. | Must |

*Pesquisa e disponibilidade*

| CÓDIGO | DESCRIÇÃO | PRIORIDADE |
|---|---|---|
| **RF010** | O sistema deve permitir que pacientes consultem as especialidades disponíveis. | Must |

| | | |
|---|---|---|
| **RF011** | O sistema deve permitir a pesquisa de profissionais por nome e especialidade. | Must |
| **RF012** | O sistema deve permitir a visualização dos horários disponíveis de cada profissional. | Must |
| **RF013** | O sistema deve permitir que profissionais cadastrem, atualizem e bloqueiem seus horários de disponibilidade. | Must |

*Gerenciamento de agendamentos*

| CÓDIGO | DESCRIÇÃO | PRIORIDADE |
|---|---|---|
| **RF014** | O sistema deve permitir que pacientes realizem agendamentos de consultas nos horários disponíveis. | Must |
| **RF015** | O sistema deve permitir que recepcionistas realizem agendamentos em nome dos pacientes. | Must |
| **RF016** | O sistema deve permitir o cancelamento de consultas pelos usuários autorizados. | Must |
| **RF017** | O sistema deve permitir a remarcação de consultas mediante a seleção de outro horário disponível. | Must |
| **RF018** | O sistema deve impedir a confirmação de dois agendamentos para o mesmo profissional no mesmo horário. | Must |
| **RF019** | O sistema deve permitir que pacientes e profissionais consultem seus respectivos históricos de agendamentos. | Should |
| **RF020** | O sistema deve permitir que profissionais de saúde e funcionários da recepção devidamente autorizados registrem o comparecimento ou a ausência dos pacientes nas consultas agendadas. | Should |

*Notificações e administração*

| CÓDIGO | DESCRIÇÃO | PRIORIDADE |
|---|---|---|
| **RF021** | O sistema deve enviar notificações de confirmação após a realização de um agendamento. | Should |
| **RF022** | O sistema deve enviar lembretes aos pacientes sobre consultas próximas. | Should |
| **RF023** | O sistema deve disponibilizar relatórios administrativos sobre agendamentos, cancelamentos e ausências. | Could |
| **RF024** | O sistema deve permitir que administradores configurem e gerenciem as permissões de acesso dos diferentes perfis de usuários. | Must |
| **RF025** | O sistema deve garantir que cada clínica tenha acesso somente aos seus próprios dados, pacientes, profissionais, agendas e agendamentos, respeitando as permissões dos usuários vinculados à instituição. | Must |

| | | |
|---|---|---|
| **RF026** | O sistema deverá verificar a existência de agendamentos antes de permitir que um profissional bloqueie determinado horário. Caso exista uma consulta confirmada no período selecionado, o sistema deverá impedir o bloqueio e informar que há um agendamento associado àquele horário. | Must |

### 1.2.2.1 Justificativa das prioridades

A classificação dos requisitos considera a finalidade principal do VittaCare e as necessidades identificadas durante a concepção do projeto.

Os requisitos classificados como Must Have representam as funcionalidades indispensáveis para o funcionamento da plataforma, incluindo autenticação, gerenciamento de cadastros, pesquisa de profissionais, gerenciamento de disponibilidade e realização de agendamentos.

Os requisitos classificados como Should Have representam funcionalidades importantes para melhorar a experiência dos usuários e a organização dos atendimentos, como atualização de informações cadastrais, consulta de históricos e registro de comparecimentos.

O requisito RF023 foi classificado como Could Have, pois os relatórios administrativos oferecem informações relevantes para a gestão da clínica, mas não são indispensáveis para a execução das operações principais do sistema.

Conforme estabelecido no Documento de Visão e Escopo, algumas funcionalidades não fazem parte da versão inicial do VittaCare.

| CÓDIGO | FUNCIONALIDADE | PRIORIDADE |
|---|---|---|
| **RF-W01** | Processamento de pagamentos de consultas. | Won't |
| **RF-W02** | Realização de consultas por videoconferência. | Won't |
| **RF-W03** | Diagnóstico médico automatizado. | Won't |
| **RF-W04** | Prescrição eletrônica de medicamentos. | Won't |
| **RF-W05** | Gerenciamento completo de prontuários eletrônicos. | Won't |
| **RF-W06** | Integração com equipamentos médicos. | Won't |
| **RF-W07** | Inteligência artificial para análise clínica. | Won't |

Essas funcionalidades foram excluídas para manter o foco do desenvolvimento nos processos de agendamento e organização administrativa das clínicas e ambulatórios.

## 1.3 Funcionalidades fora da versão atual

### 1.3.1 Requisitos Não Funcionais (RNF)

Os requisitos não funcionais estabelecem os critérios de qualidade e as condições técnicas que deverão ser atendidas pelo VittaCare.

Diferentemente dos requisitos funcionais, que descrevem as operações realizadas pelo sistema, os requisitos não funcionais determinam como essas operações deverão ser executadas.

Para permitir a verificação do cumprimento dos requisitos, foram estabelecidas métricas quantitativas e critérios objetivos de avaliação.

| CÓDIGO | DESCRIÇÃO |
|---|---|
| **RNF001** | O sistema deverá responder a pelo menos 95% das requisições comuns de consulta e navegação em até 300 milissegundos (p95), considerando condições normais de operação. Esse limite não se aplica às operações especificadas individualmente nos requisitos RNF002 e RNF003. |
| **RNF002** | As operações de pesquisa de profissionais e consulta de horários disponíveis deverão apresentar tempo de resposta inferior a 2 segundos em pelo menos 95% das requisições (p95). |
| **RNF003** | O processamento de um agendamento, incluindo a validação da disponibilidade, verificação de conflitos e registro no banco de dados, deverá ser concluído em até 2 segundos em pelo menos 95% das operações (p95). |

| CÓDIGO | DESCRIÇÃO |
|---|---|
| **RNF004** | A infraestrutura deverá suportar até 10.000 usuários cadastrados, considerando um cenário de pico de até 1.000 usuários simultâneos, mantendo os limites de desempenho estabelecidos nos requisitos RNF001, RNF002 e RNF003. |
| **RNF005** | A infraestrutura deverá permitir o aumento de capacidade computacional do servidor de aplicação, com ampliação de CPU e memória, sem exigir alterações no código-fonte da aplicação. |

| CÓDIGO | DESCRIÇÃO |
|---|---|
| **RNF006** | O sistema deverá apresentar disponibilidade mensal mínima de 99,9%, considerando o período de operação e excluindo manutenções programadas previamente comunicadas. |

| | |
|---|---|
| **RNF007** | O banco de dados deverá possuir mecanismos automatizados de backup, com suporte à recuperação das informações em situações de falha, exclusão acidental ou indisponibilidade da infraestrutura.<br><br>A política de proteção deverá estabelecer os seguintes parâmetros:<br>• **RPO:** o intervalo máximo aceitável de perda de dados deverá ser de até 15 minutos.<br>• **RTO:** o tempo máximo para recuperação do serviço deverá ser de até 4 horas.<br>• **Retenção diária:** as cópias de segurança deverão ser mantidas por pelo menos 7 dias.<br>• **Retenção semanal:** deverá ser mantida uma cópia semanal por pelo menos 4 semanas. |
| **RNF008** | O sistema deverá garantir a integridade dos agendamentos, impedindo registros incompletos e conflitos de horários mesmo diante de falhas durante as operações. |

| CÓDIGO | DESCRIÇÃO |
|---|---|
| **RNF009** | Todas as comunicações entre cliente e servidor deverão utilizar HTTPS com TLS 1.2 ou superior. |
| **RNF010** | 100% dos endpoints responsáveis pelo acesso a informações restritas deverão exigir autenticação e autorização conforme o perfil do usuário. |
| **RNF011** | As senhas dos usuários deverão ser armazenadas utilizando algoritmos próprios para hash de senhas, como Argon2id ou bcrypt, sendo proibido o armazenamento em texto puro. |
| **RNF012** | 100% dos campos classificados como dados pessoais ou sensíveis deverão possuir finalidade de tratamento, classificação e regras de acesso documentadas no dicionário de dados. |
| **RNF013** | O sistema deverá registrar 100% das operações administrativas críticas, incluindo alterações de permissões, cadastros e agendamentos, mantendo informações sobre o usuário responsável e o momento da operação. |
| **RNF014** | A arquitetura deverá garantir a segregação lógica dos dados de cada clínica, impedindo que usuários de uma instituição acessem informações pertencentes a outra, inclusive em operações de consulta, alteração e exclusão. |

| CÓDIGO | DESCRIÇÃO |
|---|---|
| **RNF015** | As principais operações dos pacientes, incluindo pesquisa de profissionais, consulta de horários e realização de agendamentos, deverão ser concluídas em no máximo 5 etapas de interação cada. |

| | |
|---|---|
| **RNF016** | A plataforma deverá ser compatível com as versões estáveis mais recentes dos navegadores Google Chrome, Mozilla Firefox e Microsoft Edge. |
| **RNF017** | 100% das funcionalidades principais deverão permitir navegação por teclado e apresentar identificação textual dos elementos interativos. |

Os requisitos de desempenho foram definidos para garantir que as operações mais frequentes, principalmente pesquisas e agendamentos, sejam executadas sem atrasos significativos.

Os requisitos de escalabilidade consideram a possibilidade de crescimento da quantidade de usuários e da utilização da plataforma, permitindo ampliar os recursos computacionais conforme a demanda.

Os requisitos de disponibilidade e confiabilidade buscam garantir a continuidade das operações e a integridade das informações, especialmente dos registros de agendamentos.

Os requisitos de segurança foram definidos considerando que o VittaCare armazenará dados pessoais e poderá tratar dados sensíveis relacionados à saúde. Dessa forma, são necessários mecanismos de autenticação, autorização, proteção das informações e rastreabilidade das operações, observando a Lei Geral de Proteção de Dados (LGPD).

Por fim, os requisitos de usabilidade e acessibilidade buscam garantir que os diferentes perfis de usuários consigam utilizar a plataforma de maneira simples, compreensível e acessível.

---

# 2.Histórias do usuário

As histórias de usuário representam as necessidades dos diferentes perfis que utilizarão o VittaCare, descrevendo as funcionalidades a partir da perspectiva de cada usuário.

Cada história possui critérios de aceitação escritos no formato Given/When/Then, utilizados para estabelecer as condições necessárias para considerar uma funcionalidade corretamente implementada.

Também foram atribuídos Story Points, que representam estimativas relativas de esforço e complexidade. Esses valores serão utilizados posteriormente no planejamento das sprints.

### Justificativa das métricas dos requisitos não funcionais

- RNF001 a RNF003 (300 ms e 2 s): consultas e navegação comuns devem parecer instantâneas ao usuário; pesquisa e agendamento envolvem mais processamento (validação, verificação de conflito e gravação) e por isso

admitem até 2 segundos. Utiliza-se o percentil 95 para ignorar picos isolados sem esconder a experiência da maioria.

- RNF004 (10.000 usuários cadastrados e 1.000 simultâneos): estimativa a partir da previsão de 20 clínicas contratantes, considerando seus pacientes, profissionais e funcionários, com pico concentrado em horário comercial.

- RNF006 (99,9%): corresponde a cerca de 43 minutos de indisponibilidade por mês. O agendamento é uma operação diária da clínica, e a indisponibilidade impede o atendimento na recepção.

- RNF007 (RPO de 15 minutos e RTO de 4 horas): perder mais que alguns minutos de agendamentos gera conflitos reais de horário; quatro horas é uma janela de recuperação aceitável para clínicas sem plantão 24 horas.

- RNF009 a RNF014: dados de saúde exigem criptografia em trânsito, controle de acesso, hash de senha, rastreabilidade das operações críticas e isolamento entre clínicas, conforme a LGPD.

- RNF015 a RNF017: o limite de 5 etapas mede a simplicidade das principais operações do paciente; Chrome, Firefox e Edge cobrem os navegadores de uso corrente; a navegação por teclado atende aos requisitos de acessibilidade da Lei Brasileira de Inclusão.

## 2.1 Histórias de Usuário — Paciente

Como paciente, quero realizar meu cadastro na plataforma, para poder acessar os serviços disponibilizados pelo VittaCare.

**Critérios de aceitação:**

**Given:** o usuário está na tela de cadastro.

**When:** informa os dados obrigatórios e confirma o cadastro.

**Then:** o sistema valida as informações e registra o novo paciente.

Story Points: 3.

Como paciente, quero entrar na plataforma utilizando meu e-mail e senha, para acessar minhas funcionalidades pessoais.

**Critérios de aceitação:**

**Given:** o paciente possui cadastro ativo.

**When:** informa credenciais válidas.

**Then:** o sistema autentica o usuário e disponibiliza as funcionalidades permitidas ao seu perfil.

Story Points: 3.

Como paciente, quero recuperar minha senha, para voltar a acessar minha conta caso a esqueça.

**Critérios de aceitação:**

**Given:** o paciente está na tela de recuperação de senha.

**When:** informa um e-mail cadastrado e solicita a recuperação.

**Then:** o sistema inicia o processo seguro de redefinição de senha.

Story Points: 5.

Como paciente, quero consultar as especialidades disponíveis, para encontrar o tipo de atendimento que necessito.

**Critérios de aceitação:**

**Given:** o paciente acessa a área de pesquisa.

**When:** solicita a visualização das especialidades.

**Then:** o sistema apresenta as especialidades cadastradas e disponíveis.

Story Points: 2.

Como paciente, quero pesquisar profissionais por nome ou especialidade, para encontrar o profissional desejado.

**Critérios de aceitação:**

**Given:** existem profissionais cadastrados.

**When:** o paciente informa um nome ou seleciona uma especialidade.

**Then:** o sistema apresenta os profissionais correspondentes à pesquisa.

Story Points: 3.

Como paciente, quero visualizar os horários disponíveis dos profissionais, para escolher o melhor horário para minha consulta.

**Critérios de aceitação:**

**Given:** o paciente selecionou um profissional.

**When:** solicita a visualização da agenda.

**Then:** o sistema apresenta os horários disponíveis e atualizados.

Story Points: 5.

Como paciente, quero agendar uma consulta em um horário disponível, para garantir meu atendimento.

**Critérios de aceitação:**

**Given:** o paciente está autenticado e existe um horário disponível.

**When:** seleciona o horário e confirma o agendamento.

**Then:** o sistema registra a consulta e atualiza a disponibilidade do profissional.

Story Points: 8.

Como paciente, quero cancelar uma consulta agendada, para liberar o horário caso não possa comparecer.

**Critérios de aceitação:**

**Given:** o paciente possui uma consulta agendada.

**When:** solicita o cancelamento e confirma a operação.

**Then:** o sistema atualiza o status da consulta e disponibiliza novamente o horário, conforme as regras de cancelamento.

Story Points: 3.

Como paciente, quero consultar meu histórico de agendamentos, para acompanhar minhas consultas anteriores e futuras.

**Critérios de aceitação:**

**Given:** o paciente está autenticado e possui registros de consultas.

**When:** acessa a área de histórico.

**Then:** o sistema apresenta somente os agendamentos associados à sua conta.

Story Points: 3.

Como paciente, quero receber lembretes sobre minhas consultas próximas, para evitar esquecimentos.

**Critérios de aceitação:**

**Given:** o paciente possui uma consulta confirmada e um lembrete programado.

**When:** chega o momento definido para o envio.

**Then:** o sistema envia uma notificação com as informações da consulta.

Story Points: 5.

Como paciente, quero remarcar uma consulta para outro horário disponível, para adaptar meu atendimento a mudanças na minha rotina.

**Critérios de aceitação:**

**Given:** o paciente possui uma consulta agendada.

**When:** seleciona a opção de remarcação e escolhe outro horário disponível.

**Then:** o sistema atualiza o agendamento, libera o horário anterior e registra o novo horário, sem gerar consultas duplicadas.

Story Points: 5.

## 2.2 Histórias de Usuário — Profissional de Saúde

Como profissional de saúde, quero cadastrar e atualizar meus horários disponíveis, para controlar os períodos em que posso realizar atendimentos.

**Critérios de aceitação:**

**Given:** o profissional está autenticado.

**When:** cadastra, altera ou bloqueia um horário.

**Then:** o sistema atualiza sua disponibilidade e impede novos agendamentos em horários bloqueados.

Story Points: 5.

Como profissional de saúde, quero visualizar minha agenda diária e semanal, para organizar meus atendimentos.

**Critérios de aceitação:**

**Given:** o profissional possui consultas registradas.

**When:** acessa sua agenda e seleciona um período.

**Then:** o sistema apresenta os atendimentos correspondentes ao período selecionado.

Story Points: 3.

Como profissional de saúde, quero registrar o comparecimento ou a ausência dos pacientes, para manter atualizado o histórico dos atendimentos.

**Critérios de aceitação:**

**Given:** existe uma consulta agendada.

**When:** o profissional registra o comparecimento ou a ausência.

**Then:** o sistema atualiza o status da consulta e mantém o registro associado ao atendimento.

Story Points: 3.

## 2.3 Histórias de Usuário — Recepcionista

Como recepcionista, quero realizar agendamentos em nome dos pacientes, para auxiliar pessoas que não conseguem utilizar a plataforma diretamente.

**Critérios de aceitação:**

**Given:** a recepcionista está autenticada e o paciente possui cadastro.

**When:** seleciona o paciente, o profissional e um horário disponível.

**Then:** o sistema registra o agendamento e atualiza a agenda do profissional.

Story Points: 5.

Como recepcionista, quero consultar e atualizar os cadastros dos pacientes, para manter as informações administrativas organizadas.

**Critérios de aceitação:**

**Given:** a recepcionista possui permissão para gerenciar cadastros.

**When:** pesquisa um paciente e solicita a atualização de seus dados.

**Then:** o sistema apresenta as informações autorizadas e salva as alterações válidas.

Story Points: 3.

Como recepcionista, quero registrar o comparecimento ou a ausência dos pacientes nas consultas agendadas, para manter o controle administrativo dos atendimentos atualizado.

**Critérios de aceitação:**

**Given:** a recepcionista está autenticada e possui permissão para registrar o comparecimento dos pacientes.

**When:** acessa uma consulta agendada e registra o comparecimento ou a ausência do paciente.

**Then:** o sistema atualiza o status da consulta e registra quem realizou a alteração.

Story Points: 3.

## 2.4 Histórias de Usuário — Administrador

Como administrador, quero cadastrar, atualizar e gerenciar profissionais de saúde, para manter a equipe da clínica organizada na plataforma.

**Critérios de aceitação:**

**Given:** o administrador está autenticado e possui permissão.

**When:** cadastra ou atualiza os dados de um profissional.

**Then:** o sistema valida e registra as informações.

Story Points: 3.

Como administrador, quero definir os níveis de acesso dos usuários, para garantir que cada perfil utilize somente as funcionalidades autorizadas.

**Critérios de aceitação:**

**Given:** o administrador possui permissão para gerenciar acessos.

**When:** altera o perfil ou as permissões de um usuário.

**Then:** o sistema atualiza as permissões e aplica as novas regras de acesso.

Story Points: 5.

Como administrador, quero visualizar relatórios de agendamentos, cancelamentos e ausências, para acompanhar o funcionamento da clínica.

**Critérios de aceitação:**

**Given:** existem registros de consultas no sistema.

**When:** o administrador seleciona o período e solicita um relatório.

**Then:** o sistema apresenta os dados correspondentes ao período escolhido.

Story Points: 5.

Como administrador, quero cadastrar, ativar e desativar contas de usuários, para controlar o acesso à plataforma.

**Critérios de aceitação:**

**Given:** o administrador está autenticado e possui permissão para gerenciar contas.

**When:** cadastra uma nova conta ou solicita a ativação ou desativação de uma conta existente.

**Then:** o sistema valida as informações, registra a operação e atualiza a situação da conta.

Story Points: 5.

---

Como usuário, quero visualizar e atualizar minhas informações cadastrais, para manter meus dados pessoais atualizados na plataforma.

**Critérios de aceitação:**

**Given:** o usuário está autenticado.

**When:** acessa seus dados cadastrais e realiza alterações nas informações permitidas.

**Then:** o sistema valida os dados e salva as alterações, permitindo que o usuário visualize as informações atualizadas.

Story Points: 3.

---

Como administrador, quero cadastrar, atualizar e desativar especialidades médicas, para manter organizadas as categorias de atendimento disponíveis na plataforma.

**Critérios de aceitação:**

**Given:** o administrador está autenticado e possui permissão para gerenciar especialidades.

**When:** cadastra, atualiza ou desativa uma especialidade.

**Then:** o sistema valida a operação e atualiza as especialidades disponíveis para consulta e associação aos profissionais.

Story Points: 3.

---

Como administrador, quero associar profissionais de saúde às suas respectivas especialidades, para permitir que os pacientes encontrem profissionais conforme o tipo de atendimento desejado.

**Critérios de aceitação:**

**Given:** existem profissionais e especialidades cadastrados na plataforma.

**When:** o administrador seleciona um profissional e associa uma ou mais especialidades.

**Then:** o sistema registra a associação e disponibiliza as informações para pesquisa de profissionais.

Story Points: 3.

## 2.5 Resumo das histórias de usuário

| Perfil | Quantidade de histórias |
|---|---|
| **Paciente** | 11 |

| | |
|---|---|
| **Profissional de saúde** | 3 |
| **Recepcionista** | 3 |
| **Administrador** | 7 |
| **Total** | **24** |

**Os Story Points representam estimativas iniciais de complexidade, não correspondendo diretamente a horas de desenvolvimento.**

---

## 2.6 Considerações finais

A especificação dos requisitos do VittaCare estabelece as funcionalidades essenciais da plataforma e os critérios técnicos necessários para seu funcionamento.

Os 26 requisitos funcionais definem as operações disponíveis para os diferentes perfis de usuários, enquanto os 17 requisitos não funcionais estabelecem condições relacionadas ao desempenho, à escalabilidade, à disponibilidade, à segurança e à acessibilidade do sistema.

As 24 histórias de usuário complementam essa especificação, apresentando as necessidades dos pacientes, profissionais de saúde, recepcionistas e administradores, juntamente com seus respectivos critérios de aceitação e estimativas de esforço (Story Points).

Dessa forma, os requisitos e as histórias de usuário definidos servirão como base para as próximas etapas do projeto, orientando a elaboração da arquitetura de software e dos diagramas UML, de acordo com as funcionalidades e restrições estabelecidas para o VittaCare.

# VittaCare

Arquitetura – Bloco 3

## Sumário

1. ARQUITETURA DE SOFTWARE ..... 1
1.1. Visão geral da arquitetura ..... 1
1.2. Estilo arquitetural ..... 2
1.2.1. Organização em camadas ..... 2
1.3. Módulos do sistema ..... 3
1.4. Comunicação entre os componentes ..... 4
1.5. Segurança e controle de acesso ..... 4
1.5.1. Isolamento dos dados das clínicas ..... 5
1.5.2. Medidas complementares de segurança ..... 5
1.6. Stack tecnológica ..... 6
1.7. Padrões de design ..... 12
1.8. Modelo C4 ..... 13
1.8.1. C1 — Diagrama de contexto ..... 13
1.8.2. C2 — Diagrama de containers ..... 14
1.8.3. C3 — Diagrama de componentes ..... 16
1.8.4. C4 — Diagrama de código ..... 17
1.9. Architecture Decision Records (ADRs) ..... 18
1.10. Considerações finais ..... 22

# 1. ARQUITETURA DE SOFTWARE

## 1.1. Visão geral da arquitetura

O VittaCare será desenvolvido como uma aplicação web no modelo SaaS (Software as a Service), destinada ao gerenciamento de clínicas e unidades ambulatoriais. A plataforma permitirá o gerenciamento de pacientes, profissionais de saúde, especialidades, disponibilidades, agendamentos, atendimentos e relatórios administrativos.

A arquitetura será organizada no estilo monólito modular, no qual os módulos do sistema serão executados em uma única aplicação back-end, mantendo responsabilidades delimitadas e comunicação interna por interfaces e serviços. Essa abordagem preserva a simplicidade operacional necessária à versão inicial, sem impedir a evolução futura de módulos específicos.

A solução será dividida em três partes principais:

Front-end: interface de usuário desenvolvida com React.

Back-end: aplicação Java com Spring Boot, responsável pelas regras de negócio, autenticação, autorização, validações e disponibilização dos serviços.

Banco de dados: PostgreSQL, disponibilizado por meio do Amazon RDS, responsável pelo armazenamento persistente das informações.

A comunicação entre o front-end e o back-end ocorrerá por meio de API REST, utilizando JSON sobre HTTPS. A infraestrutura será hospedada na Amazon Web Services (AWS), com duas instâncias EC2 de aplicação distribuídas atrás de um Application Load Balancer (ALB), além de banco de dados RDS PostgreSQL em configuração Multi-AZ.

## 1.2. Estilo arquitetural

O estilo arquitetural escolhido para o VittaCare é o monólito modular com arquitetura em camadas. Nesse modelo, o back-end é implantado como uma aplicação única, mas seu código é dividido em módulos de domínio e camadas técnicas com responsabilidades específicas.

A escolha é adequada ao escopo inicial porque permite desenvolver, testar e implantar a solução com menor complexidade operacional do que uma arquitetura distribuída em microsserviços. Os módulos não deverão acessar diretamente os repositórios uns dos outros sem passar pelas interfaces ou serviços definidos, preservando o baixo acoplamento.

### 1.2.1. Organização em camadas

Camada de apresentação (Front-end): desenvolvida com React, apresenta as telas e permite a interação dos pacientes, profissionais de saúde, recepcionistas e administradores.

Camada de controle (Controllers): recebe requisições HTTP, aplica validações de entrada e encaminha as operações para os serviços correspondentes.

Camada de serviços (Services): concentra as regras de negócio, incluindo validação de horários, agendamentos, cancelamentos, alterações de disponibilidade, permissões e isolamento por clínica.

Camada de persistência (Repositories): realiza o acesso aos dados por meio de Spring Data JPA e Hibernate, aplicando consultas e operações transacionais.

Camada de dados (Database): utiliza PostgreSQL para armazenar clínicas, usuários, pacientes, profissionais, especialidades, disponibilidades, agendamentos, atendimentos e registros de auditoria.

## 1.3. Módulos do sistema

| Módulo | Responsabilidade |
|---|---|
| Autenticação | Realizar login, validação de credenciais, emissão e validação de tokens JWT. |
| Usuários e permissões | Gerenciar contas, perfis de acesso, papéis e permissões. |
| Clínicas | Gerenciar organizações cadastradas e o contexto de isolamento de cada clínica. |
| Pacientes | Cadastrar, consultar e atualizar informações dos pacientes. |
| Profissionais | Gerenciar dados dos profissionais de saúde. |
| Especialidades | Cadastrar especialidades e associá-las aos profissionais. |
| Disponibilidade | Gerenciar horários disponíveis e bloqueados dos profissionais. |
| Agendamentos | Criar, consultar, confirmar, cancelar e reagendar consultas, impedindo conflitos. |
| Atendimentos | Registrar comparecimento e ausência relacionados aos agendamentos. |
| Notificações | Enviar confirmações e lembretes relacionados aos agendamentos. |
| Relatórios e auditoria | Disponibilizar informações administrativas e registrar operações críticas. |

Os módulos serão organizados dentro da mesma aplicação Spring Boot. A separação lógica não representa serviços independentes: todos compartilham o processo de execução do monólito, a configuração de segurança e a infraestrutura de persistência.

## 1.4. Comunicação entre os componentes

A comunicação entre o front-end e o back-end será realizada por meio de API REST, utilizando HTTPS e mensagens JSON. O fluxo básico será:

O usuário realiza uma ação na interface React.

O front-end envia uma requisição HTTPS para o endpoint correspondente.

O Controller recebe a requisição, valida os dados de entrada e encaminha a operação.

O Service verifica as regras de negócio, as permissões e o contexto da clínica associado ao usuário autenticado.

Quando necessário, o Service solicita operações ao Repository.

O Repository realiza a consulta ou alteração no PostgreSQL por meio do Spring Data JPA/Hibernate.

O back-end retorna uma resposta HTTP com os dados ou mensagem de erro em JSON.

O front-end apresenta o resultado ao usuário.

As operações de agendamento e bloqueio de disponibilidade deverão utilizar transações e restrições de integridade no banco de dados para evitar conflitos em solicitações concorrentes. A documentação dos endpoints será mantida com Swagger e OpenAPI.

## 1.5. Segurança e controle de acesso

Como o VittaCare armazenará dados pessoais e informações relacionadas à prestação de serviços de saúde, a arquitetura deverá aplicar autenticação, autorização, isolamento entre clínicas, proteção das comunicações e auditoria das operações relevantes.

A autenticação será realizada com Spring Security e JWT. Após validar as credenciais, o back-end emitirá um token assinado, com validade definida, que será enviado nas requisições subsequentes. O sistema deverá validar assinatura, expiração e permissões antes de permitir o acesso a recursos protegidos.

Paciente: poderá consultar e atualizar os próprios dados permitidos e realizar operações relacionadas aos seus agendamentos.

Profissional de saúde: poderá gerenciar sua disponibilidade e consultar sua agenda e os atendimentos autorizados.

Recepcionista: poderá cadastrar pacientes e realizar operações administrativas de agendamento conforme as permissões atribuídas.

Administrador: poderá gerenciar usuários, profissionais, especialidades, permissões e relatórios da clínica.

As permissões serão verificadas no back-end, em todos os endpoints protegidos. A interface não será considerada mecanismo de segurança: ocultar uma opção de menu não substitui a autorização no servidor.

### 1.5.1. Isolamento dos dados das clínicas

Por ser uma plataforma SaaS destinada a diferentes clínicas, o VittaCare adotará isolamento lógico dos dados. Cada registro pertencente a uma organização deverá estar associado a uma clínica por meio do identificador clinica_id, incluindo usuários, pacientes, profissionais, especialidades, disponibilidades, agendamentos, atendimentos e registros administrativos aplicáveis.

O back-end obterá o contexto da clínica a partir da identidade autenticada e das associações autorizadas do usuário. O clinica_id eventualmente enviado pelo front-end será tratado apenas como dado de entrada e nunca como prova de autorização. Antes de consultar, inserir, atualizar ou excluir registros, os serviços deverão validar se o registro pertence à clínica do contexto autenticado.

As consultas e operações de persistência deverão aplicar filtros por clinica_id. Chaves estrangeiras e restrições compostas deverão ser utilizadas, quando aplicável, para impedir associações entre registros de clínicas diferentes. Serão realizados testes automatizados de acesso cruzado entre clínicas.

### 1.5.2. Medidas complementares de segurança

Utilização obrigatória de HTTPS com TLS 1.2 ou superior entre cliente e serviços públicos.

Armazenamento de senhas com algoritmo próprio para hash de senhas, preferencialmente Argon2id ou BCrypt, com parâmetros adequados.

Validação de entradas, limites de tamanho e tratamento seguro de erros na API.

Autenticação e autorização em 100% dos endpoints que exponham informações restritas.

Proteção de credenciais e configurações sensíveis por mecanismo de gerenciamento de segredos; não armazenar segredos no repositório Git.

Restrição de acesso ao banco de dados em sub-redes privadas, permitindo conexão apenas a partir das instâncias de aplicação autorizadas.

Registro das operações administrativas críticas, incluindo usuário responsável, clínica, ação, entidade afetada e data/hora.

Aplicação de controles contra vulnerabilidades comuns, incluindo validação de entrada, proteção contra injeção, controle de sessão/token e configuração segura dos componentes.

## 1.6. Stack tecnológica

A stack tecnológica foi definida considerando os requisitos funcionais e não funcionais do VittaCare, especialmente a necessidade de interface web, API segura, integridade relacional dos agendamentos, isolamento de dados entre clínicas, disponibilidade mensal mínima de 99,9%, suporte à carga inicial projetada e recuperação de dados.

| Camada | Tecnologia escolhida | Alternativa descartada | Justificativa técnica |
|---|---|---|---|
| **Front-end** | React | Angular | A interface será composta por telas e componentes reutilizáveis para os quatro perfis de usuário. React permite organizar componentes e integrar-se à API REST sem impor uma estrutura de servidor ao front-end. |
| **Linguagem do back-end** | Java | JavaScript com Node.js | Java oferece tipagem estática e recursos consolidados para estruturar regras de negócio, validações e operações transacionais |

| | | | |
|---|---|---|---|
| | | | necessárias aos agendamentos. |
| **Framework do back-end** | Spring Boot | Express.js | Spring Boot integra recursos para construção de APIs, injeção de dependência, validação, segurança e persistência, atendendo à organização em camadas proposta. |
| **Comunicação** | API REST com JSON sobre HTTPS | GraphQL | As operações previstas são recursos e ações bem delimitados, como cadastro, consulta e agendamento. REST permite contratos HTTP objetivos e integração simples com o front-end. |
| **Banco de dados** | PostgreSQL no Amazon RDS Multi-AZ | MySQL ou MongoDB | O modelo relacional atende às relações entre clínicas, usuários, pacientes, profissionais, especialidades, disponibilidades e agendamentos, permitindo FKs, constraints e transações para |

| | | | |
|---|---|---|---|
| | | | preservar integridade. O PostgreSQL oferece índice único parcial nativo, usado para impedir mais de um agendamento ativo por disponibilidade, recurso que o MySQL não oferece. |
| **Persistência** | Spring Data JPA / Hibernate | JDBC puro | O mapeamento objeto-relacional reduz código repetitivo nas operações CRUD e integra-se aos repositories. Consultas específicas poderão usar JPQL ou SQL quando necessário. |
| **Autenticação e autorização** | Spring Security + JWT | Autenticação implementada manualmente | Centraliza filtros de segurança e autorização no back-end, permitindo proteger endpoints conforme identidade, perfil e contexto da clínica. |
| **Documentação da API** | Swagger / OpenAPI | Documentação exclusivamente manual | Define um contrato padronizado de |

| | | | |
|---|---|---|---|
| | | | endpoints, parâmetros, respostas e autenticação, facilitando integração e testes. |
| **Versionamento** | Git e GitHub | Compartilhamento manual de arquivos | Mantém histórico, branches, revisão de alterações e colaboração entre integrantes. |
| **Containerização** | Docker | Configuração manual de ambientes | Padroniza a execução da aplicação e reduz diferenças entre desenvolvimento, homologação e produção. |
| **Hospedagem da aplicação** | Amazon EC2 em duas instâncias atrás de ALB | Uma única instância ou servidor físico | Duas instâncias permitem distribuir requisições e manter capacidade de atendimento caso uma instância falhe. O ALB executa health checks e deixa de encaminhar tráfego a destinos não saudáveis. |
| **Balanceamento** | AWS Application Load Balancer | Distribuição manual de tráfego | Distribui requisições HTTPS entre as instâncias EC2 e permite verificações de |

| | | | |
|---|---|---|---|
| | | | integridade dos destinos. |
| **Banco de dados gerenciado** | Amazon RDS PostgreSQL Multi-AZ | Banco instalado manualmente em EC2 | A configuração Multi-AZ oferece redundância gerenciada do banco, alinhada ao requisito de disponibilidade. A instância e o armazenamento deverão ser dimensionados e monitorados conforme carga real. |
| **Armazenamento de arquivos** | Amazon S3, quando necessário | Armazenamento no disco da aplicação | Mantém arquivos separados das instâncias de aplicação e permite acesso independente do ciclo de vida dos containers. |
| **Monitoramento** | Amazon CloudWatch | Acompanhamento exclusivamente manual | Permite coletar métricas, logs e alarmes para acompanhar erros, desempenho e disponibilidade. |
| **Backup e recuperação** | Backups automatizados do RDS com recuperação point-in-time (PITR) e AWS Backup para | Cópias manuais sem política centralizada | PITR e logs de transação dão suporte a RPO de até 15 minutos; cópias diárias serão mantidas por pelo menos 7 dias e snapshots |

| | | | |
|---|---|---|---|
| | cópias semanais | | semanais por pelo menos 4 semanas. O RTO de até 4 horas deverá ser validado em testes periódicos de restauração. |
| **Rede e proteção** | AWS VPC, sub-redes privadas e Security Groups | Exposição direta dos serviços à internet | Restringe a comunicação entre balanceador, aplicação e banco, reduzindo a superfície de exposição. |
| **Distribuição de conteúdo** | Amazon CloudFront | Entrega de arquivos estáticos diretamente pela aplicação | Pode distribuir conteúdo estático e arquivos do S3, reduzindo latência para esse tipo de recurso. |
| **Testes** | : JUnit 5 e Mockito (back-end); Jest e React Testing Library (front-end) | Testes exclusivamente manuais | Permitem verificar regras de negócio, endpoints, autorização e regressões de forma repetível; o JUnit acompanha o starter de testes do Spring Boot e o Jest cobre os componentes React.. |

A infraestrutura de produção prevista considera duas instâncias EC2 com configuração inicial de 2 vCPUs e 2 GB de memória cada, distribuídas atrás de um Application Load Balancer. O banco de dados será Amazon RDS PostgreSQL com

implantação Multi-AZ e armazenamento inicial estimado de 50 GB gp3. Os valores de capacidade são premissas de dimensionamento inicial e deverão ser validados por testes de carga antes da entrada em produção.

Os ambientes DEV, STAGING e PROD deverão ser separados. Desenvolvimento e homologação utilizarão recursos e bases de dados próprios, sem compartilhar dados reais de produção. O acesso ao banco de produção será restrito às instâncias autorizadas da aplicação.

## 1.7. Padrões de design

Os padrões de design serão utilizados para organizar o código, separar responsabilidades e facilitar a manutenção e os testes do VittaCare. A aplicação dos padrões deverá permanecer compatível com o monólito modular, evitando complexidade desnecessária.

| Padrão | Onde será aplicado | Motivo da utilização |
|---|---|---|
| Repository | Camada de persistência dos módulos de pacientes, profissionais, disponibilidade e agendamentos. | Isolar o acesso ao banco de dados das regras de negócio e concentrar consultas e operações de persistência. |
| Service Layer | Serviços de autenticação, agendamento, disponibilidade, usuários e clínicas. | Centralizar regras de negócio, validações e transações, evitando que regras fiquem espalhadas pelos Controllers. |
| DTO (Data Transfer Object) | Comunicação entre front-end e back-end. | Definir os dados recebidos e enviados pela API, evitando expor diretamente entidades e campos internos do banco. |
| Dependency Injection | Controllers, Services e Repositories gerenciados pelo Spring. | Reduzir acoplamento e facilitar substituição de dependências e testes. |
| MVC | Organização da camada web do back-end e integração com a interface. | Separar responsabilidades de requisição, processamento e |

| | | |
|---|---|---|
| | | apresentação dos dados retornados pela API. |
| Strategy (quando houver variação de canal) | Serviço de notificações, caso sejam implementados diferentes provedores ou canais. | Permitir variar o mecanismo de envio sem alterar a regra de negócio do agendamento. Será aplicado somente se houver mais de uma estratégia concreta. |

Os padrões serão aplicados conforme a necessidade de cada módulo. Não serão adotados CQRS, Saga ou microsserviços na versão inicial, pois o projeto não possui, neste momento, requisitos que justifiquem a complexidade adicional desses padrões.

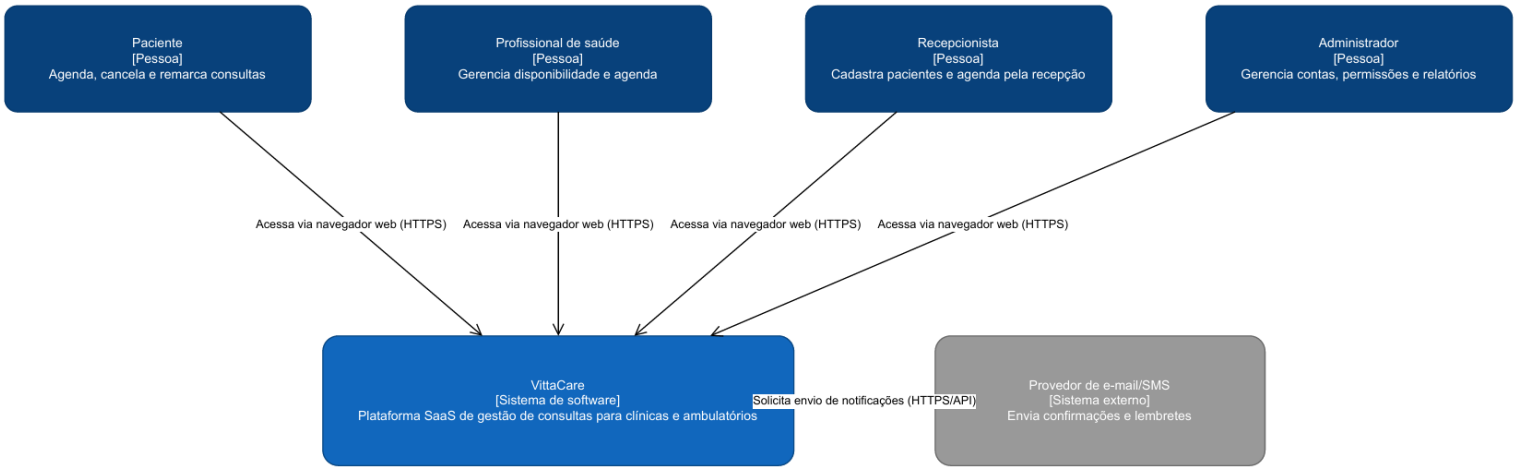
## 1.8. Modelo C4

O modelo C4 será utilizado para representar a arquitetura do VittaCare em quatro níveis de detalhamento: contexto, containers, componentes e código. Os diagramas devem manter consistência com o monólito modular, com a stack tecnológica e com a infraestrutura definida neste documento.

### 1.8.1. C1 — Diagrama de contexto

O diagrama de contexto apresenta o VittaCare como um sistema central e demonstra sua relação com os usuários e sistemas externos. Os atores são Paciente, Profissional de Saúde, Recepcionista e Administrador. O sistema externo previsto é o provedor de e-mail/SMS utilizado para notificações. Os usuários acessam o VittaCare por navegador web; o VittaCare envia solicitações de notificação ao provedor externo por HTTPS/API.

**Diagrama C4 nível 1 — Contexto (VittaCare)**
Autores: Gabriela Dias, João Batista | Data: 07/11/2026
Requisitos: RF001–RF026, RNF006, RNF009–RNF011, RNF014



### 1.8.2. C2 — Diagrama de containers

O diagrama de containers apresenta as principais unidades executáveis e os serviços de infraestrutura utilizados pelo VittaCare. Embora o back-end seja implantado em duas instâncias para disponibilidade, as duas instâncias executam a mesma aplicação monolítica modular; não representam microsserviços.

Navegador Web / React: executa a interface dos usuários e comunica-se com a API por HTTPS (porta 443), utilizando JSON.

Application Load Balancer (AWS): recebe o tráfego HTTPS público e distribui as requisições entre as instâncias EC2, utilizando health checks para identificar destinos não saudáveis.

EC2 A — aplicação VittaCare: executa Docker e a aplicação Java/Spring Boot, disponibilizando API REST internamente na porta configurada (por exemplo, 8080).

EC2 B — aplicação VittaCare: executa a mesma imagem e versão da aplicação Java/Spring Boot, também registrada como destino do ALB. As instâncias devem estar distribuídas em zonas de disponibilidade distintas, conforme disponibilidade regional e configuração contratada.

Amazon RDS PostgreSQL Multi-AZ: armazena os dados persistentes; a comunicação da aplicação com o banco ocorre pela porta TCP 5432, em rede privada e com criptografia em trânsito habilitada.

Amazon S3: armazena arquivos necessários à plataforma, sem utilizar o disco local das instâncias como armazenamento persistente compartilhado.

Amazon CloudWatch: recebe métricas, logs e alarmes dos recursos da aplicação e da infraestrutura.

Backups automatizados do RDS com point-in-time recovery (PITR): permitem restaurar o banco a um ponto no tempo, com logs de transação; a configuração deverá manter janela de recuperação compatível com RPO de até 15 minutos e retenção diária mínima de 7 dias.

AWS Backup: manterá snapshots semanais por pelo menos 4 semanas; o RTO de até 4 horas deverá ser comprovado por testes de restauração.

Amazon CloudFront: distribui conteúdo estático, quando utilizado, integrado ao S3.

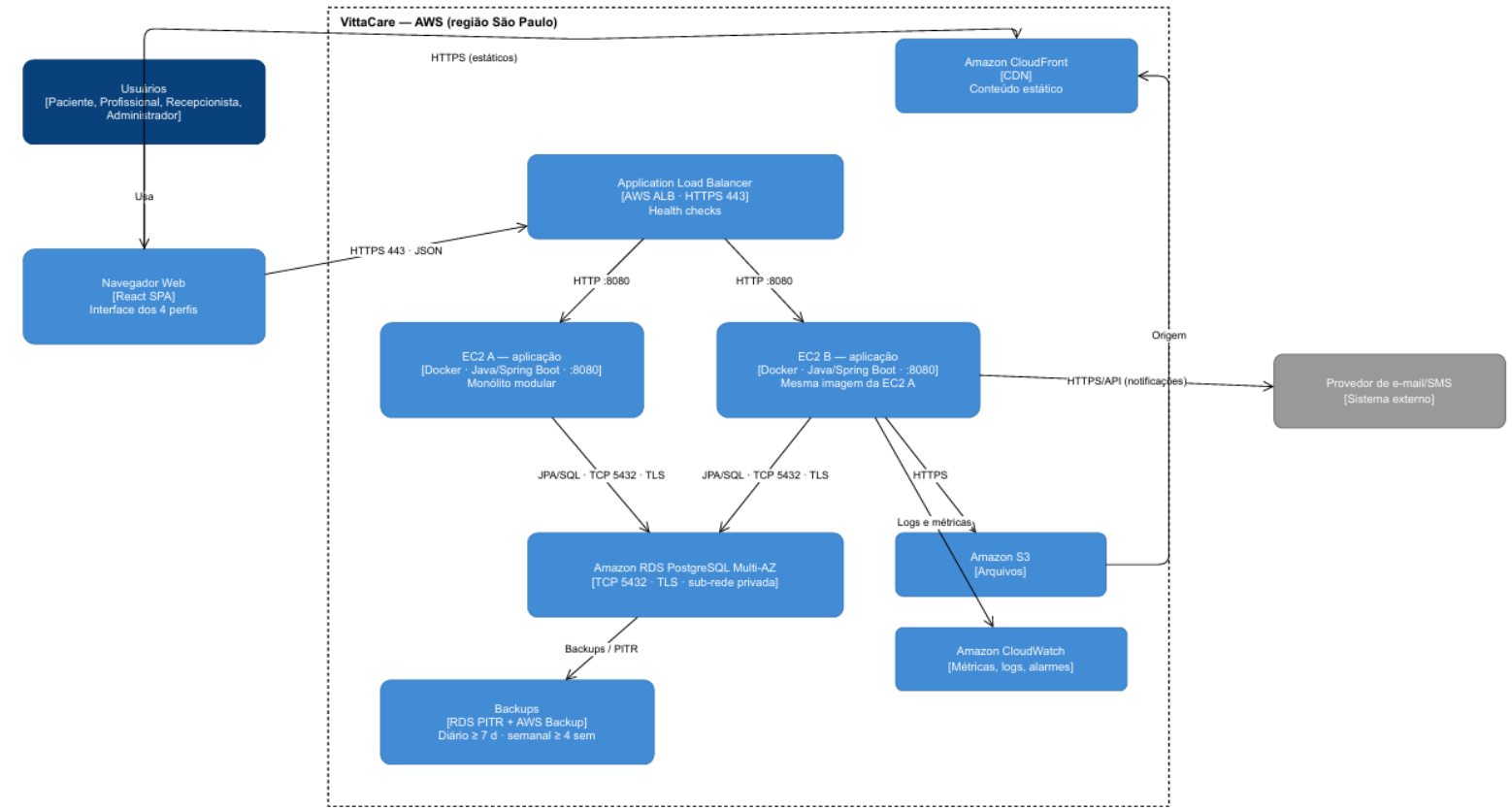
A política de recuperação deverá combinar point-in-time recovery (PITR) do RDS, com retenção diária mínima de 7 dias e logs de transação, e snapshots semanais gerenciados pelo AWS Backup com retenção mínima de 4 semanas. O RPO de até

15 minutos depende da configuração e disponibilidade dos backups contínuos; o RTO de até 4 horas deverá ser verificado por exercícios de restauração documentados. O backup diário isolado, por si só, não atende ao RPO de 15 minutos.

O ALB deverá encaminhar tráfego somente para instâncias saudáveis. As instâncias EC2 não deverão expor diretamente a porta da aplicação à internet; o acesso deverá ser permitido apenas a partir do Security Group do ALB. O banco RDS permanecerá em sub-redes privadas e aceitará conexões apenas das instâncias de aplicação autorizadas.

Os ambientes DEV, STAGING e PROD serão separados. O desenho de implantação deverá representar os recursos de produção com duas instâncias EC2 e ALB, e os ambientes não produtivos com recursos isolados e dimensionamento próprio. A topologia de produção é a referência para a disponibilidade definida no RNF006.

**Diagrama C4 nível 2 — Containers (VittaCare)**
Autores: Gabriela Dias, João Batista | Data: 07/11/2026
Requisitos: RF001–RF026, RNF006, RNF009–RNF011, RNF014 | ADR-001, ADR-007

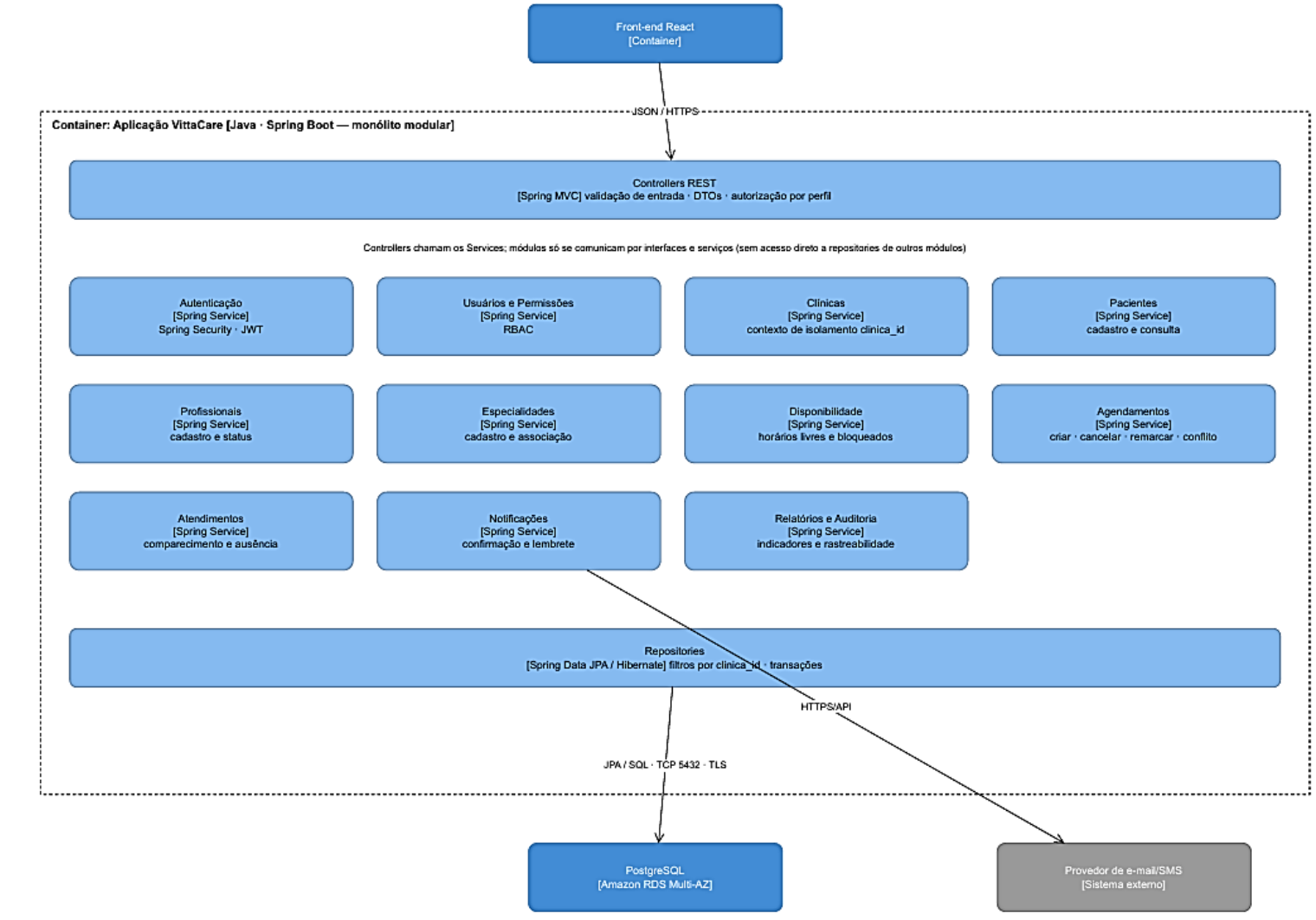


### 1.8.3. C3 — Diagrama de componentes

O diagrama de componentes detalha a organização interna da aplicação monolítica. O front-end React consome a API REST disponibilizada pelos Controllers. No back-end, os Controllers dependem dos Services; os Services aplicam as regras de negócio e utilizam os Repositories para persistência. Os módulos de autenticação, usuários e clínicas, pacientes, profissionais e especialidades, disponibilidade, agendamentos, atendimentos, notificações e relatórios/auditoria permanecem dentro do mesmo container de aplicação.

Os Repositories utilizam Spring Data JPA/Hibernate para acessar o PostgreSQL. O módulo de notificações integra-se ao provedor externo por HTTPS. As dependências entre módulos deverão ocorrer por interfaces e serviços internos, evitando acesso direto a estruturas internas de outros módulos.

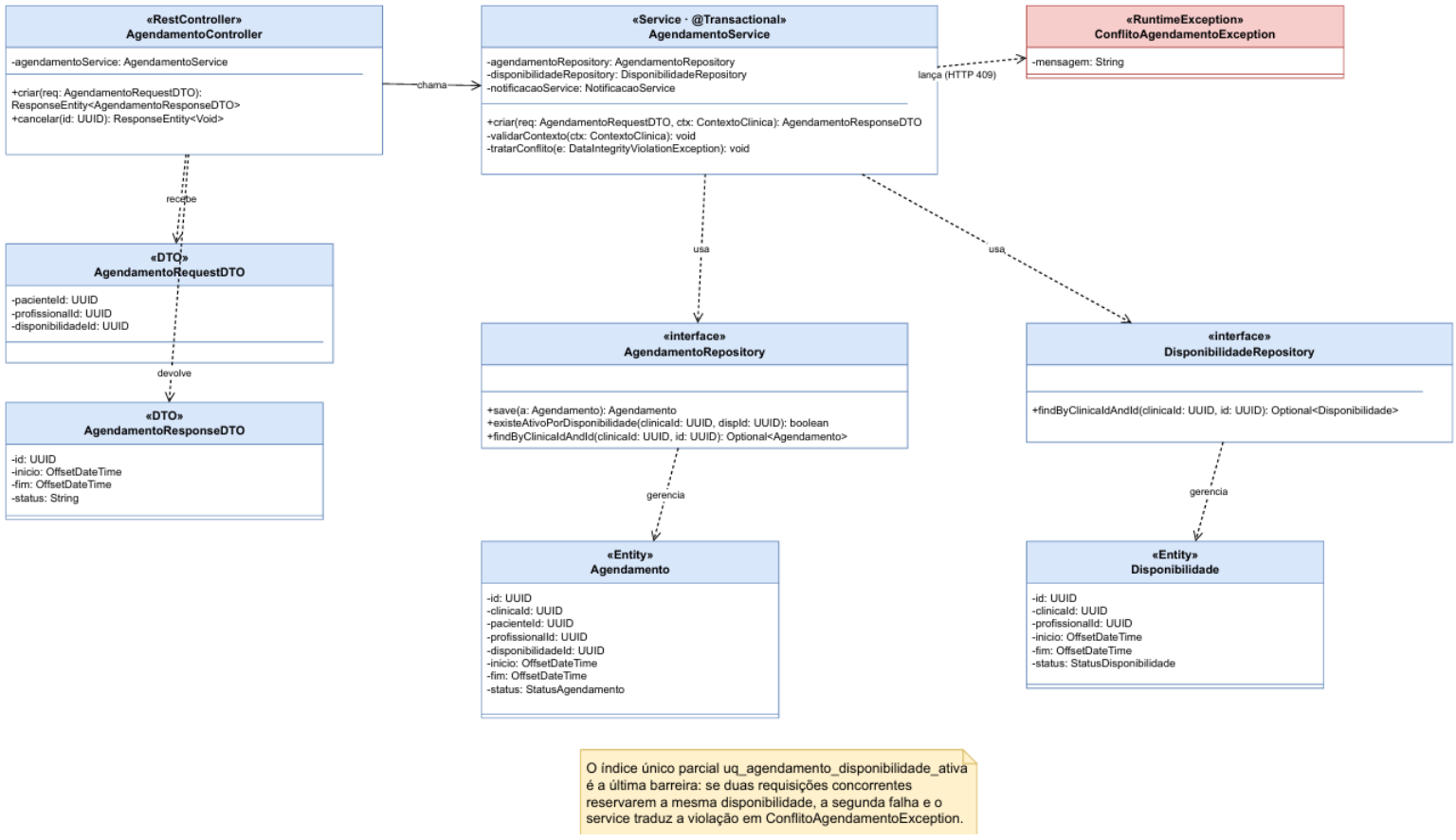
**Diagrama C4 nível 3 — Componentes do container Aplicação Spring Boot**
Autores: Gabriela Dias, João Batista | Data: 07/11/2026
Requisitos: RF001–RF026, RNF006, RNF009–RNF011, RNF014 | ADR-001, ADR-003, ADR-009



### 1.8.4. C4 — Diagrama de código

O diagrama de código detalhará um componente crítico: o serviço de agendamento. A estrutura proposta inclui AgendamentoController, AgendamentoService, DisponibilidadeRepository, AgendamentoRepository, entidades Agendamento e Disponibilidade, DTOs de entrada e saída e o mecanismo de transação do Spring. O AgendamentoService valida o contexto da clínica, verifica a disponibilidade, executa a criação em transação e trata conflitos de concorrência antes de retornar o resultado ao Controller.

**Diagrama C4 nível 4 — Código do componente crítico: serviço de agendamento**
Autores: Gabriela Dias, João Batista | Data: 07/11/2026
Requisitos: RF001–RF026, RNF006, RNF009–RNF011, RNF014 | RF014, RF018, RNF003, RNF008



## 1.9. Architecture Decision Records (ADRs)

Os ADRs registram as decisões arquiteturais relevantes, incluindo contexto, decisão, alternativas descartadas e consequências. As decisões abaixo deverão ser revisadas caso os requisitos ou as premissas de dimensionamento sejam alterados.

**ADR-001 — Adoção da arquitetura de Monólito Modular**

Contexto: o VittaCare possui módulos de negócio relacionados e uma equipe que precisa desenvolver e manter a primeira versão com complexidade operacional controlada.

Decisão: adotar monólito modular com separação em camadas e módulos de domínio.

Alternativas descartadas: microsserviços independentes, por exigirem implantação, observabilidade, comunicação distribuída e gestão de dados por serviço; monólito sem modularização, por dificultar a separação de responsabilidades.

Consequências: implantação e transações mais simples; será necessário impor limites entre módulos e evitar dependências circulares.

**ADR-002 — Utilização do React no Front-end**

Contexto: o sistema possui quatro perfis de usuário, com interfaces e fluxos distintos que consomem uma API REST, e a equipe prevista conta com um desenvolvedor front-end (Bloco 7). A interface precisa oferecer navegação por teclado e compatibilidade com Chrome, Firefox e Edge (RNF016 e RNF017).

Decisão: utilizar React para construir a interface baseada em componentes.

Alternativas descartadas: Angular, por ser um framework completo que impõe TypeScript, injeção de dependência e estrutura própria de módulos, aumentando a curva de aprendizado e o volume de código-base para um escopo de telas de cadastro, agenda e consulta e para uma equipe front-end de uma pessoa; páginas renderizadas exclusivamente no servidor, por não atenderem tão diretamente à interação de agenda e seleção de horários planejada.

Consequências: componentes reutilizáveis e integração desacoplada via API; será necessário padronizar rotas, estado, validação e acessibilidade, pois o React não impõe esses padrões.

**ADR-003 — Utilização de Java e Spring Boot no Back-end**

Contexto: o sistema exige regras de negócio, validações, autorização, transações e integração com PostgreSQL.

Decisão: utilizar Java com Spring Boot para implementar a API e os serviços.

Alternativas descartadas: Node.js com Express, por exigir composição manual de mais recursos estruturais e de segurança que já são integrados ao ecossistema Spring adotado.

Consequências: organização em camadas, injeção de dependência e integração com segurança e persistência; a equipe deverá manter testes e configuração padronizada.

**ADR-004 — Utilização do PostgreSQL**

Contexto: o domínio contém entidades relacionadas e regras de integridade, especialmente em agendamentos e no isolamento por clínica. A regra mais crítica (RF018 e RNF008) exige impedir dois agendamentos ativos para a mesma disponibilidade, mesmo com requisições concorrentes.

Decisão: utilizar PostgreSQL como banco relacional, hospedado no Amazon RDS.

Alternativas descartadas: MongoDB, por não oferecer o mesmo modelo relacional e o conjunto de constraints (FKs, FKs compostas, CHECK e UNIQUE) que atendem diretamente à integridade do domínio e ao isolamento por clinica_id. MySQL, por não oferecer índices únicos parciais nativos: o VittaCare depende de um índice único parcial (uq_agendamento_disponibilidade_ativa, restrito aos status AGENDADO e CONFIRMADO) como última barreira contra reservas duplicadas, e reproduzi-lo no MySQL exigiria soluções alternativas, como colunas geradas, aumentando a complexidade da regra mais crítica do sistema.

Consequências: uso de FKs, constraints, transações e índices; o modelo lógico inclui clinica_id e restrições adequadas ao isolamento multi-tenant; a equipe passa a depender de recursos específicos do PostgreSQL, como o índice parcial, o que deve ser considerado em eventual troca de banco.

**ADR-005 — Comunicação por API REST e JSON**

Contexto: front-end e back-end serão desenvolvidos separadamente e precisam de contratos explícitos.

Decisão: disponibilizar API REST com JSON sobre HTTPS.

Alternativas descartadas: GraphQL, pois as operações previstas podem ser representadas por recursos e endpoints REST; comunicação direta com o banco, por comprometer segurança e separação de camadas.

Consequências: contratos HTTP padronizados e integração simples; será necessário versionar e documentar endpoints.

**ADR-006 — Autenticação com Spring Security e JWT**

Contexto: os quatro perfis possuem permissões diferentes e o sistema é multi-tenant.

Decisão: utilizar Spring Security para autenticação/autorização e JWT assinado para identificar sessões de acesso à API.

Alternativas descartadas: autenticação implementada manualmente, por aumentar risco de falhas; sessão tradicional como único mecanismo, por não corresponder ao modelo de API stateless planejado.

Consequências: validação de assinatura, expiração, papéis e contexto de clínica em cada requisição protegida; tokens deverão ter validade e armazenamento seguros.

**ADR-007 — Hospedagem na AWS com redundância da aplicação**

Contexto: RNF006 define disponibilidade mensal mínima de 99,9%; o estudo de viabilidade estima duas EC2, ALB e RDS Multi-AZ.

Decisão: hospedar a aplicação em duas instâncias EC2 atrás de um Application Load Balancer, com RDS PostgreSQL Multi-AZ e recursos complementares AWS.

Alternativas descartadas: uma única EC2 sem balanceamento, por criar ponto único de falha na camada de aplicação; servidor físico próprio, por transferir à equipe a gestão de hardware e disponibilidade.

Consequências: distribuição de tráfego e health checks; o desenho e a operação deverão contemplar zonas de disponibilidade, monitoramento, custos de transferência e testes de failover. A topologia contribui para o RNF006, mas não garante sozinha 99,9%: a disponibilidade deverá ser medida ponta a ponta.

**ADR-008 — Utilização de Docker**

Contexto: os ambientes DEV, STAGING e PROD precisam executar a aplicação de maneira consistente.

Decisão: empacotar o back-end em containers Docker.

Alternativas descartadas: configuração manual independente em cada servidor, por aumentar diferenças entre ambientes.

Consequências: imagens versionadas e implantação repetível; será necessário manter imagens atualizadas, reduzir privilégios e proteger o processo de build.

**ADR-009 — Utilização de Spring Data JPA e Hibernate**

Contexto: a aplicação possui operações CRUD e relacionamentos entre entidades persistidas no PostgreSQL.

Decisão: utilizar Spring Data JPA e Hibernate como solução principal de persistência.

Alternativas descartadas: JDBC puro como padrão de todas as operações, por exigir maior volume de código repetitivo; consultas SQL manuais para todos os casos, por dificultar padronização.

Consequências: repositories e mapeamento objeto-relacional; consultas críticas deverão ser avaliadas quanto a índices, N+1, bloqueios e desempenho.

**ADR-010 — Documentação da API com Swagger e OpenAPI**

Contexto: o front-end depende de contratos claros para consumir endpoints e testar respostas.

Decisão: documentar a API com OpenAPI e disponibilizar visualização por ferramentas Swagger.

Alternativas descartadas: documentação exclusivamente manual, por aumentar o risco de divergência; ausência de documentação formal, por dificultar integração e testes.

Consequências: contratos consultáveis e padronizados; a documentação deverá ser atualizada junto às alterações da API.

**ADR-011 — Versionamento com Git e GitHub**

Contexto: o desenvolvimento será realizado em equipe e exige histórico, colaboração e revisão.

Decisão: utilizar Git para controle de versão e GitHub para hospedagem e colaboração.

Alternativas descartadas: compartilhamento manual de arquivos, por dificultar rastreabilidade e integração; controle de versão centralizado, por não ser a opção selecionada para o fluxo da equipe.

Consequências: uso de branches, commits e pull requests; será necessário definir convenções e regras de revisão.

**ADR-012 — Isolamento Lógico dos Dados por Clínica**

Contexto: o VittaCare será SaaS multi-tenant e não poderá permitir acesso cruzado aos dados de clínicas diferentes.

Decisão: utilizar banco compartilhado com isolamento lógico por clinica_id, validado no back-end em todas as operações e reforçado por constraints relacionais.

Alternativas descartadas: banco sem identificação de organização, por não separar os registros; banco independente por clínica, por aumentar a complexidade operacional e de custos na versão inicial.

Consequências: consultas e operações deverão aplicar escopo de clínica; serão obrigatórios testes de acesso cruzado e auditoria das operações sensíveis.

## 1.10. Considerações finais

A arquitetura proposta para o VittaCare busca equilibrar organização, segurança, disponibilidade, manutenção e possibilidade de evolução. A adoção de um monólito modular permite separar responsabilidades sem introduzir, na versão inicial, a complexidade de uma arquitetura distribuída.

A utilização de React, Java com Spring Boot e PostgreSQL estabelece uma base compatível com a aplicação web e com as regras de integridade do domínio. A infraestrutura AWS proposta — duas instâncias EC2 atrás de um ALB e banco RDS PostgreSQL Multi-AZ — está alinhada ao objetivo de disponibilidade, enquanto CloudWatch e a política de backup apoiam monitoramento e recuperação.

Os requisitos de 99,9% de disponibilidade, 1.000 usuários simultâneos, RPO de até 15 minutos, RTO de até 4 horas e isolamento lógico entre clínicas deverão ser verificados por testes, monitoramento e configuração efetiva da infraestrutura. A arquitetura representa uma proposta técnica; a comprovação desses níveis dependerá de validação operacional antes da entrada em produção.

# VittaCare

Diagramas UML – Bloco 4

# Sumário

1. Introdução ..... 1
2. Diagrama de Casos de Uso ..... 2
3. Diagrama de Classes ..... 4
4. Diagramas de Sequência ..... 6
   4.1. Autenticação de usuário ..... 6
   4.2. Agendamento de consulta ..... 8
   4.3. Bloqueio de disponibilidade com agendamento ativo ..... 10
5. Diagrama de Componentes ..... 12
6. Diagrama de Implantação ..... 14
7. Diagramas de Atividades ..... 16
   7.1. Processo de agendamento ..... 16
   7.2. Processo administrativo de cadastro de profissional ..... 18
8. Considerações finais ..... 20

# 1. Introdução

Este documento apresenta a modelagem UML do sistema VittaCare, desenvolvida com o objetivo de representar sua estrutura, seu comportamento e sua organização tecnológica.

A modelagem foi elaborada com base nos documentos de concepção e escopo, requisitos e arquitetura do projeto. Foram utilizados diagramas de casos de uso,

classes, sequência, componentes, implantação e atividades, permitindo visualizar diferentes aspectos do funcionamento do sistema.

A documentação busca demonstrar como os requisitos definidos são traduzidos em elementos estruturais e comportamentais, contribuindo para a compreensão do projeto e orientando seu desenvolvimento.

## 2. Diagrama de Casos de Uso

O diagrama de casos de uso apresenta as principais funcionalidades do VittaCare e suas interações com os atores do sistema.

Foram identificados quatro atores principais:

- **Paciente:** realiza cadastro, consulta especialidades e profissionais, verifica horários disponíveis e gerencia seus agendamentos.
- **Profissional de saúde:** administra sua disponibilidade e consulta seus agendamentos, podendo registrar atendimentos e ausências.
- **Recepcionista:** realiza operações administrativas relacionadas aos pacientes e aos agendamentos, incluindo marcações em nome dos pacientes.
- **Administrador:** gerencia usuários, profissionais, especialidades e permissões de acesso.

As funcionalidades representadas contemplam os processos de autenticação, gerenciamento de usuários, consulta de profissionais, disponibilidade de horários, agendamento, cancelamento, reagendamento e administração do sistema.

Também foram representadas relações de inclusão e extensão entre os casos de uso, quando aplicáveis, para demonstrar dependências e comportamentos complementares entre funcionalidades.

O diagrama permite visualizar a distribuição de responsabilidades entre os atores e delimitar as funcionalidades previstas para o sistema.

**Diagrama de Casos de Uso — VittaCare**
Autores: Gabriela Dias, João Batista | Data: 07/11/2026
Atores: Paciente, Profissional de saúde, Recepcionista, Administrador e Provedor externo de e-mail/SMS. «Usuário» é a generalização dos quatro perfis.



# 3. Diagrama de Classes

O diagrama de classes representa a estrutura estática do VittaCare, identificando as principais entidades do sistema, seus atributos, métodos e relacionamentos.

Foram propostas doze classes de domínio — Clínica, Usuário, Paciente, Profissional de Saúde, Recepcionista, Administrador, Especialidade, Disponibilidade, Agendamento, Atendimento, Notificação e Registro de Auditoria — e seis enumerações: StatusAgendamento, StatusUsuario, StatusDisponibilidade, StatusPresenca, TipoNotificacao e PerfilUsuario.

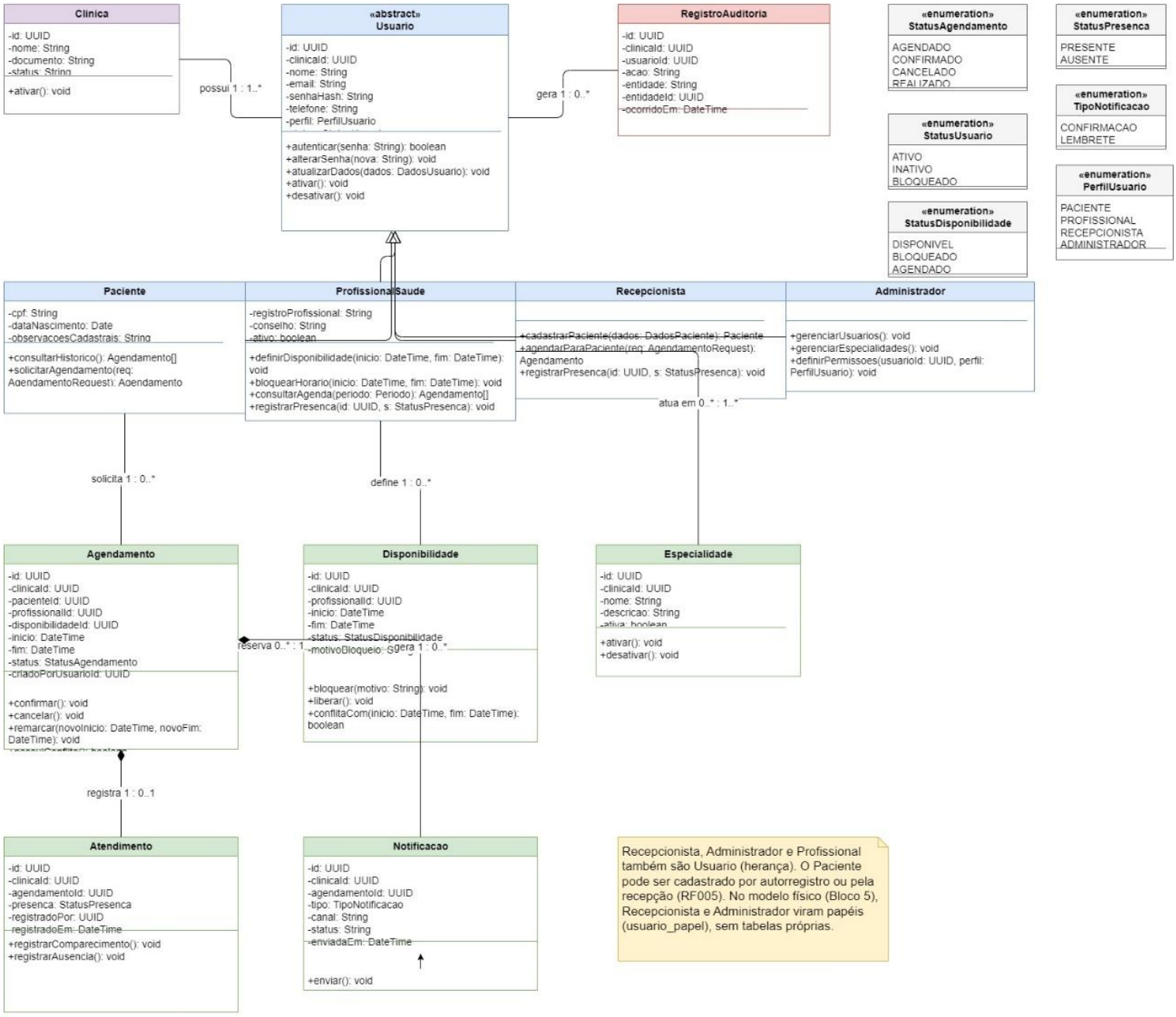
Cada classe apresenta atributos tipados e métodos com suas respectivas visibilidades, conforme a notação UML. Os relacionamentos representam as associações necessárias para estruturar os dados e as interações entre os elementos do sistema.

Paciente, Profissional de Saúde, Recepcionista e Administrador herdam de Usuário, pois compartilham os atributos de autenticação e diferem nas operações que podem executar. No modelo físico (Bloco 5), Recepcionista e Administrador são materializados como papéis e permissões, e Paciente e Profissional como tabelas de perfil associadas ao usuário. Atendimento e Notificação estão ligados ao Agendamento por composição, pois dependem do seu ciclo de vida. Os status seguem os valores definidos no banco de dados: o agendamento passa pelos status AGENDADO, CONFIRMADO, CANCELADO e REALIZADO.

A modelagem considera, especialmente, a necessidade de relacionar pacientes a agendamentos, profissionais às suas especialidades e disponibilidades, além de manter a separação dos dados entre clínicas.

Essa representação fornece a organização das entidades e serve como referência para a implementação das estruturas de dados e das regras de negócio. Os relacionamentos propostos foram validados na definição do modelo de banco de dados (Bloco 5).

**Diagrama de Classes — VittaCare**
Autores: Gabriela Dias, João Batista | Data: 07/11/2026
Entidades de domínio (monólito modular). Todas as entidades de negócio carregam clinicaId (isolamento lógico — ADR-012 / RNF014).



# 4. Diagramas de Sequência

Os diagramas de sequência representam a troca de mensagens entre os componentes do sistema ao longo da execução de processos específicos. Foram elaborados três cenários: autenticação, agendamento e tratamento de uma situação crítica relacionada ao bloqueio de horários.

## 4.1. Autenticação de usuário

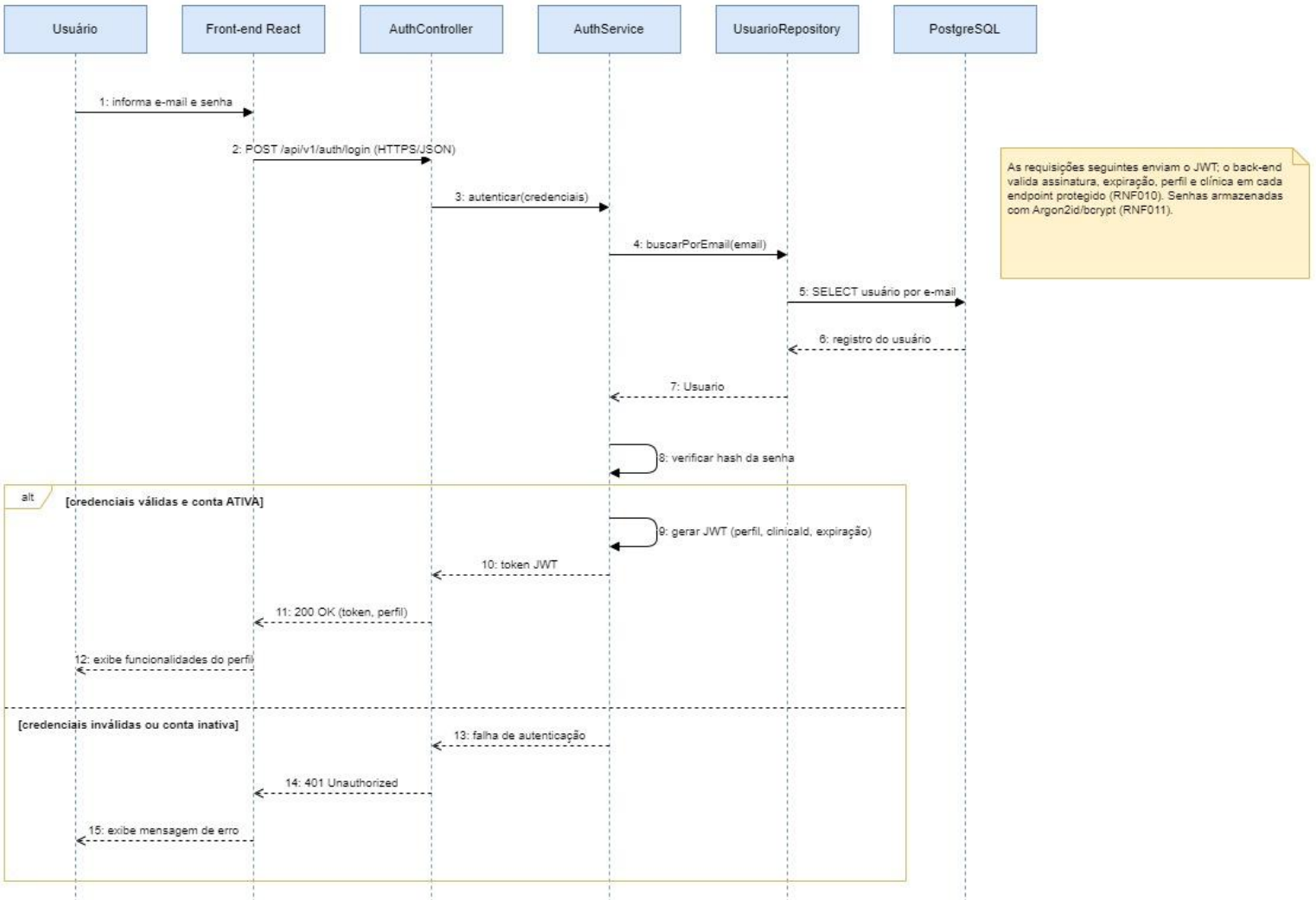
O primeiro diagrama representa o processo de autenticação no VittaCare.

O fluxo começa quando o usuário informa seu e-mail e sua senha na interface do sistema. Essas informações são encaminhadas ao serviço responsável pela autenticação, que consulta os dados necessários no banco de dados e verifica as credenciais.

Caso a autenticação seja válida, o sistema permite o acesso e disponibiliza as funcionalidades correspondentes às permissões do usuário. Caso contrário, a tentativa é recusada e uma mensagem de erro é apresentada.

Esse processo está relacionado principalmente aos requisitos RF001 e RF024, que tratam da autenticação e do gerenciamento de permissões de acesso.

**Diagrama de Sequência — Autenticação de usuário**
Autores: Gabriela Dias, João Batista | Data: 07/11/2026
Requisitos: RF001, RF024, RNF009 a RNF011



## 4.2. Agendamento de consulta

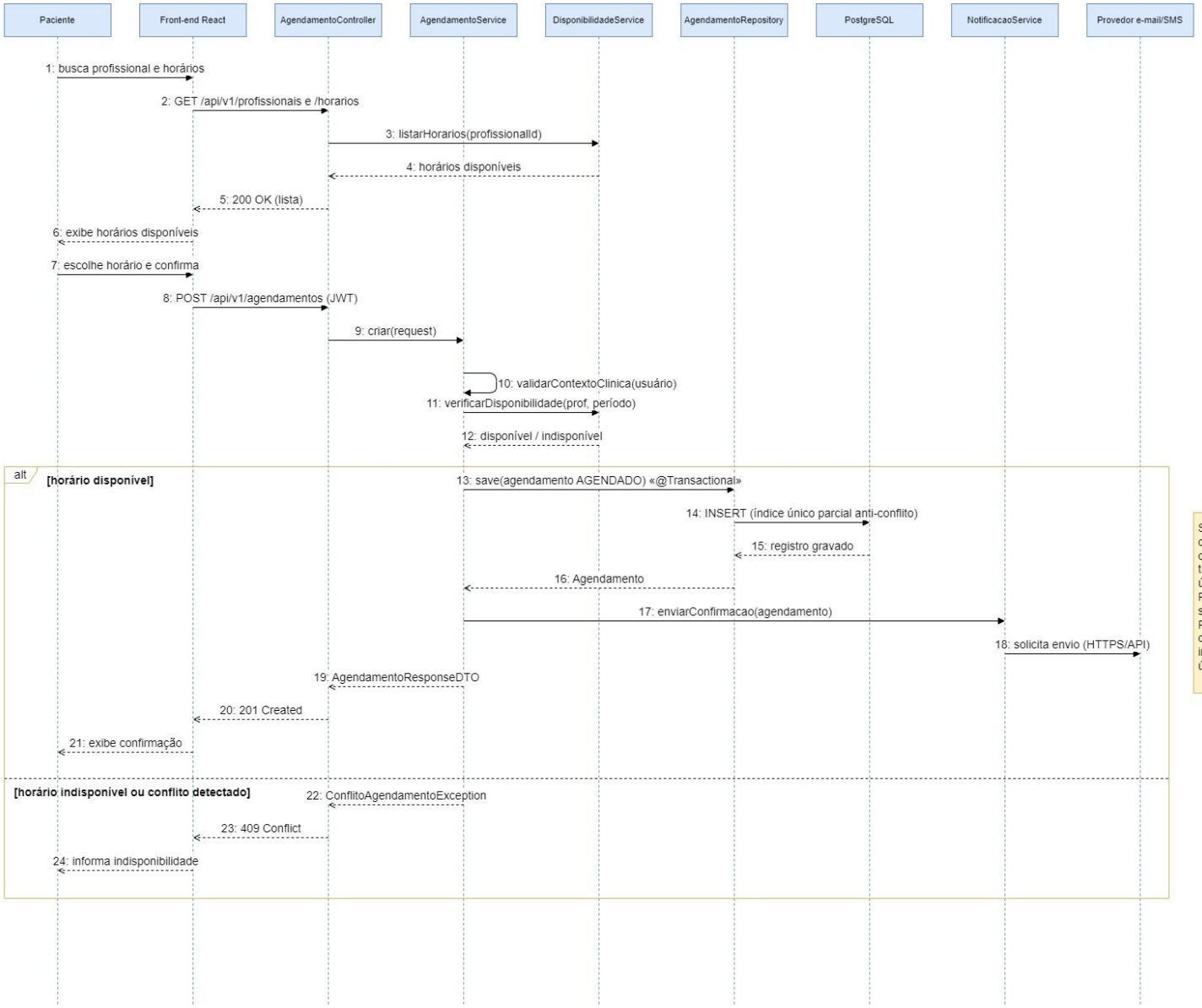
O segundo diagrama apresenta o fluxo principal de agendamento de uma consulta.

Inicialmente, o paciente consulta os profissionais e os horários disponíveis. Após selecionar o profissional e o horário desejado, o sistema verifica se a disponibilidade continua válida.

Se o horário estiver disponível, o agendamento é registrado no banco de dados com status AGENDADO e o sistema envia uma confirmação ao paciente. Caso o horário não esteja mais disponível, o agendamento não é concluído e o usuário recebe uma informação sobre a indisponibilidade.

Esse fluxo contempla os requisitos RF010, RF011, RF012, RF014, RF018 e RF021, relacionados à consulta de profissionais, verificação de horários, realização do agendamento, prevenção de conflitos e confirmação da marcação.

**Diagrama de Sequência — Agendamento de consulta (fluxo principal)**
Autores: Gabriela Dias, João Batista | Data: 07/11/2026
Requisitos: RF010, RF011, RF012, RF014, RF018, RF021, RNF003, RNF008



## 4.3. Bloqueio de disponibilidade com agendamento ativo

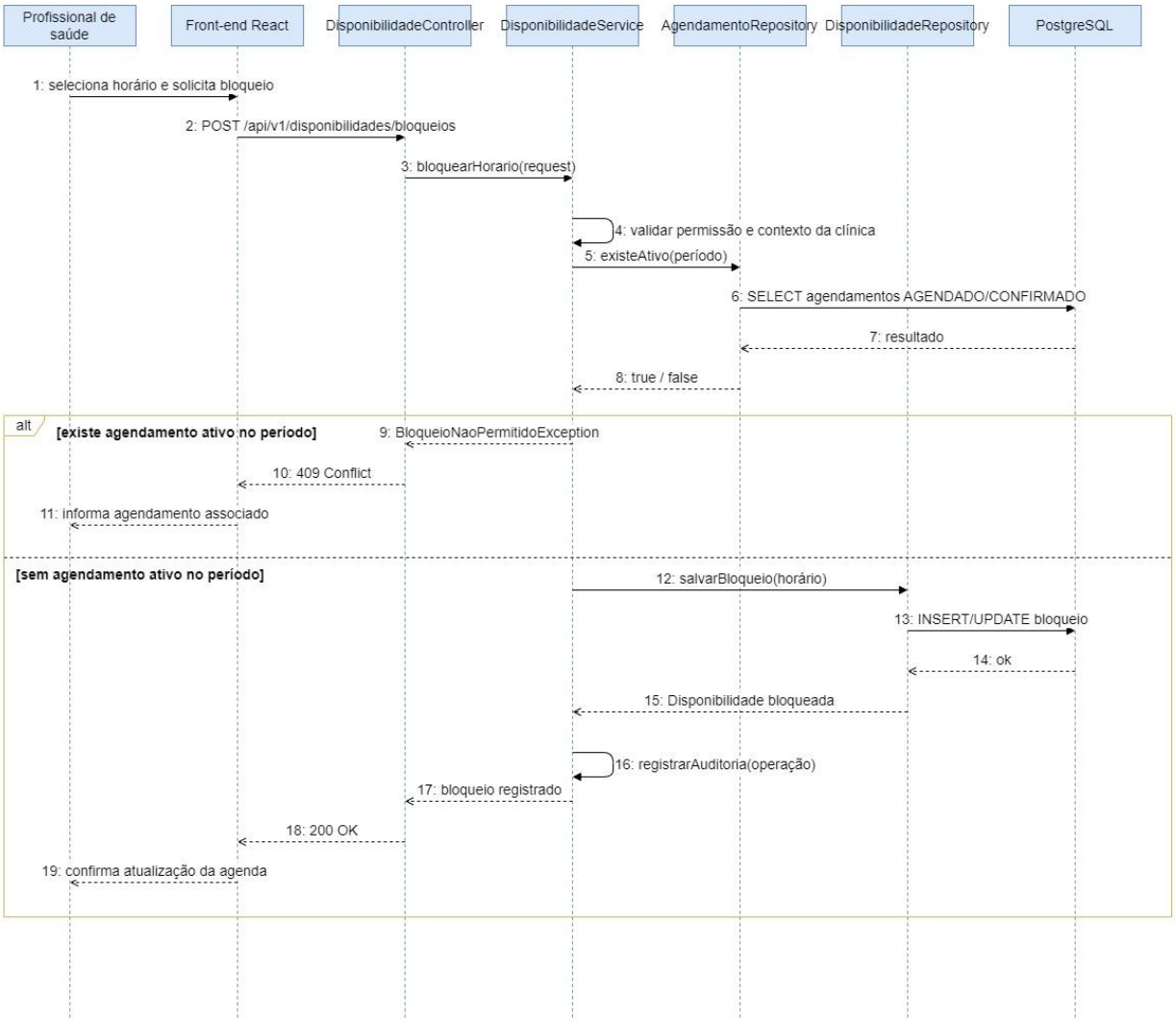
O terceiro diagrama representa um cenário de erro crítico previsto no requisito RF026.

Quando um profissional tenta bloquear determinado horário, o sistema verifica se existe algum agendamento ativo (com status AGENDADO ou CONFIRMADO) associado àquele período.

Caso exista, o bloqueio é impedido e o sistema informa ao usuário que não é possível realizar a operação (resposta HTTP 409). Dessa forma, evita-se que uma disponibilidade seja bloqueada quando já existe um compromisso registrado.

A verificação e a gravação do bloqueio ocorrem na mesma transação, com bloqueio de linha, para impedir que um novo agendamento seja criado entre as duas etapas (RNF008). Esse comportamento preserva a integridade dos agendamentos e garante o cumprimento da regra de negócio estabelecida no requisito RF026.

**Diagrama de Sequência — Bloqueio de disponibilidade com agendamento ativo (erro crítico)**
Autores: Gabriela Dias, João Batista | Data: 07/11/2026
Requisitos: RF013, RF026, RNF008




A verificação (passo 5) e a gravação do bloqueio (passo 12) devem ocorrer na mesma transação, com bloqueio de linha, para impedir que um novo agendamento entre no intervalo entre as duas etapas (RNF008).

# 5. Diagrama de Componentes

O diagrama de componentes apresenta a organização lógica do VittaCare, destacando seus principais módulos e as dependências entre eles.

A arquitetura segue o modelo de monólito modular com separação em camadas, conforme definido na documentação de arquitetura.

Foram representados os seguintes componentes:

responsável pela interação com pacientes, profissionais, recepcionistas e administradores.

responsável pela validação de credenciais e controle de acesso.

responsável pelo gerenciamento das contas e informações cadastrais.

responsável pelo cadastro e pela associação entre profissionais e especialidades.

responsável pelo gerenciamento dos horários disponíveis e bloqueados.

responsável pela criação, consulta, cancelamento e reagendamento de consultas.

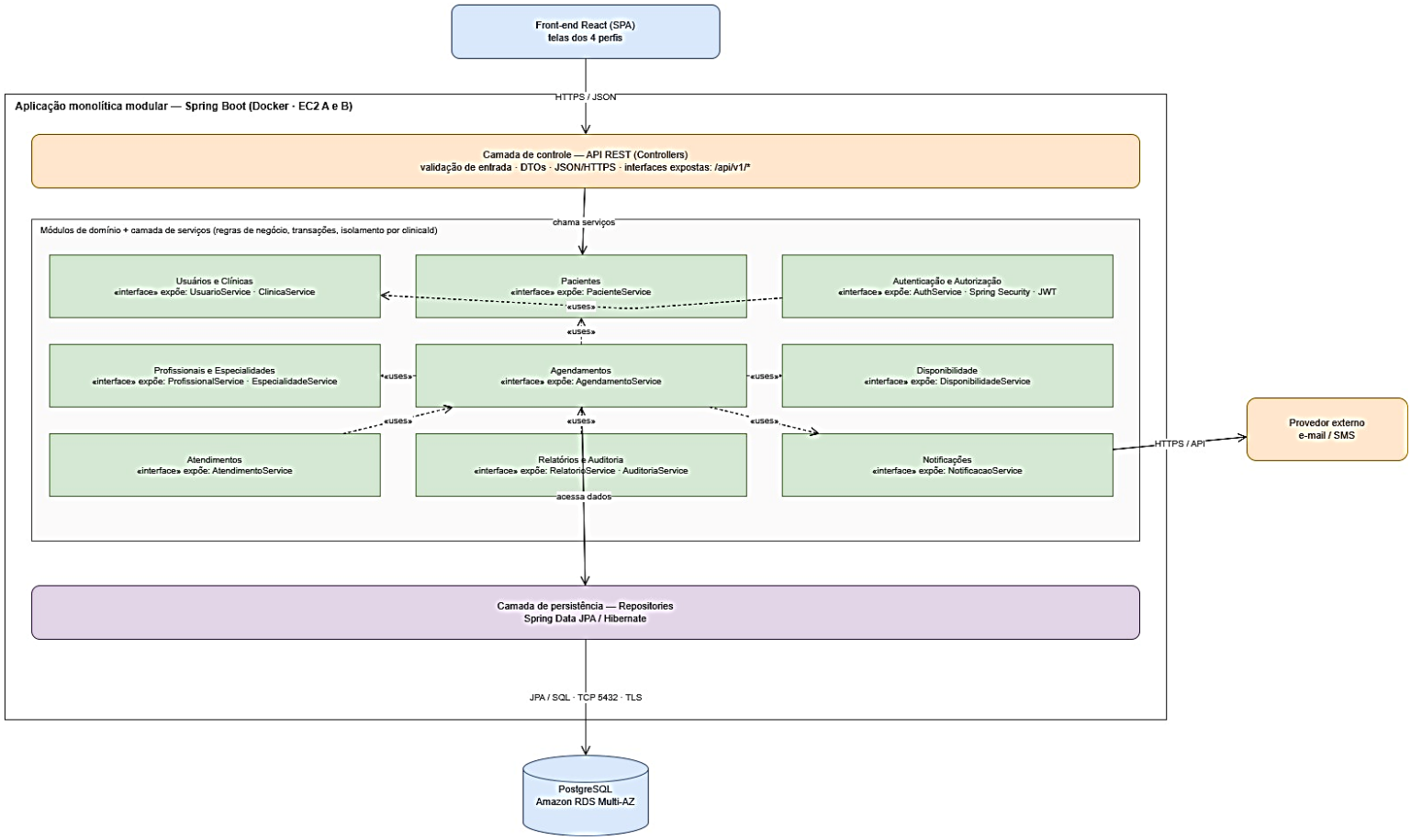
responsável pelas confirmações e pelos lembretes de consultas.

responsável pela geração de informações administrativas.

responsável pelo armazenamento persistente das informações do sistema.

A comunicação entre a interface e o back-end ocorre por meio de serviços e interfaces REST, utilizando JSON. Os módulos compartilham a estrutura interna da aplicação, mantendo suas responsabilidades organizadas e reduzindo o acoplamento entre funcionalidades.

**Diagrama de Componentes — VittaCare**
Autores: Gabriela Dias, João Batista | Data: 07/11/2026
Monólito modular em camadas (ADR-001). Setas tracejadas = dependência via interfaces/serviços internos; módulos não acessam repositories de outros módulos.



# 6. Diagrama de Implantação

O diagrama de implantação representa a distribuição física e lógica dos elementos necessários para executar o VittaCare.

A infraestrutura considera a utilização de serviços em nuvem AWS, conforme indicado na arquitetura do projeto, além da separação dos ambientes de desenvolvimento, homologação e produção.

Foram representados os principais elementos da implantação:

dispositivo utilizado para acessar a aplicação por meio de um navegador.

interface responsável pela apresentação das funcionalidades.

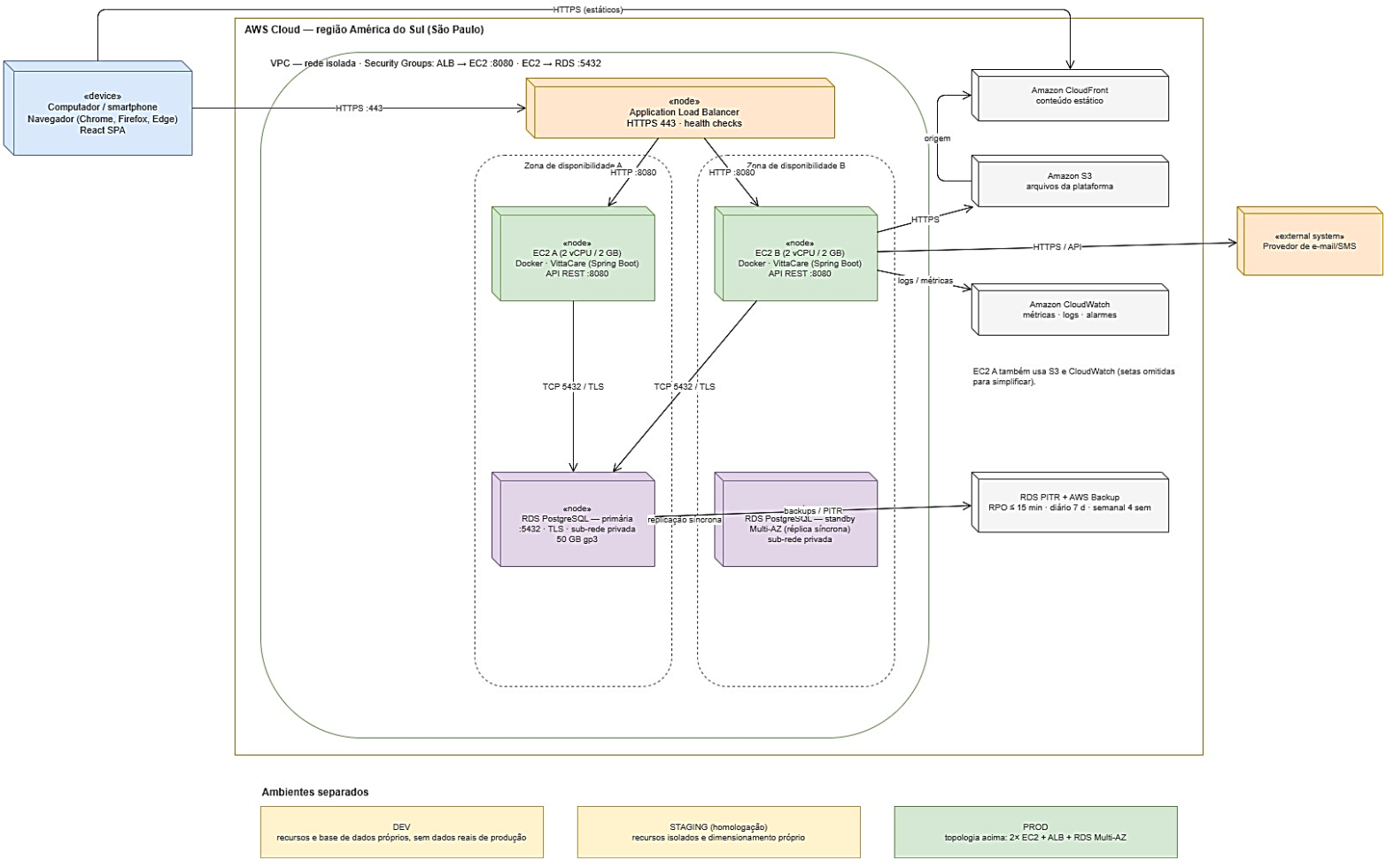
aplicação responsável pelo processamento das regras de negócio e pela comunicação com os demais serviços. responsável pelo armazenamento das informações.

utilizados para integrações necessárias, como o envio de notificações.

A comunicação entre os elementos ocorre por meio de protocolos de rede adequados à função de cada componente. A representação também considera a separação dos ambientes, permitindo que alterações sejam desenvolvidas e testadas antes de serem disponibilizadas em produção.

As portas apresentadas no diagrama representam uma proposta de configuração técnica e deverão ser confirmadas durante a implantação efetiva da infraestrutura.

**Diagrama de Implantação — VittaCare (produção na AWS)**
Autores: Gabriela Dias, João Batista | Data: 07/11/2026
Topologia de referência para o RNF006 (99,9%). Portas e dimensionamentos são propostas iniciais, a validar na implantação.




**Ambientes separados**

| DEV | STAGING (homologação) | PROD |
|---|---|---|
| recursos e base de dados próprios, sem dados reais de produção | recursos isolados e dimensionamento próprio | topologia acima: 2× EC2 + ALB + RDS Multi-AZ |

# 7. Diagramas de Atividades

Os diagramas de atividades representam os fluxos de trabalho do sistema, destacando as etapas, decisões e possíveis caminhos de execução dos processos.

## 7.1. Processo de agendamento

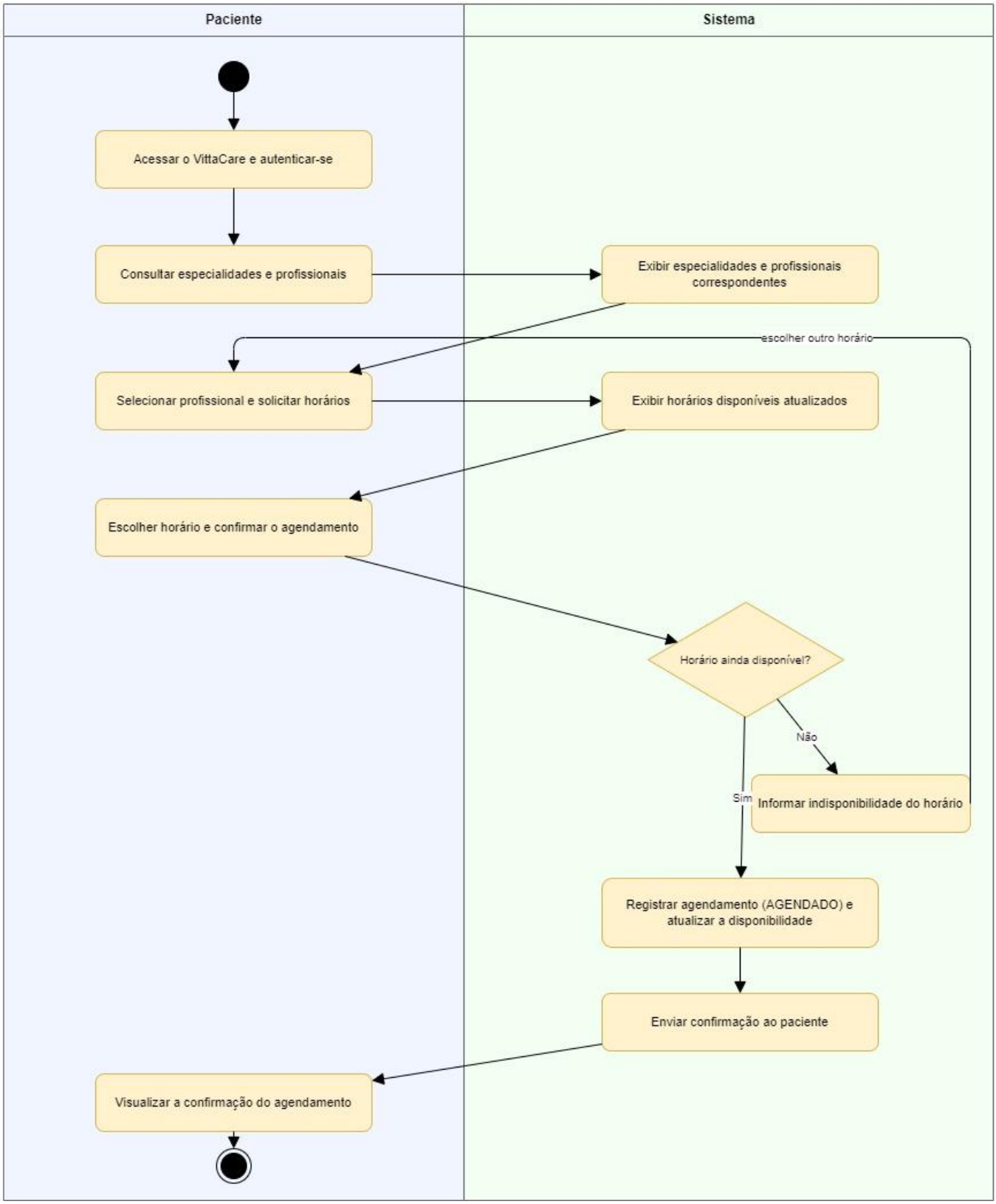
O primeiro diagrama representa o processo de agendamento de consultas.

O fluxo inicia com o acesso do paciente ao sistema, seguido da consulta de especialidades e profissionais disponíveis. Após selecionar um profissional, o paciente consulta os horários e escolhe aquele que deseja agendar.

O sistema verifica a disponibilidade do horário selecionado. Se estiver disponível, o agendamento é registrado e uma confirmação é enviada. Caso contrário, o usuário deve selecionar outra opção.

O processo demonstra as etapas necessárias para concluir um agendamento e evidencia a importância da validação de disponibilidade antes da confirmação.

**Diagrama de Atividades — Processo de agendamento de consulta**
Autores: Gabriela Dias, João Batista | Data: 07/11/2026
Raias: Paciente e Sistema (RF010 a RF012, RF014, RF018, RF021)



## 7.2. Processo administrativo de cadastro de profissional

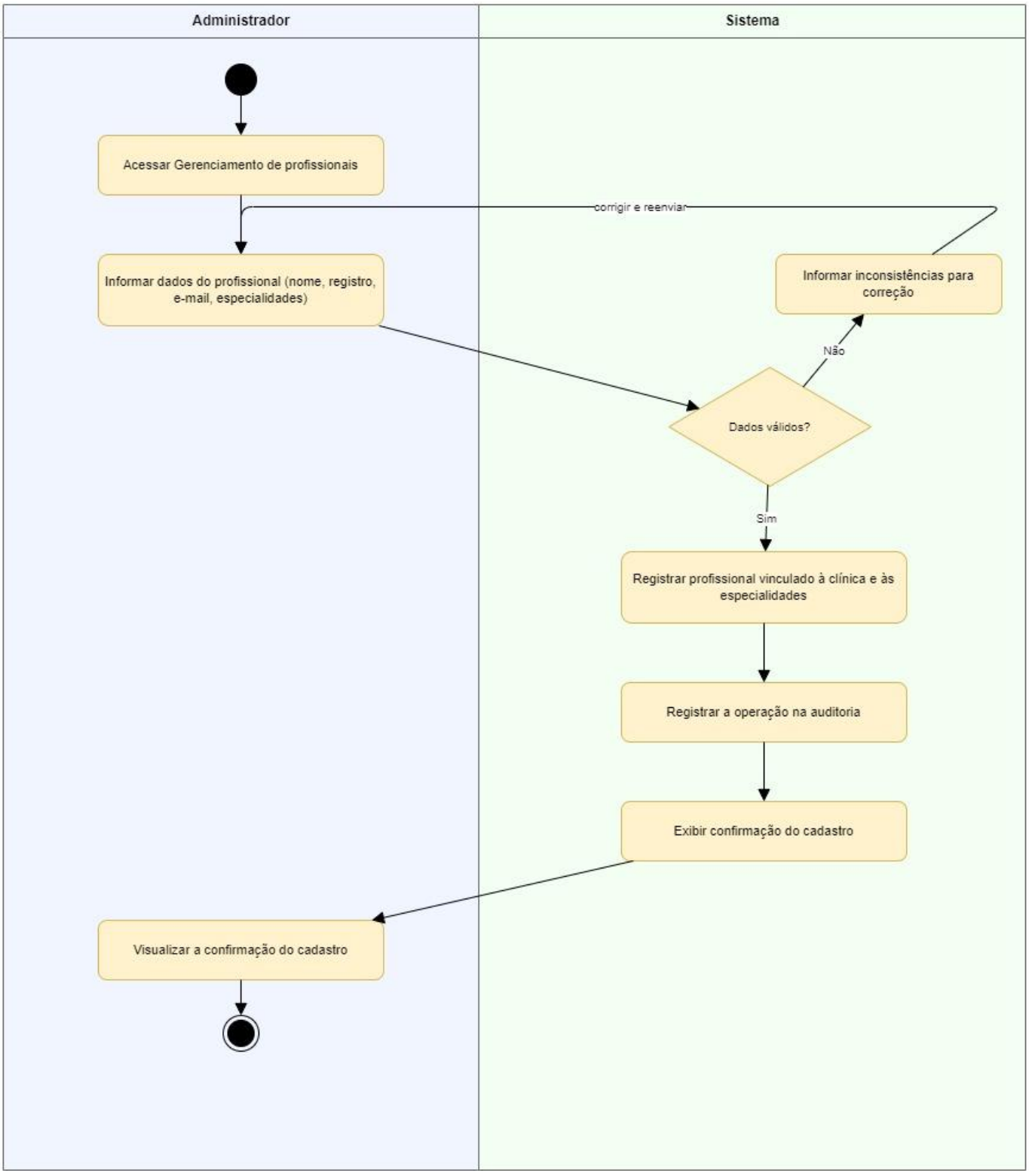
O segundo diagrama representa um processo administrativo relacionado ao cadastro de profissionais de saúde.

O fluxo começa quando o administrador acessa a funcionalidade de gerenciamento de profissionais e informa os dados necessários para o cadastro.

Em seguida, o sistema valida as informações fornecidas. Caso os dados estejam corretos, o cadastro é registrado. Caso sejam identificadas inconsistências, o sistema informa a necessidade de correção antes da conclusão da operação.

Esse processo demonstra a atuação administrativa na manutenção dos dados dos profissionais e contribui para a organização das informações utilizadas nas demais funcionalidades.

**Diagrama de Atividades — Cadastro de profissional de saúde (processo administrativo)**
Autores: Gabriela Dias, João Batista | Data: 07/11/2026
Raias: Administrador e Sistema (RF007, RF008, RF009, RNF013, US018)



# 8. Considerações finais

A modelagem UML desenvolvida para o VittaCare permite representar os principais aspectos estruturais, funcionais e tecnológicos do sistema.

Os diagramas de casos de uso demonstram as funcionalidades e os atores envolvidos; o diagrama de classes apresenta a estrutura das entidades; os diagramas de sequência detalham interações importantes; o diagrama de componentes representa a organização modular; o diagrama de implantação apresenta a infraestrutura prevista; e os diagramas de atividades descrevem os principais fluxos operacionais e administrativos.

Em conjunto, essas representações contribuem para reduzir ambiguidades, facilitar a comunicação entre os integrantes da equipe e orientar as etapas de implementação e validação do sistema.

# VittaCare

Banco de Dados – Bloco 5

Gabriela Dias Magalhães

# Sumário

1. Introdução ..... 2
2.Decisão Tecnológica do Banco de Dados ..... 2
2.1 Justificativa técnica ..... 3
2.2 Decisão registrada ..... 4
3. Modelo conceitual (ER) ..... 4
3.1 Entidades do domínio ..... 5
3.2 Relacionamentos e cardinalidades ..... 6
3.3 Representação textual do ER ..... 7
4. Modelo lógico ..... 7
4.1 Convenções ..... 8
4.2 Estrutura lógica resumida ..... 8
4.3 Índices e justificativas ..... 10
5. Modelo físico — SQL executável ..... 10
5.1 Regras de execução e concorrência ..... 13
6. Dicionário de dados ..... 14
clinica ..... 14
usuario ..... 14
paciente ..... 15
profissional ..... 16
especialidade ..... 16
profissional_especialidade ..... 16
disponibilidade ..... 17
agendamento ..... 17
atendimento ..... 18
notificacao ..... 19
papel / permissao / usuario_papel / papel_permissao ..... 19
registro_auditoria ..... 19
7. Estratégia de dados: backup, LGPD e migrations ..... 20
7.1 Backup e recuperação ..... 20
7.2 LGPD e proteção de dados ..... 21

7.3 Migrations e evolução do esquema ..... 23
7.4 Estratégia para migrations sem downtime ..... 23
8. Regras de integridade e rastreabilidade ..... 24
8.1 Pontos que exigem validação na implementação..... 25
9. Considerações finais ..... 26

# 1. Introdução

Este documento apresenta a modelagem de dados do VittaCare, plataforma web SaaS destinada ao gerenciamento de clínicas e ambulatórios. O objetivo é transformar as entidades e regras identificadas nos documentos de concepção, requisitos, arquitetura e UML em uma estrutura relacional consistente, segura e implementável.

A modelagem contempla pacientes, usuários, profissionais de saúde, especialidades, disponibilidades, agendamentos, atendimentos, notificações e recursos administrativos. O sistema não armazenará prontuários eletrônicos completos, prescrições, diagnósticos automatizados nem dados de pagamento, pois essas funcionalidades estão fora do escopo definido para a versão atual.

A proposta adota banco compartilhado com isolamento lógico por clínica, utilizando a coluna clinica_id nas tabelas que pertencem a uma instituição. O contexto da clínica deverá ser obtido a partir da identidade autenticada no back-end; um clinica_id enviado pelo front-end nunca será considerado prova de autorização.

O modelo foi elaborado para atender, principalmente, aos requisitos RF001–RF026, com destaque para autenticação, gerenciamento de cadastros, disponibilidade, prevenção de agendamentos duplicados, registro de comparecimento, permissões e isolamento entre clínicas.

# 2.Decisão Tecnológica do Banco de Dados

A escolha do banco deve acompanhar a arquitetura definida no Bloco 3. Como o VittaCare utiliza Java com Spring Boot, Spring Data JPA/Hibernate e uma arquitetura de monólito modular, foi selecionado um banco relacional compatível com transações, chaves estrangeiras, restrições de integridade e consultas relacionais.

| Critério | PostgreSQL — escolhido | MongoDB — alternativa descartada | MySQL — alternativa avaliada |
|---|---|---|---|
| Modelo de dados | Relacional, adequado às entidades e associações do domínio. | Documento, exigiria tratamento adicional para relações e integridade entre documentos. | Relacional e tecnicamente viável. |
| Integridade | Suporte a PK, FK, UNIQUE, CHECK, transações e índices. | Não oferece o mesmo conjunto de constraints relacionais para este modelo. | Suporta PK, FK, constraints e transações. |
| Agendamentos | Permite transações e índices únicos para reduzir conflitos concorrentes. | Regras de integridade entre documentos exigiriam maior implementação na aplicação. | Também atende às operações transacionais. |
| Multi-tenant | Permite FKs compostas e constraints envolvendo clinica_id. | O isolamento dependeria mais de validações implementadas pela aplicação. | Também permite isolamento lógico relacional. |
| Decisão | Selecionado por aderência direta à arquitetura e às regras do domínio. | Descartado por menor aderência ao conjunto de relações e constraints exigidas. | Não oferece índice único parcial nativo, necessário para impedir mais de um agendamento ativo por disponibilidade |

## 2.1 Justificativa técnica

- O domínio possui relações explícitas entre clínicas, usuários, pacientes, profissionais, especialidades, disponibilidades e agendamentos.
- A confirmação de uma consulta exige consistência transacional e prevenção de duas marcações ativas para o mesmo horário.
- O isolamento lógico por clínica pode ser reforçado por chaves estrangeiras compostas que associam os registros ao mesmo clinica_id.
- PostgreSQL é compatível com Spring Data JPA/Hibernate e com a hospedagem prevista no Amazon RDS Multi-AZ.

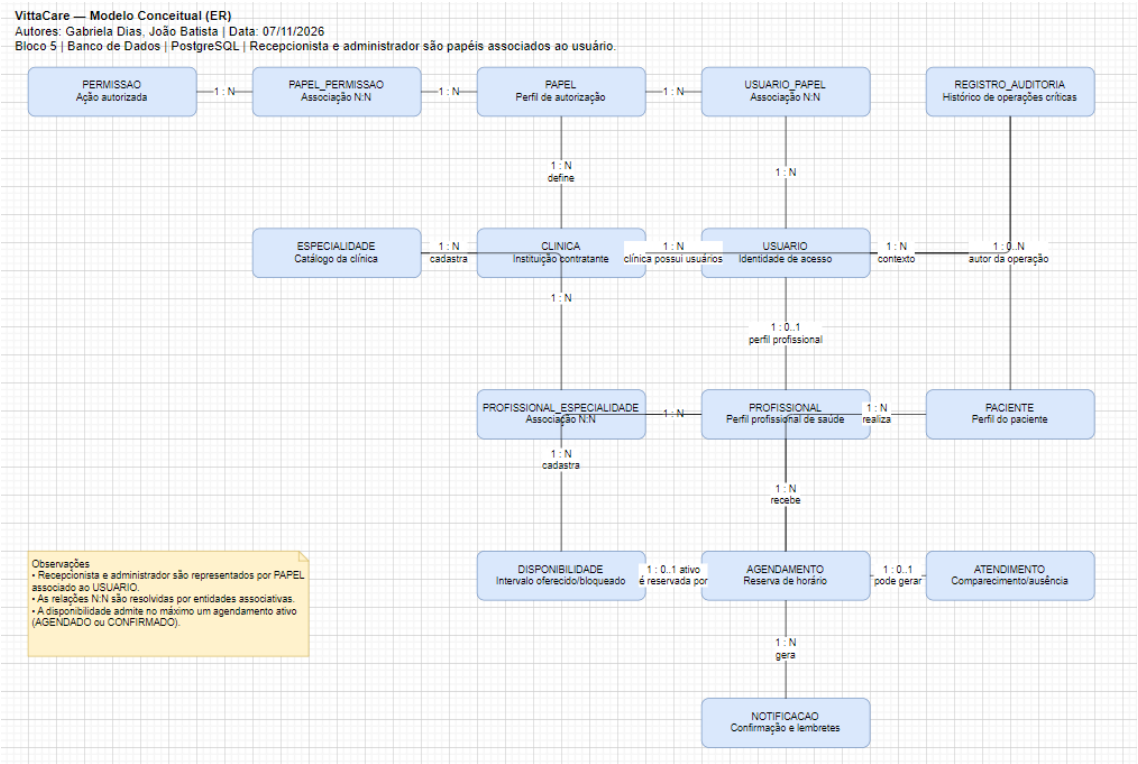
Alternativas: MongoDB foi descartado por não oferecer o mesmo modelo relacional e conjunto de constraints que atendem diretamente às relações e à integridade do domínio. MySQL foi considerado, mas não apresentou vantagem suficiente para substituir a tecnologia já definida no Bloco 3.

## 2.2 Decisão registrada

| Item | Definição |
|---|---|
| Banco de dados | PostgreSQL |
| Hospedagem | Amazon RDS, configuração Multi-AZ |
| Persistência na aplicação | Spring Data JPA / Hibernate |
| Modelo de tenancy | Banco compartilhado com isolamento lógico por clinica_id |
| Estratégia de integridade | PKs, FKs, FKs compostas, UNIQUE, CHECK e transações |
| Responsável pelas regras | Back-end Spring Boot; o banco reforça as regras por constraints |

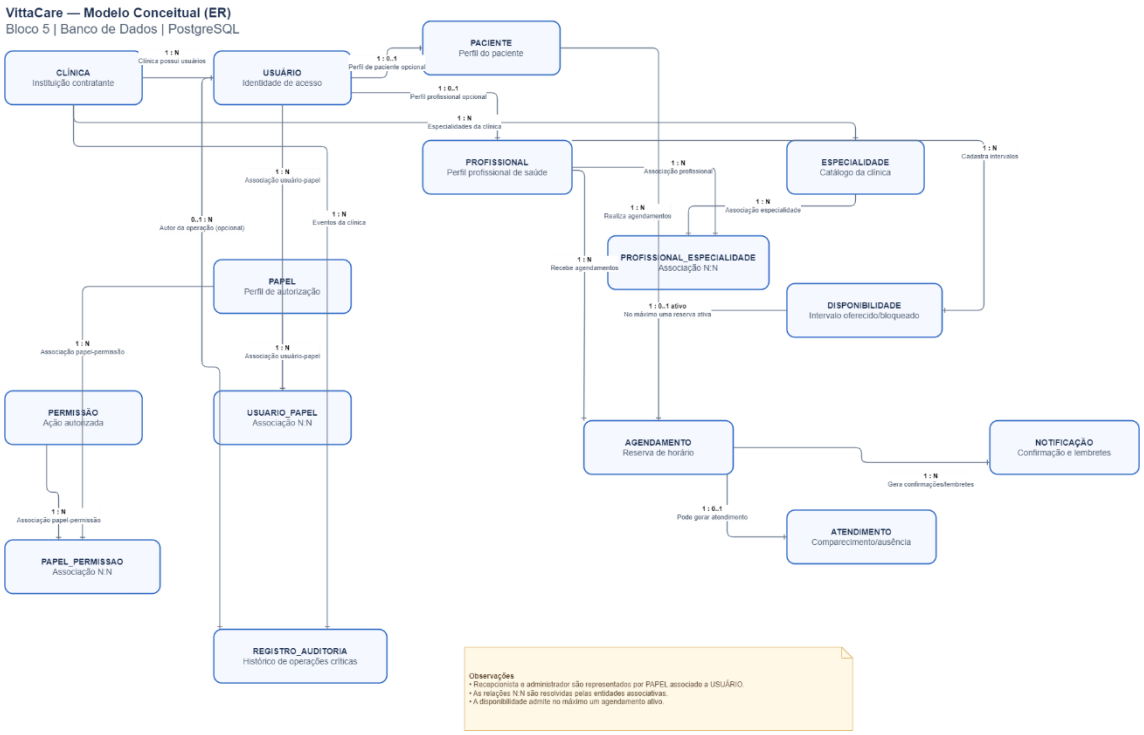
# 3. Modelo conceitual (ER)

O modelo conceitual identifica as entidades do domínio e seus relacionamentos, sem se prender inicialmente aos detalhes de implementação. As cardinalidades representam as regras previstas para a versão atual do VittaCare.



## 3.1 Entidades do domínio

| Entidade | Descrição de negócio |
|---|---|
| Clínica | Instituição contratante que delimita o conjunto de dados acessível no sistema. |
| Usuário | Identidade de acesso, com e-mail, senha protegida, situação da conta e vínculo com a clínica. |
| Paciente | Perfil de paciente associado a uma identidade de usuário. |
| Profissional de saúde | Perfil profissional responsável por disponibilidades e consultas. |
| Especialidade | Especialidade médica cadastrada para uma clínica. |
| Profissional_Especialidade | Associação entre profissionais e suas especialidades. |
| Disponibilidade | Intervalo de horário oferecido ou bloqueado por um profissional. |
| Agendamento | Reserva de uma disponibilidade para um paciente. |
| Atendimento | Registro de comparecimento ou ausência vinculado ao agendamento. |
| Notificação | Confirmação ou lembrete relacionado a um agendamento. |
| Papel e Permissão | Estrutura de autorização para diferenciar paciente, profissional, recepcionista e administrador. |
| Registro de auditoria | Histórico de operações administrativas e críticas. |




## 3.2 Relacionamentos e cardinalidades

| Relacionamento | Cardinalidade | Regra |
|---|---|---|
| Clínica — Usuário | 1 : N | Uma clínica possui vários usuários; cada usuário pertence a uma clínica na versão inicial. |
| Usuário — Paciente | 1 : 0..1 | Um usuário pode possuir um perfil de paciente. |
| Usuário — Profissional | 1 : 0..1 | Um usuário pode possuir um perfil profissional. |
| Clínica — Especialidade | 1 : N | Cada especialidade cadastrada pertence a uma clínica. |
| Profissional — Especialidade | N : N | Um profissional pode possuir várias especialidades e uma especialidade pode estar associada a vários profissionais. |
| Profissional — Disponibilidade | 1 : N | Um profissional cadastra vários intervalos de disponibilidade. |
| Paciente — Agendamento | 1 : N | Um paciente pode realizar vários agendamentos. |

| | | |
|---|---|---|
| Profissional — Agendamento | 1 : N | Um profissional pode receber vários agendamentos em horários distintos. |
| Disponibilidade — Agendamento | 1 : 0..1 ativo | Uma disponibilidade pode ter no máximo um agendamento ativo. |
| Agendamento — Atendimento | 1 : 0..1 | Um agendamento pode gerar um registro de comparecimento ou ausência. |
| Agendamento — Notificação | 1 : N | Um agendamento pode gerar confirmação e lembretes. |
| Usuário — Papel | N : N | Um usuário pode receber um ou mais papéis, conforme autorização administrativa. |
| Papel — Permissão | N : N | Cada papel reúne as permissões que pode executar. |

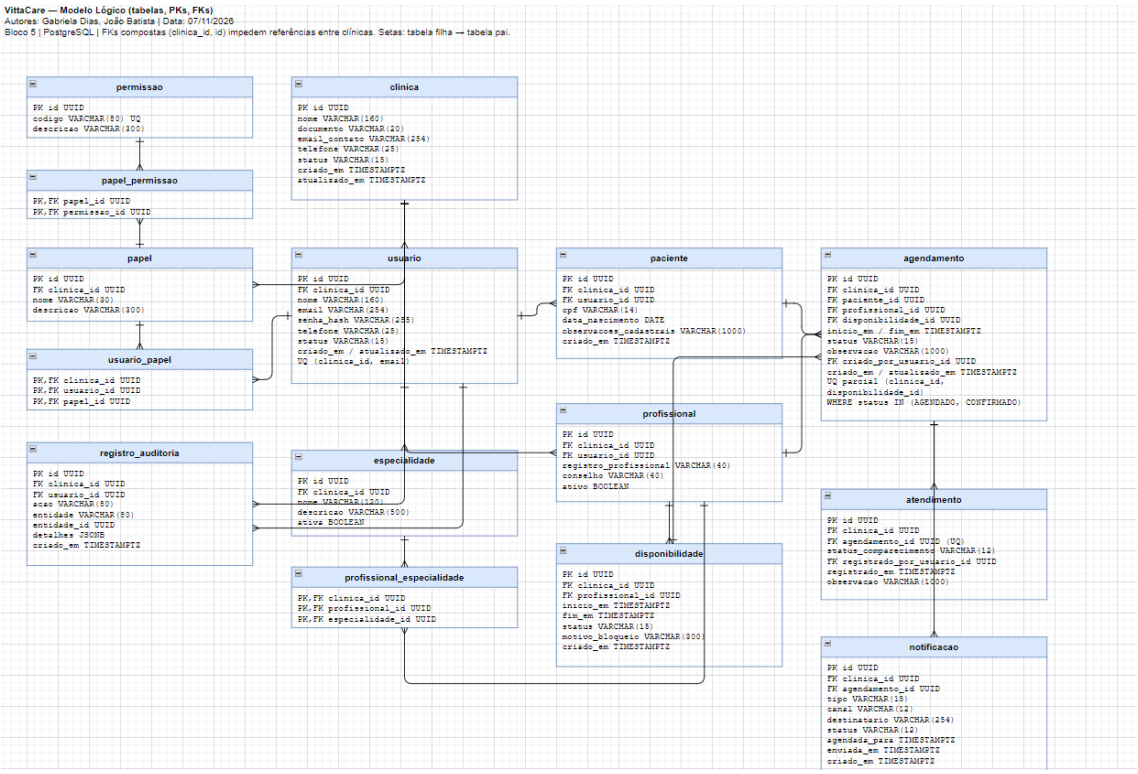
### 3.3 Representação textual do ER

CLINICA 1——N USUARIO; USUARIO 1——0..1 PACIENTE; USUARIO 1——0..1 PROFISSIONAL; CLINICA 1——N ESPECIALIDADE; PROFISSIONAL N——N ESPECIALIDADE (por PROFISSIONAL_ESPECIALIDADE); PROFISSIONAL 1——N DISPONIBILIDADE; PACIENTE 1——N AGENDAMENTO; PROFISSIONAL 1——N AGENDAMENTO; DISPONIBILIDADE 1——0..1 AGENDAMENTO ATIVO; AGENDAMENTO 1——0..1 ATENDIMENTO; AGENDAMENTO 1——N NOTIFICACAO; USUARIO N——N PAPEL (por USUARIO_PAPEL); PAPEL N——N PERMISSAO (por PAPEL_PERMISSAO).

Observação de mapeamento UML: recepcionista e administrador são representados como papéis associados ao usuário, em vez de tabelas com dados duplicados. Essa decisão mantém os atributos comuns de autenticação em USUARIO e concentra as diferenças de acesso na estrutura de autorização.

## 4. Modelo lógico

O modelo lógico converte as entidades em tabelas relacionais, definindo atributos, tipos, chaves primárias (PK), chaves estrangeiras (FK), restrições e índices. Os identificadores são UUID para facilitar a geração de IDs pela aplicação Java e evitar dependência de sequências numéricas distribuídas.




## 4.1 Convenções

• Todos os identificadores principais utilizam UUID.

• Tabelas pertencentes a uma clínica possuem clinica_id e, quando aplicável, FKs compostas (clinica_id, id) para impedir referências cruzadas entre instituições.

• Datas e horários são armazenados com TIMESTAMPTZ, mantendo referência de fuso horário.

• Valores monetários não fazem parte do escopo atual do sistema.

• Senhas são armazenadas somente como hash; nunca em texto puro.

• Campos de situação utilizam VARCHAR com CHECK para restringir os valores permitidos.

• O modelo lógico está normalizado até a Terceira Forma Normal (3FN): cada tabela descreve uma única entidade, os atributos dependem somente da chave primária e os dados que se repetem (especialidade, papel e permissão) ficam em tabelas próprias. Evitou-se a desnormalização para preservar a integridade e reduzir anomalias de atualização.

## 4.2 Estrutura lógica resumida

| Tabela | PK | Principais FKs | Finalidade |
|---|---|---|---|
| clinica | id | — | Organização contratante. |

| | | | |
|---|---|---|---|
| usuario | id | clinica_id → clinica | Identidade, autenticação e estado da conta. |
| paciente | id | (clinica_id, usuario_id) → usuario | Dados cadastrais do paciente. |
| profissional | id | (clinica_id, usuario_id) → usuario | Dados profissionais. |
| especialidade | id | clinica_id → clinica | Catálogo de especialidades por clínica. |
| profissional_especialidade | (profissional_id, especialidade_id) | FKs compostas para profissional e especialidade | Associação N:N. |
| disponibilidade | id | (clinica_id, profissional_id) → profissional | Intervalos disponíveis ou bloqueados. |
| agendamento | id | FKs para clínica, paciente, profissional e disponibilidade | Reserva e situação da consulta. |
| atendimento | id | (clinica_id, agendamento_id) → agendamento | Comparecimento ou ausência. |
| notificacao | id | (clinica_id, agendamento_id) → agendamento | Confirmações e lembretes. |
| papel | id | clinica_id → clinica | Papéis de acesso. |
| permissao | id | — | Permissões cadastradas. |
| usuario_papel | (usuario_id, papel_id) | FKs compostas | Papéis atribuídos a usuários. |
| papel_permissao | (papel_id, permissao_id) | FKs | Permissões de cada papel. |
| registro_auditoria | id | FKs para clínica e usuário | Rastreabilidade de operações críticas. |

## 4.3 Índices e justificativas

| Índice | Justificativa |
|---|---|
| usuario (clinica_id, email) | Acelera autenticação e impede e-mails duplicados dentro da mesma clínica. |
| profissional_especialidade (clinica_id, especialidade_id) | Apoia a pesquisa de profissionais por especialidade. |
| disponibilidade (clinica_id, profissional_id, inicio_em) | Apoia a consulta de horários de um profissional em ordem cronológica. |
| agendamento (clinica_id, paciente_id, inicio_em) | Apoia o histórico de consultas do paciente. |
| agendamento (clinica_id, profissional_id, inicio_em) | Apoia a agenda do profissional. |
| Índice único parcial em agendamento(disponibilidade_id) | Impede mais de um agendamento ativo para a mesma disponibilidade. |
| notificacao (clinica_id, agendamento_id, status) | Apoia o processamento de notificações pendentes. |
| registro_auditoria (clinica_id, criado_em) | Apoia consultas de auditoria por clínica e período. |

# 5. Modelo físico — SQL executável

O script a seguir utiliza sintaxe PostgreSQL. Ele cria as tabelas principais, chaves, restrições e índices. A aplicação deverá gerar os UUIDs e executar as operações por meio do Spring Data JPA/Hibernate ou migrations versionadas.

```
CREATE TABLE clinica (
  id UUID PRIMARY KEY,
  nome VARCHAR(160) NOT NULL,
  documento VARCHAR(20),
  email_contato VARCHAR(254),
  telefone VARCHAR(25),
  status VARCHAR(15) NOT NULL DEFAULT 'ATIVA'
    CHECK (status IN ('ATIVA', 'INATIVA')),
  criado_em TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  atualizado_em TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE usuario (
  id UUID PRIMARY KEY,
  clinica_id UUID NOT NULL REFERENCES clinica(id) ON DELETE RESTRICT,
  nome VARCHAR(160) NOT NULL,
  email VARCHAR(254) NOT NULL,
  senha_hash VARCHAR(255) NOT NULL,
  telefone VARCHAR(25),
  status VARCHAR(15) NOT NULL DEFAULT 'ATIVO'
    CHECK (status IN ('ATIVO', 'INATIVO', 'BLOQUEADO')),
  criado_em TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  atualizado_em TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT uq_usuario_clinica_id UNIQUE (clinica_id, id),
  CONSTRAINT uq_usuario_email_clinica UNIQUE (clinica_id, email)
);

CREATE TABLE paciente (
 id UUID PRIMARY KEY,
```

```
 clinica_id UUID NOT NULL,
 usuario_id UUID NOT NULL,
 cpf VARCHAR(14),
 data_nascimento DATE,
 observacoes_cadastrais VARCHAR(1000),
 criado_em TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
 CONSTRAINT uq_paciente_clinica_id UNIQUE (clinica_id, id),
 CONSTRAINT uq_paciente_usuario UNIQUE (clinica_id, usuario_id),
 CONSTRAINT uq_paciente_cpf UNIQUE (clinica_id, cpf),
 CONSTRAINT fk_paciente_usuario
 FOREIGN KEY (clinica_id, usuario_id)
 REFERENCES usuario (clinica_id, id) ON DELETE RESTRICT
);

CREATE TABLE profissional (
  id UUID PRIMARY KEY,
  clinica_id UUID NOT NULL,
  usuario_id UUID NOT NULL,
  registro_profissional VARCHAR(40),
  conselho VARCHAR(40),
  ativo BOOLEAN NOT NULL DEFAULT TRUE,
  criado_em TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT uq_profissional_clinica_id UNIQUE (clinica_id, id),
  CONSTRAINT uq_profissional_usuario UNIQUE (clinica_id, usuario_id),
  CONSTRAINT fk_profissional_usuario
    FOREIGN KEY (clinica_id, usuario_id)
    REFERENCES usuario (clinica_id, id) ON DELETE RESTRICT
);

CREATE TABLE especialidade (
  id UUID PRIMARY KEY,
  clinica_id UUID NOT NULL REFERENCES clinica(id) ON DELETE RESTRICT,
  nome VARCHAR(120) NOT NULL,
  descricao VARCHAR(500),
  ativa BOOLEAN NOT NULL DEFAULT TRUE,
  CONSTRAINT uq_especialidade_clinica_id UNIQUE (clinica_id, id),
  CONSTRAINT uq_especialidade_nome UNIQUE (clinica_id, nome)
);

CREATE TABLE profissional_especialidade (
  clinica_id UUID NOT NULL,
  profissional_id UUID NOT NULL,
  especialidade_id UUID NOT NULL,
  PRIMARY KEY (clinica_id, profissional_id, especialidade_id),
  FOREIGN KEY (clinica_id, profissional_id)
    REFERENCES profissional (clinica_id, id) ON DELETE RESTRICT,
  FOREIGN KEY (clinica_id, especialidade_id)
    REFERENCES especialidade (clinica_id, id) ON DELETE RESTRICT
);

CREATE TABLE disponibilidade (
  id UUID PRIMARY KEY,
  clinica_id UUID NOT NULL,
  profissional_id UUID NOT NULL,
  inicio_em TIMESTAMPTZ NOT NULL,
  fim_em TIMESTAMPTZ NOT NULL,
  status VARCHAR(15) NOT NULL DEFAULT 'DISPONIVEL'
    CHECK (status IN ('DISPONIVEL', 'BLOQUEADO', 'AGENDADO')),
  motivo_bloqueio VARCHAR(300),
  criado_em TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT ck_disponibilidade_periodo CHECK (fim_em > inicio_em),
  CONSTRAINT uq_disponibilidade_clinica_id UNIQUE (clinica_id, id),
  CONSTRAINT fk_disponibilidade_profissional
    FOREIGN KEY (clinica_id, profissional_id)
    REFERENCES profissional (clinica_id, id) ON DELETE RESTRICT
);

CREATE TABLE agendamento (
  id UUID PRIMARY KEY,
  clinica_id UUID NOT NULL,
  paciente_id UUID NOT NULL,
  profissional_id UUID NOT NULL,
  disponibilidade_id UUID NOT NULL,
  inicio_em TIMESTAMPTZ NOT NULL,
  fim_em TIMESTAMPTZ NOT NULL,
  status VARCHAR(15) NOT NULL DEFAULT 'AGENDADO'
    CHECK (status IN ('AGENDADO', 'CONFIRMADO', 'CANCELADO', 'REALIZADO')),
  observacao VARCHAR(1000),
  criado_por_usuario_id UUID,
  criado_em TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  atualizado_em TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
```

```
  CONSTRAINT ck_agendamento_periodo CHECK (fim_em > inicio_em),
  CONSTRAINT uq_agendamento_clinica_id UNIQUE (clinica_id, id),
  CONSTRAINT fk_agendamento_paciente
    FOREIGN KEY (clinica_id, paciente_id)
    REFERENCES paciente (clinica_id, id) ON DELETE RESTRICT,
  CONSTRAINT fk_agendamento_profissional
    FOREIGN KEY (clinica_id, profissional_id)
    REFERENCES profissional (clinica_id, id) ON DELETE RESTRICT,
  CONSTRAINT fk_agendamento_disponibilidade
    FOREIGN KEY (clinica_id, disponibilidade_id)
    REFERENCES disponibilidade (clinica_id, id) ON DELETE RESTRICT,
  CONSTRAINT fk_agendamento_criador
    FOREIGN KEY (clinica_id, criado_por_usuario_id)
    REFERENCES usuario (clinica_id, id) ON DELETE RESTRICT
);

-- Uma disponibilidade só pode possuir um agendamento ativo.
CREATE UNIQUE INDEX uq_agendamento_disponibilidade_ativa
  ON agendamento (clinica_id, disponibilidade_id)
  WHERE status IN ('AGENDADO', 'CONFIRMADO');

CREATE TABLE atendimento (
  id UUID PRIMARY KEY,
  clinica_id UUID NOT NULL,
  agendamento_id UUID NOT NULL,
  status_comparecimento VARCHAR(12) NOT NULL
    CHECK (status_comparecimento IN ('PRESENTE', 'AUSENTE')),
  registrado_por_usuario_id UUID NOT NULL,
  registrado_em TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  observacao VARCHAR(1000),
  CONSTRAINT uq_atendimento_agendamento UNIQUE (clinica_id, agendamento_id),
  FOREIGN KEY (clinica_id, agendamento_id)
    REFERENCES agendamento (clinica_id, id) ON DELETE RESTRICT,
  FOREIGN KEY (clinica_id, registrado_por_usuario_id)
    REFERENCES usuario (clinica_id, id) ON DELETE RESTRICT
);

CREATE TABLE notificacao (
  id UUID PRIMARY KEY,
  clinica_id UUID NOT NULL,
  agendamento_id UUID NOT NULL,
  tipo VARCHAR(15) NOT NULL
    CHECK (tipo IN ('CONFIRMACAO', 'LEMBRETE')),
  canal VARCHAR(12) NOT NULL
    CHECK (canal IN ('EMAIL', 'SISTEMA')),
  destinatario VARCHAR(254) NOT NULL,
  status VARCHAR(12) NOT NULL DEFAULT 'PENDENTE'
    CHECK (status IN ('PENDENTE', 'ENVIADA', 'FALHA')),
  agendada_para TIMESTAMPTZ,
  enviada_em TIMESTAMPTZ,
  criado_em TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (clinica_id, agendamento_id)
    REFERENCES agendamento (clinica_id, id) ON DELETE RESTRICT
);

CREATE TABLE papel (
  id UUID PRIMARY KEY,
  clinica_id UUID NOT NULL REFERENCES clinica(id) ON DELETE RESTRICT,
  nome VARCHAR(30) NOT NULL
    CHECK (nome IN ('PACIENTE', 'PROFISSIONAL', 'RECEPCIONISTA', 'ADMINISTRADOR')),
  descricao VARCHAR(300),
  CONSTRAINT uq_papel_clinica_id UNIQUE (clinica_id, id),
  CONSTRAINT uq_papel_nome UNIQUE (clinica_id, nome)
);

CREATE TABLE permissao (
  id UUID PRIMARY KEY,
  codigo VARCHAR(80) NOT NULL UNIQUE,
  descricao VARCHAR(300) NOT NULL
);

CREATE TABLE usuario_papel (
  clinica_id UUID NOT NULL,
  usuario_id UUID NOT NULL,
  papel_id UUID NOT NULL,
  PRIMARY KEY (clinica_id, usuario_id, papel_id),
  FOREIGN KEY (clinica_id, usuario_id)
    REFERENCES usuario (clinica_id, id) ON DELETE RESTRICT,
  FOREIGN KEY (clinica_id, papel_id)
    REFERENCES papel (clinica_id, id) ON DELETE RESTRICT
);
```

```
CREATE TABLE papel_permissao (
  papel_id UUID NOT NULL REFERENCES papel(id) ON DELETE RESTRICT,
  permissao_id UUID NOT NULL REFERENCES permissao(id) ON DELETE RESTRICT,
  PRIMARY KEY (papel_id, permissao_id)
);

CREATE TABLE registro_auditoria (
  id UUID PRIMARY KEY,
  clinica_id UUID NOT NULL REFERENCES clinica(id) ON DELETE RESTRICT,
  usuario_id UUID,
  acao VARCHAR(80) NOT NULL,
  entidade VARCHAR(80) NOT NULL,
  entidade_id UUID,
  detalhes JSONB,
  criado_em TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (clinica_id, usuario_id)
    REFERENCES usuario (clinica_id, id) ON DELETE RESTRICT
);

-- Índices para as consultas mais frequentes.
CREATE INDEX ix_disponibilidade_profissional_inicio
  ON disponibilidade (clinica_id, profissional_id, inicio_em);

CREATE INDEX ix_agendamento_paciente_inicio
  ON agendamento (clinica_id, paciente_id, inicio_em DESC);

CREATE INDEX ix_agendamento_profissional_inicio
  ON agendamento (clinica_id, profissional_id, inicio_em);

CREATE INDEX ix_notificacao_pendente
  ON notificacao (clinica_id, status, agendada_para);

CREATE INDEX ix_auditoria_clinica_data
  ON registro_auditoria (clinica_id, criado_em DESC);
```

## 5.1 Regras de execução e concorrência

• A criação de agendamento deve ocorrer em transação. O serviço valida o contexto da clínica, consulta a disponibilidade e tenta inserir o registro.

• Ciclo de vida do agendamento: AGENDADO (reserva criada, disponibilidade ocupada) → CONFIRMADO (presença confirmada pelo paciente ou pela recepção, por exemplo após o lembrete) → REALIZADO (comparecimento ou ausência registrados na tabela atendimento). CANCELADO pode ocorrer a partir de AGENDADO ou CONFIRMADO. Os status AGENDADO e CONFIRMADO são considerados ativos: ocupam a disponibilidade (índice único parcial) e impedem seu bloqueio (RF026).

• O índice único parcial uq_agendamento_disponibilidade_ativa é a última barreira contra duas reservas ativas para a mesma disponibilidade. Se duas requisições concorrentes tentarem reservar o mesmo horário, uma delas deverá falhar por violação de unicidade.

• A alteração de status da disponibilidade e a criação ou cancelamento do agendamento devem ser realizadas de forma coordenada na mesma transação.

• Antes de bloquear um horário, o serviço deve verificar agendamentos ativos associados, na mesma transação do bloqueio. O banco impede duplicidade de reserva, mas a regra RF026 também precisa ser validada na camada de serviço.

• As FKs compostas garantem que paciente, profissional, disponibilidade, usuário e agendamento referenciados pertençam à mesma clínica.

Nota de implementação: o script é uma base executável para PostgreSQL. A equipe deve validar o mapeamento JPA, os nomes físicos das colunas e a estratégia de geração de UUIDs antes de integrar as migrations à aplicação.

# 6. Dicionário de dados

O dicionário descreve os campos relevantes, seus tipos, restrições e significado de negócio. Para manter a leitura objetiva, os campos de auditoria padrão (criado_em e atualizado_em) são descritos quando aparecem nas tabelas.

## clinica

| Campo | Tipo | Restrições | Descrição de negócio |
|---|---|---|---|
| id | UUID | PK, NOT NULL | Identificador da clínica. |
| nome | VARCHAR(160) | NOT NULL | Nome institucional. |
| documento | VARCHAR(20) | Opcional | Identificador documental da instituição, se coletado. |
| email_contato | VARCHAR(254) | Opcional | E-mail institucional para contato. |
| telefone | VARCHAR(25) | Opcional | Telefone institucional. |
| status | VARCHAR(15) | CHECK, NOT NULL | Situação da clínica: ATIVA ou INATIVA. |
| criado_em / atualizado_em | TIMESTAMPTZ | NOT NULL | Datas de criação e última atualização. |

## usuario

| Campo | Tipo | Restrições | Descrição de negócio |
|---|---|---|---|
| id | UUID | PK | Identificador da conta. |

| | | | |
|---|---|---|---|
| clinica_id | UUID | FK, NOT NULL | Clínica à qual a conta está vinculada. |
| nome | VARCHAR(160) | NOT NULL | Nome de exibição do usuário. |
| email | VARCHAR(254) | UNIQUE por clínica | Identificador de login. |
| senha_hash | VARCHAR(255) | NOT NULL | Hash da senha; nunca armazenar senha em texto puro. |
| telefone | VARCHAR(25) | Opcional | Contato do usuário. |
| status | VARCHAR(15) | CHECK | ATIVO, INATIVO ou BLOQUEADO. |
| criado_em / atualizado_em | TIMESTAMPTZ | NOT NULL | Controle temporal da conta. |

## paciente

| Campo | Tipo | Restrições | Descrição de negócio |
|---|---|---|---|
| id | UUID | PK | Identificador do perfil de paciente. |
| clinica_id | UUID | FK composta | Clínica à qual o paciente pertence. |
| usuario_id | UUID | FK composta, UNIQUE | Conta de acesso associada ao paciente. |
| cpf | VARCHAR(14) | Opcional, UNIQUE por clínica | CPF do paciente; dado pessoal usado na identificação cadastral. |
| data_nascimento | DATE | Opcional | Data de nascimento, dado pessoal. |
| observacoes_cadastrais | VARCHAR(1000) | Opcional | Observações cadastrais limitadas; não |

| | | | |
|---|---|---|---|
| | | | usar como prontuário. |

## profissional

| Campo | Tipo | Restrições | Descrição de negócio |
|---|---|---|---|
| id | UUID | PK | Identificador do perfil profissional. |
| clinica_id | UUID | FK composta | Clínica de vínculo. |
| usuario_id | UUID | FK composta, UNIQUE | Conta associada ao profissional. |
| registro_profissional | VARCHAR(40) | Opcional | Número de registro no conselho profissional. |
| conselho | VARCHAR(40) | Opcional | Conselho profissional correspondente. |
| ativo | BOOLEAN | NOT NULL | Indica se o profissional está ativo. |

## especialidade

| Campo | Tipo | Restrições | Descrição de negócio |
|---|---|---|---|
| id | UUID | PK | Identificador da especialidade. |
| clinica_id | UUID | FK, NOT NULL | Clínica responsável pelo cadastro. |
| nome | VARCHAR(120) | UNIQUE por clínica | Nome da especialidade. |
| descricao | VARCHAR(500) | Opcional | Descrição da especialidade. |
| ativa | BOOLEAN | NOT NULL | Indica se a especialidade está disponível. |

## profissional_especialidade

| Campo | Tipo | Restrições | Descrição de negócio |
|---|---|---|---|

| | | | |
|---|---|---|---|
| clinica_id | UUID | PK composta, FK | Clínica do relacionamento. |
| profissional_id | UUID | PK composta, FK | Profissional associado. |
| especialidade_id | UUID | PK composta, FK | Especialidade associada. |

## disponibilidade

| Campo | Tipo | Restrições | Descrição de negócio |
|---|---|---|---|
| id | UUID | PK | Identificador do intervalo. |
| clinica_id | UUID | FK composta | Clínica do profissional. |
| profissional_id | UUID | FK composta | Profissional dono do intervalo. |
| inicio_em / fim_em | TIMESTAMPTZ | CHECK fim > início | Limites do intervalo de disponibilidade. |
| status | VARCHAR(15) | CHECK | DISPONIVEL, BLOQUEADO ou AGENDADO. |
| motivo_bloqueio | VARCHAR(300) | Opcional | Justificativa do bloqueio, quando aplicável. |
| criado_em | TIMESTAMPTZ | NOT NULL | Data de criação. |

## agendamento

| Campo | Tipo | Restrições | Descrição de negócio |
|---|---|---|---|
| id | UUID | PK | Identificador do agendamento. |
| clinica_id | UUID | FK composta | Clínica à qual o agendamento pertence. |
| paciente_id | UUID | FK composta | Paciente atendido. |
| profissional_id | UUID | FK composta | Profissional responsável. |
| disponibilidade_id | UUID | FK composta | Intervalo reservado. |

| | | | |
|---|---|---|---|
| inicio_em / fim_em | TIMESTAMPTZ | CHECK fim > início | Período da consulta. |
| status | VARCHAR(15) | CHECK | AGENDADO, CONFIRMADO, CANCELADO ou REALIZADO. |
| observacao | VARCHAR(1000) | Opcional | Observação administrativa do agendamento. |
| criado_por_usuario_id | UUID | FK composta, opcional | Usuário que registrou a marcação, incluindo recepcionista. |
| criado_em / atualizado_em | TIMESTAMPTZ | NOT NULL | Datas de criação e atualização. |

## atendimento

| Campo | Tipo | Restrições | Descrição de negócio |
|---|---|---|---|
| id | UUID | PK | Identificador do registro de atendimento. |
| clinica_id | UUID | FK composta | Clínica do atendimento. |
| agendamento_id | UUID | FK composta, UNIQUE | Agendamento correspondente; no máximo um registro. |
| status_comparecimento | VARCHAR(12) | CHECK | PRESENTE ou AUSENTE. |
| registrado_por_usuario_id | UUID | FK composta | Usuário autorizado que registrou o comparecimento. |
| registrado_em | TIMESTAMPTZ | NOT NULL | Data e hora do registro. |
| observacao | VARCHAR(1000) | Opcional | Observação administrativa, sem substituir prontuário. |

## notificacao

| Campo | Tipo | Restrições | Descrição de negócio |
|---|---|---|---|
| id | UUID | PK | Identificador da notificação. |
| clinica_id | UUID | FK | Clínica responsável. |
| agendamento_id | UUID | FK composta | Agendamento que originou a notificação. |
| tipo | VARCHAR(15) | CHECK | CONFIRMACAO ou LEMBRETE. |
| canal | VARCHAR(12) | CHECK | EMAIL ou SISTEMA. |
| destinatario | VARCHAR(254) | NOT NULL | Destino da notificação. |
| status | VARCHAR(12) | CHECK | PENDENTE, ENVIADA ou FALHA. |
| agendada_para / enviada_em | TIMESTAMPTZ | Opcional | Datas de programação e envio. |
| criado_em | TIMESTAMPTZ | NOT NULL | Data de criação. |

## papel / permissao / usuario_papel / papel_permissao

| Campo | Tipo | Restrições | Descrição de negócio |
|---|---|---|---|
| papel.id / permissao.id | UUID | PK | Identificadores dos papéis e permissões. |
| papel.nome | VARCHAR(30) | CHECK, UNIQUE por clínica | PACIENTE, PROFISSIONAL, RECEPCIONISTA ou ADMINISTRADOR. |
| permissao.codigo | VARCHAR(80) | UNIQUE | Código estável da permissão, por exemplo AGENDAMENTO_CANCELAR. |
| usuario_papel | UUIDs | PK/FKs compostas | Associa contas aos papéis de uma clínica. |
| papel_permissao | UUIDs | PK/FKs | Associa permissões aos papéis. |

## registro_auditoria

| Campo | Tipo | Restrições | Descrição de negócio |
|---|---|---|---|

| | | | |
|---|---|---|---|
| id | UUID | PK | Identificador do evento de auditoria. |
| clinica_id | UUID | FK | Clínica em cujo contexto ocorreu a operação. |
| usuario_id | UUID | FK composta, opcional | Usuário responsável pela ação, quando identificado. |
| acao | VARCHAR(80) | NOT NULL | Operação realizada. |
| entidade | VARCHAR(80) | NOT NULL | Tipo de registro afetado. |
| entidade_id | UUID | Opcional | Identificador do registro afetado. |
| detalhes | JSONB | Opcional | Metadados mínimos da operação; não registrar senhas ou tokens. |
| criado_em | TIMESTAMPTZ | NOT NULL | Data e hora do evento. |

# 7. Estratégia de dados: backup, LGPD e migrations

## 7.1 Backup e recuperação

O banco será hospedado no Amazon RDS PostgreSQL em configuração Multi-AZ, conforme a arquitetura. A redundância Multi-AZ apoia a disponibilidade do serviço, mas não substitui backups: exclusões acidentais, corrupção lógica e alterações indevidas exigem mecanismos próprios de recuperação.

| Item | Política proposta | Objetivo |
|---|---|---|
| Backups automáticos | Diariamente, por meio dos backups automáticos do Amazon RDS. | Permitir recuperação do banco após falha ou erro operacional. |
| Retenção | Mínimo de 7 dias para backups diários e de 4 semanas para cópias semanais (AWS Backup), | Manter janela suficiente para identificar problemas não |

| | | |
|---|---|---|
| | conforme o RNF007; a janela do point-in-time recovery do RDS inicia em 7 dias e pode ser ampliada após validação de custo | percebidos imediatamente. |
| Point-in-Time Recovery | Habilitar recuperação para um instante dentro da janela de retenção oferecida pelo RDS. | Reduzir perda de dados em incidentes. |
| Snapshots | Snapshot manual antes de mudanças estruturais relevantes e releases de maior risco. | Criar ponto de retorno controlado. |
| Testes de restauração | Executar teste trimestral em ambiente isolado e registrar o resultado. | Verificar se o backup é realmente restaurável. |
| Acesso aos backups | Restringir a administradores autorizados e registrar operações. | Reduzir risco de acesso indevido. |

Metas de recuperação: a arquitetura registra RPO de até 15 minutos e RTO de até 4 horas como objetivos a validar. A política final deverá ser testada na configuração real do RDS e da aplicação; a simples ativação de backup ou Multi-AZ não comprova, isoladamente, o atendimento dessas metas.

## 7.2 LGPD e proteção de dados

O VittaCare tratará dados pessoais e poderá tratar dados pessoais sensíveis relacionados à saúde. A aplicação dos princípios da LGPD deverá ser incorporada desde o desenho do banco, com coleta mínima, finalidade definida, acesso restrito, segurança e retenção controlada.

| Campo / grupo | Classificação | Cuidados previstos |
|---|---|---|
| usuario.nome, usuario.email, usuario.telefone | Dado pessoal | Acesso conforme perfil; exibição somente quando necessária; proteção em trânsito. |
| paciente.data_nascimento | Dado pessoal | Coleta somente quando necessária ao processo da |

| | | |
|---|---|---|
| | | clínica; restringir visualização. |
| paciente.cpf | Dado pessoal | Coleta somente quando necessária ao cadastro; acesso restrito por perfil e por clínica; mascarar na exibição quando possível; nunca registrar em logs. |
| profissional.registro_profissional e conselho | Dado pessoal profissional | Acesso administrativo e profissional conforme necessidade. |
| agendamento.inicio_em, fim_em e status | Dado pessoal quando associado ao paciente | Restringir por clínica e por perfil; evitar exposição pública. |
| atendimento.status_comparecimento | Dado pessoal associado a serviço de saúde | Acesso restrito; não ampliar para conteúdo clínico fora do escopo. |
| usuario.senha_hash | Credencial protegida | Armazenar somente hash seguro, preferencialmente Argon2id ou BCrypt; nunca registrar em logs. |
| registro_auditoria.detalhes | Metadados potencialmente pessoais | Registrar somente o necessário; excluir senhas, tokens e conteúdo sensível desnecessário. |

- Aplicar autorização no back-end em 100% dos endpoints que exponham informações restritas.
- Aplicar filtro por clinica_id em consultas, inserções, atualizações e exclusões.
- Obter o contexto da clínica a partir da identidade autenticada e das associações autorizadas.
- Não utilizar o campo de observação como substituto de prontuário eletrônico.

- Definir política de retenção e descarte seguro de acordo com a finalidade e as obrigações legais aplicáveis.
- Manter trilha de auditoria para operações administrativas críticas, sem armazenar credenciais ou tokens.
- Restringir acesso ao banco a partir das instâncias autorizadas, em sub-redes privadas, conforme a arquitetura.

A definição da base legal, dos prazos legais de retenção e dos procedimentos para atendimento aos direitos dos titulares deverá ser validada pela instituição responsável e por orientação jurídica/encarregado de dados. Este documento estabelece controles técnicos propostos, não substitui uma avaliação jurídica.

## 7.3 Migrations e evolução do esquema

As alterações do esquema serão versionadas por migrations, preferencialmente com Flyway integrado ao Spring Boot. Cada migration deverá ser imutável após aplicação em ambiente compartilhado, possuir identificador sequencial e ser revisada junto ao código que depende dela.

| Etapa | Procedimento |
|---|---|
| 1. Desenvolvimento | Criar migration versionada e testar em banco local com a mesma versão principal do PostgreSQL. |
| 2. Validação | Executar testes de integração, constraints, índices e consultas críticas. |
| 3. Staging | Aplicar migration em ambiente de homologação com dados sintéticos ou anonimizados. |
| 4. Produção | Aplicar mudanças compatíveis antes de publicar código que dependa delas. |
| 5. Monitoramento | Verificar duração, locks, erros, integridade e métricas após a aplicação. |

## 7.4 Estratégia para migrations sem downtime

1. Expandir: adicionar coluna ou tabela nova como opcional, sem remover ou renomear imediatamente estruturas utilizadas pela versão atual.
2. Compatibilizar: publicar código que consiga ler o formato antigo e o novo, quando necessário.
3. Migrar: preencher dados em lotes pequenos, monitorando carga e bloqueios.
4. Trocar: direcionar a aplicação para a nova estrutura após validar consistência.

5. Contrair: remover coluna ou estrutura antiga somente em uma migration posterior, quando nenhuma versão ativa depender dela.

Índices grandes deverão ser planejados para criação concorrente quando aplicável no PostgreSQL. Operações que exigem locks longos devem ser agendadas e testadas previamente. Uma migration não deve ser considerada sem downtime apenas por estar versionada: o impacto real depende do comando, do volume de dados e da estratégia de implantação.

# 8. Regras de integridade e rastreabilidade

| Regra | Implementação no banco / aplicação | Requisitos relacionados |
|---|---|---|
| Autenticação | E-mail único por clínica, hash de senha e status de conta; validação de credenciais no Spring Security. | RF001, RF002, RF003 |
| Cadastro de paciente | Tabela paciente vinculada a usuario e clínica por FK composta. | RF005, RF006 |
| Profissionais e especialidades | Tabelas profissional, especialidade e associação N:N. | RF007, RF008, RF009 |
| Pesquisa e disponibilidade | Índices por clínica, profissional e início do intervalo. | RF010, RF011, RF012, RF013 |
| Agendamento | FKs para paciente, profissional e disponibilidade; transação no serviço. | RF014, RF015, RF018 |
| Cancelamento e remarcação | Atualização controlada de status e criação de nova reserva na mesma unidade transacional quando aplicável. | RF016, RF017 |
| Prevenção de duplicidade | Índice único parcial para uma reserva ativa por disponibilidade. | RF018 |
| Histórico | Índices para consulta por paciente e profissional. | RF019 |
| Comparecimento | Tabela atendimento com UNIQUE por agendamento. | RF020 |

| | | |
|---|---|---|
| Notificações | Tabela notificacao vinculada ao agendamento. | RF021, RF022 |
| Relatórios | Consultas agregadas sobre agendamentos, cancelamentos e ausências. | RF023 |
| Permissões | Papéis, permissões e associações; autorização efetiva no back-end. | RF024 |
| Isolamento multi-tenant | clinica_id nas tabelas e FKs compostas para impedir referências entre clínicas. | RF025 |
| Bloqueio de horário | Validação transacional no serviço antes de alterar disponibilidade; não bloquear horário com agendamento ativo. | RF026 |

## 8.1 Pontos que exigem validação na implementação

- Definir como o paciente seleciona a clínica no autorregistro, por exemplo por endereço específico ou convite, sem confiar em clinica_id arbitrário enviado pelo navegador.
- Definir política de normalização do e-mail (por exemplo, armazenar em minúsculas) antes de aplicar a restrição de unicidade.
- Definir a granularidade das disponibilidades: intervalos cadastrados pelo profissional ou slots derivados de uma agenda. O script representa intervalos reserváveis individualmente.
- Garantir que o horário do agendamento corresponda ao intervalo da disponibilidade e que o profissional do agendamento seja o mesmo profissional dono da disponibilidade. Essa consistência deverá ser garantida por validação no serviço e pode ser reforçada com constraints adicionais após a decisão final sobre a granularidade.
- Definir catálogo inicial de permissões e papéis, com carga de dados de referência por migration.
- Definir processo de anonimização ou descarte de dados quando houver solicitação válida e não existir obrigação de retenção.

# 9. Considerações finais

A modelagem proposta mantém a escolha tecnológica do Bloco 3 e traduz as entidades e regras dos Blocos 1, 2 e 4 para um banco relacional PostgreSQL. O uso de chaves estrangeiras, constraints, índices e transações oferece uma base consistente para os fluxos centrais do VittaCare, especialmente o agendamento e o isolamento lógico entre clínicas.

O modelo físico apresentado é executável em PostgreSQL e fornece uma estrutura inicial para implementação. Antes da entrada em produção, deverão ser concluídos os testes de concorrência, validação do mapeamento JPA, revisão das permissões, definição das políticas de retenção e testes reais de backup e recuperação.

Dessa forma, o Bloco 5 entrega a passagem do modelo conceitual para uma estrutura lógica e física que pode orientar a implementação do banco de dados do VittaCare.

# VittaCare

Planejamento e estimativas – Bloco 7

## Sumário


1. Introdução ..... 3
2. Premissas de planejamento ..... 3
3. WBS / EAP — Estrutura Analítica do Projeto ..... 4
4. Estimativa de esforço — Story Points ..... 6
5. Velocidade da equipe e cronograma ..... 7
6. Composição da equipe e referências salariais ..... 8
7. Matriz de riscos ..... 9
8. Acompanhamento e controle ..... 11
9. Considerações finais ..... 11
10. Referências de salários ..... 12

## 1. Introdução

Este documento apresenta o planejamento de execução do VittaCare, plataforma web SaaS destinada à gestão de clínicas e ambulatórios. O objetivo é decompor o trabalho necessário, estimar o esforço de desenvolvimento, propor uma composição de equipe com referências salariais de mercado e identificar riscos que possam afetar prazo, custo, segurança e qualidade.

O planejamento mantém as decisões dos blocos anteriores: aplicação web em React, back-end Java com Spring Boot organizado como monólito modular, API REST sobre HTTPS, PostgreSQL no Amazon RDS Multi-AZ, persistência com Spring Data JPA/Hibernate e isolamento lógico dos dados por clínica por meio de clinica_id.

O escopo considerado é o da versão atual descrita nos documentos de concepção e requisitos. Não estão incluídos prontuário eletrônico completo, prescrição eletrônica, diagnóstico automatizado, teleconsulta, processamento de pagamentos ou gestão financeira completa.

## 2. Premissas de planejamento

| Item | Premissa adotada |
|---|---|
| Modelo de desenvolvimento | Scrum, com sprints de duas semanas. |
| Duração estimada | 16 semanas, equivalentes a 8 sprints. |
| Equipe principal | 2 desenvolvedores back-end, 1 desenvolvedor front-end e 1 QA, com apoio parcial de PO, UX/UI, DevOps e segurança. |
| Velocidade de referência | 20 Story Points por sprint para a equipe de desenvolvimento, a validar após as duas primeiras sprints. |
| Esforço total planejado | 160 Story Points para o MVP descrito neste documento. |
| Regime de contratação | Referência de salário-base mensal bruto CLT. Encargos, benefícios, equipamentos e tributos não estão somados ao custo salarial-base. |
| Infraestrutura | Os custos de AWS não são recalculados neste bloco; devem ser consolidados com o Bloco 6, caso ele seja desenvolvido. |
| Natureza dos números | Estimativas para planejamento acadêmico, não propostas comerciais nem orçamento fechado. |

## 3. WBS / EAP — Estrutura Analítica do Projeto

A WBS organiza o escopo em entregas hierárquicas. A decomposição abaixo possui três níveis: projeto, entregas principais e pacotes de trabalho. Os pacotes representam resultados verificáveis.

| Código | Entrega / pacote de trabalho | Resultado esperado |
|---|---|---|
| 1.0 | VittaCare — MVP | Entrega do produto web funcional e validado |
| 1.1 | Gestão e descoberta | Planejamento e alinhamento |
| 1.1.1 | Consolidar escopo e prioridades | Backlog priorizado por MoSCoW |
| 1.1.2 | Refinar histórias e critérios de aceite | Histórias com Given/When/Then |
| 1.1.3 | Planejar sprints e acompanhar indicadores | Backlog, burndown e revisões |
| 1.2 | Experiência e interface | Experiência dos quatro perfis |
| 1.2.1 | Mapear jornadas de paciente, profissional, recepção e administrador | Fluxos principais documentados |
| 1.2.2 | Criar wireframes e protótipo navegável | Protótipo validado |
| 1.2.3 | Implementar componentes React responsivos | Interface integrada à API |
| 1.3 | Autenticação e usuários | Acesso e gerenciamento de contas |
| 1.3.1 | Implementar login, JWT e Spring Security | Autenticação funcional |
| 1.3.2 | Implementar recuperação de senha e status da conta | Fluxos de recuperação e ativação |
| 1.3.3 | Implementar papéis e permissões (RBAC) | Acesso por perfil |
| 1.4 | Cadastros e organização da clínica | Dados-base do sistema |
| 1.4.1 | Implementar cadastro e atualização de pacientes | RF005 e RF006 |
| 1.4.2 | Implementar cadastro de profissionais | RF007 |
| 1.4.3 | Implementar especialidades e associações | RF008 e RF009 |
| 1.4.4 | Implementar isolamento por clinica_id | RF025 |

| | | |
|---|---|---|
| 1.5 | Agenda e disponibilidade | Gestão dos horários profissionais |
| 1.5.1 | Implementar cadastro, edição e bloqueio de horários | RF013 |
| 1.5.2 | Consultar especialidades, profissionais e horários | RF010, RF011 e RF012 |
| 1.5.3 | Validar conflitos e concorrência de horários | RF018 e RF026 |
| 1.6 | Agendamentos e atendimentos | Fluxo central de consultas |
| 1.6.1 | Implementar agendamento pelo paciente | RF014 |
| 1.6.2 | Implementar agendamento pela recepção | RF015 |
| 1.6.3 | Implementar cancelamento e remarcação | RF016 e RF017 |
| 1.6.4 | Implementar histórico e registro de presença/ausência | RF019 e RF020 |
| 1.7 | Notificações e relatórios | Comunicação e visão administrativa |
| 1.7.1 | Enviar confirmação de agendamento | RF021 |
| 1.7.2 | Enviar lembretes de consultas próximas | RF022 |
| 1.7.3 | Disponibilizar relatório administrativo básico | RF023 — prioridade Could |
| 1.8 | Qualidade, segurança e entrega | Produto testado e preparado para implantação |
| 1.8.1 | Criar testes unitários e de integração | Cobertura dos fluxos críticos |
| 1.8.2 | Executar testes E2E e de concorrência | Validação dos fluxos completos |
| 1.8.3 | Revisar LGPD, autorização e auditoria | Controles de proteção revisados |
| 1.8.4 | Preparar deploy, monitoramento e documentação | Versão candidata à produção |

## 4. Estimativa de esforço — Story Points

Os Story Points representam esforço relativo, considerando complexidade, incerteza e volume de trabalho. Não correspondem diretamente a horas. A estimativa será revista durante o refinamento do backlog.

| Funcionalidade / grupo | Rastreabilidade | Story Points |
|---|---|---|
| Autenticação, recuperação de senha e sessão | RF001–RF003 | 13 |
| Cadastro e atualização de usuários e pacientes | RF004–RF006 | 13 |
| Cadastro de profissionais, especialidades e associações | RF007–RF009 | 13 |
| Pesquisa de profissionais e consulta de disponibilidade | RF010–RF012 | 13 |
| Gestão de disponibilidade e | RF013 | 13 |
| bloqueios | | |
| Agendamento pelo paciente e pela recepção | RF014–RF015 | 21 |
| Cancelamento, remarcação e histórico | RF016–RF019 | 13 |
| Registro de comparecimento e ausência | RF020 | 5 |
| Confirmações e lembretes | RF021–RF022 | 8 |
| Relatórios administrativos básicos | RF023 | 8 |
| Papéis, permissões e isolamento entre clínicas | RF024–RF025 | 21 |
| Prevenção de conflito e bloqueio com consulta confirmada | RF018 e RF026 | 13 |
| Testes, correções, segurança e estabilização transversal | RNFs e qualidade | 6 |

**Total estimado: 160 Story Points.**

A soma inclui funcionalidades de prioridade Must e Should, além do relatório RF023, classificado como Could. Caso o prazo fique comprometido, RF023 poderá ser deslocado para uma versão posterior sem comprometer os fluxos essenciais de agendamento.

As histórias de usuário do Bloco 2 somam 94 Story Points e cobrem as funcionalidades visíveis a cada perfil. Os 160 pontos deste bloco incluem, além delas, trabalho transversal: modelagem e migrations, integração com o provedor de notificações, regras de concorrência, isolamento por clínica, auditoria, testes e estabilização.

## 5. Velocidade da equipe e cronograma

Considerando uma equipe de três pessoas atuando diretamente no desenvolvimento (dois backend e um front-end), com apoio contínuo de QA, adota-se uma velocidade inicial de 20 Story Points por sprint. Para 160 pontos, são necessárias oito sprints de duas semanas, totalizando aproximadamente 16 semanas.

| Sprint | Capacidade planejada (SP) | Objetivo principal |
|---|---|---|
| Sprint 1 | 20 | Fundação do projeto, estrutura React/Spring Boot, banco, pipeline inicial e autenticação básica. |
| Sprint 2 | 20 | Recuperação de senha, usuários, papéis, permissões e estrutura multi-clínica. |
| Sprint 3 | 20 | Cadastro de pacientes, profissionais, especialidades e associações. |
| Sprint 4 | 20 | Pesquisa de profissionais, consulta de especialidades e visualização de |
| | | disponibilidade. |
| Sprint 5 | 20 | Cadastro/bloqueio de horários e primeira versão do agendamento. |
| Sprint 6 | 20 | Agendamento pela recepção, cancelamento, remarcação e prevenção de conflitos. |
| Sprint 7 | 20 | Histórico, registro de presença/ausência, notificações e relatórios básicos. |
| Sprint 8 | 20 | Testes integrados e E2E, segurança, correções, documentação e preparação da implantação. |

A velocidade de 20 SP é uma hipótese de planejamento, não um dado histórico da equipe. Corresponde a cerca de 6 a 7 pontos por desenvolvedor por sprint para os três desenvolvedores, com QA em apoio contínuo. Após as sprints 1 e 2, a velocidade real deverá ser calculada pela

média dos pontos concluídos e aceitos. O cronograma deverá ser reestimado se a velocidade observada ficar abaixo da planejada.

O número de oito sprints resulta da divisão do esforço total (160 Story Points) pela velocidade de referência (20 Story Points por sprint). Os perfis da equipe foram escolhidos para cobrir cada frente do WBS: back-end (autenticação, agenda, agendamento e concorrência), front-end (interfaces dos quatro perfis), QA (fluxos críticos e concorrência), produto (priorização e aceite), UX/UI (jornadas e protótipo), DevOps (implantação e monitoramento) e segurança (revisão de LGPD).

## 6. Composição da equipe e referências salariais

A equipe foi dimensionada para o escopo e para o prazo proposto. Os valores abaixo são referências de salário-base mensal bruto no Brasil consultadas no Glassdoor em novembro de 2026. Para o orçamento, foi adotado um valor de referência dentro da faixa publicada para cada função. A remuneração efetiva varia conforme senioridade, região, empresa e regime de contratação.

| Perfil | Quantidade / dedicação | Referência mensal | Faixa Glassdoor | Período considerado | Custo estimado |
|---|---|---|---|---|---|
| Product Owner / responsável pelo produto | 1 (50% de dedicação) | R$ 9.000 | R$ 7.000–11.000 | 4 meses | R$ 18.000 |
| Desenvolvedor back-end Java | 2 (100% cada) | R$ 8.000 | R$ 4.000–11.000 | 4 meses | R$ 64.000 |
| Desenvolvedor front-end React | 1 (100%) | R$ 5.000 | R$ 4.000–8.000 | 4 meses | R$ 20.000 |
| Analista de QA / testes | 1 (100%) | R$ 5.000 | R$ 4.000–7.000 | 3 meses | R$ 15.000 |
| Engenheiro DevOps | 1 (25% de dedicação) | R$ 9.000 | R$ 7.000–13.000 | 4 meses | R$ 9.000 |
| UX/UI | 1 (50% de | R$ 5.000 | R$ 3.000– | 2 meses | R$ 5.000 |
| Designer | dedicação) | | 7.000 | | |
| Analista de segurança da informação | 1 (10% de dedicação) | R$ 5.000 | R$ 4.000–8.000 | 1 mês equivalente | R$ 500 |

Arquiteto de software e DBA/engenheiro de dados não compõem o custo de construção do MVP neste bloco, pois suas atividades concentram-se nas fases de concepção, arquitetura e modelagem (Blocos 1 a 5) e na operação posterior, cobertas pela equipe permanente descrita no Bloco 1. As referências salariais deste bloco são valores de planejamento dentro da faixa do Glassdoor e podem diferir dos valores unitários do Bloco 1, que consideram a equipe permanente e uma contratação mista CLT/PJ.

Custo salarial direto estimado: R$ 131.500,00. O cálculo considera salário mensal × dedicação × período de alocação. Exemplo: dois desenvolvedores back-end × R$ 8.000 × quatro meses = R$ 64.000.

Reserva de contingência de planejamento (20%): R$ 26.300,00. Orçamento de mão de obra com contingência: R$ 157.800,00.

Este total não inclui encargos trabalhistas, benefícios, equipamentos, licenças, impostos, custos de infraestrutura AWS nem suporte pós-implantação. Como o regime de contratação e a estrutura tributária não foram definidos nos documentos anteriores, esses itens não foram convertidos em valores fictícios. O custo completo deverá ser atualizado quando essas premissas forem estabelecidas.

Arquiteto de software e DBA/engenheiro de dados não compõem o custo de construção do MVP neste bloco, pois suas atividades se concentram nas fases de concepção, arquitetura e modelagem (Blocos 1 a 5) e na operação posterior, cobertas pela equipe permanente descrita no Bloco 1. As referências salariais deste bloco são valores de planejamento dentro da faixa do Glassdoor e podem diferir dos valores unitários do Bloco 1, que consideram a equipe permanente e uma contratação mista CLT/PJ.

## 7. Matriz de riscos

A probabilidade e o impacto são classificados como Baixo, Médio ou Alto. A prioridade resulta da combinação qualitativa entre os dois fatores. Os riscos abaixo cobrem tecnologia, negócio, equipe, segurança e conformidade.

| ID | Categoria | Descrição do risco | Prob. | Impacto | Prioridade | Mitigação |
|---|---|---|---|---|---|---|
| R01 | Técnico | Conflito ou duplicidade de agendamentos em acessos simultâneos. | Média | Alta | Alta | Transações no PostgreSQL, constraints/índices únicos, validação no serviço e testes de concorrência. |
| R02 | Segurança | Vazamento de dados pessoais ou acesso indevido entre clínicas. | Média | Alta | Alta | Validar clinica_id no back-end a partir do usuário autenticado; RBAC, menor privilégio, logs e testes de isolamento. |

| | | | | | | |
|---|---|---|---|---|---|---|
| R03 | Regulatório | Tratamento inadequado de dados pessoais e dados sensíveis de | Média | Alta | Alta | Mapear dados, finalidade e base legal; limitar coleta; definir retenção, controle |
| | | saúde, em desacordo com a LGPD. | | | | de acesso, auditoria e processo de incidentes. |
| R04 | Técnico | Indisponibilidade do banco ou da aplicação em produção. | Média | Alta | Alta | Manter RDS MultiAZ, balanceamento, monitoramento, backups testados e procedimento de recuperação. |
| R05 | Equipe | Atraso por indisponibilidade ou sobrecarga de profissionaischave. | Média | Média | Média | Documentar decisões, realizar revisão de código, compartilhar conhecimento e manter capacidade de contingência. |
| R06 | Prazo / estimativa | Velocidade real inferior a 20 SP por sprint ou aumento de escopo. | Alta | Média | Alta | Medir velocidade após duas sprints, priorizar Must, controlar mudanças e replanejar com o Product Owner. |
| R07 | Negócio | Baixa adesão de pacientes, profissionais ou recepcionistas à plataforma. | Média | Média | Média | Validar protótipos com representantes dos perfis, oferecer interface simples e preparar orientação de uso. |

| | | | | | | |
|---|---|---|---|---|---|---|
| R08 | Integração | Falhas no envio de confirmações e lembretes por dependência de provedor externo. | Média | Média | Média | Implementar retentativas, registro de falhas, monitoramento e mecanismo alternativo de consulta do agendamento. |
| R09 | Dados | Perda ou corrupção de dados durante migração ou atualização do | Baixa | Alta | Média | Versionar migrations, testar em staging, manter backup anterior e executar plano de |
| | | esquema. | | | | rollback. |
| R10 | Qualidade | Defeitos em fluxos críticos não identificados antes da implantação. | Média | Alta | Alta | Automatizar testes unitários, integração e E2E; validar fluxos de agendamento, cancelamento, permissões e concorrência. |

## 8. Acompanhamento e controle

O acompanhamento do planejamento será realizado por meio dos seguintes indicadores:

- Velocidade por sprint: Story Points concluídos e aceitos.
- Variação de prazo: diferença entre a data planejada e a data real das entregas.
- Variação de esforço: Story Points adicionados ou removidos após o planejamento.
- Defeitos por severidade: quantidade de falhas abertas e resolvidas, especialmente nos fluxos críticos.
- Riscos prioritários: quantidade de riscos altos sem ação de mitigação concluída.
- Prontidão para entrega: critérios de aceite atendidos, testes executados e documentação atualizada.

Ao final de cada sprint, a equipe realizará revisão das entregas, retrospectiva e atualização do backlog. Mudanças de escopo deverão ser registradas e refletidas nas estimativas de esforço, prazo e custo.

## 9. Considerações finais

O planejamento propõe a execução do MVP do VittaCare em 16 semanas, distribuídas em oito sprints de duas semanas, com 160 Story Points estimados e velocidade inicial de 20 pontos por

sprint. A equipe combina desenvolvimento back-end e front-end, qualidade, produto, experiência do usuário, DevOps e apoio de segurança.

Com base nas referências salariais consultadas, o custo salarial direto estimado é de R$ 131.500,00. Com reserva de contingência de 20%, o orçamento de mão de obra passa a R$ 157.800,00. Esse valor não representa o custo total de propriedade do sistema, pois não inclui encargos, benefícios, infraestrutura, licenças ou operação.

As estimativas deverão ser refinadas com dados reais de produtividade da equipe e com a definição do regime de contratação. A gestão dos riscos, principalmente os relacionados a agendamentos concorrentes, isolamento entre clínicas e proteção de dados pessoais, é essencial para que o produto seja entregue com segurança e consistência.

## 10. Referências de salários


- Glassdoor Product Owner (Brasil), faixa base R$ 7 mil R$ 11 mil/mês, média aproximada R$ 9 mil. https://www.glassdoor.com.br/Sal%C3%A1rios/product-owners-sal%C3%A1rioSRCH_KO0%2C14.htm
- Glassdoor Desenvolvedor Java (Brasil), faixa base consultada R$ 4 mil R$ 11 mil/mês conforme perfil listado. https://www.glassdoor.com.br/Sal%C3%A1rios/software-engineerjava-developer-sal%C3%A1rio-SRCH_KO0%2C32.htm
- Glassdoor Desenvolvedor Front-end (Brasil), faixa base R$ 4 mil R$ 8 mil/mês. https://www.glassdoor.com.br/Sal%C3%A1rios/desenvolvedor-front-end-developersal%C3%A1rio-SRCH_KO0%2C33.htm
- Glassdoor Analista de Teste QA (Brasil), faixa base R$ 4 mil R$ 7 mil/mês. https://www.glassdoor.com.br/Sal%C3%A1rios/analista-de-teste-qa-sal%C3%A1rioSRCH_KO0%2C20.htm
- Glassdoor DevOps Engineer (Brasil), faixa base R$ 7 mil R$ 13 mil/mês. https://www.glassdoor.com.br/Sal%C3%A1rios/devops-engineer-sal%C3%A1rioSRCH_KO0%2C15.htm
- Glassdoor UX/UI Designer (Brasil), faixa base R$ 3 mil R$ 7 mil/mês. https://www.glassdoor.com.br/Sal%C3%A1rios/ux-ui-designer-sal%C3%A1rioSRCH_KO0%2C14.htm
- Glassdoor Analista de Segurança da Informação (Brasil), faixa base R$ 4 mil R$ 8 mil/mês. https://www.glassdoor.com.br/Sal%C3%A1rios/analista-seguranca-da-informacaosal%C3%A1rio-SRCH_KO0%2C32.htm


Consulta realizada em 07/11/2026. As páginas do Glassdoor apresentam faixas e médias estimadas com base em dados informados por usuários e modelos da plataforma; os valores podem mudar ao longo do tempo.

# VittaCare

Documentação Dev – Bloco 8

# Sumário

1. Introdução ..... 2
2. Estrutura prevista do repositório: ..... 2
2.1 O que é ..... 3
2.2 Pré-requisitos ..... 3
2.3 Passo a passo ..... 3
2.4 Endereços locais ..... 4
2.5 Rodar sem Docker ..... 4
2.6 Testes ..... 5
2.7 Problemas comuns ..... 5
3. Variáveis de ambiente ..... 5
4. Contrato de API (OpenAPI 3.0) ..... 7
4.1 Especificação ..... 7
4.2 Endpoints ..... 8
4.3 Exemplo: agendar consulta ..... 10
4.4 Erros ..... 11
5. Guia de padrões de código ..... 11
5.1 Tecnologias de referência ..... 11
5.2 Estrutura do back-end ..... 12
5.3 Convenções de nomenclatura ..... 12
5.4 Camadas ..... 13
5.5 Regras obrigatórias do domínio ..... 13
5.6 Exemplo: serviço de agendamento (alinhado ao C4 do Bloco 3) ..... 14
5.8 Boas práticas ..... 16
6. Git workflow ..... 16
6.1 Branches ..... 16
6.2 Commits (Conventional Commits) ..... 16
6.3 Processo ..... 17
6.4 Pull Requests e revisão ..... 17
6.5 Versionamento ..... 17
6.6 Relação com a Definition of Done ..... 17
7. Sprint 1: fundação do projeto e autenticação básica ..... 18

7.1 Visão geral ..... 18
7.2 Backlog da sprint ..... 18
7.3 Critérios de aceite ..... 19
7.4 Plano sugerido por dia ..... 21
7.5 DoR, DoD, riscos e escopo ..... 21
8. Coerência com os demais blocos ..... 21
9. Limites e pontos a validar ..... 22

# 1. Introdução

Este documento reúne tudo o que uma pessoa desenvolvedora precisa para começar a trabalhar no VittaCare no dia seguinte, sem fazer perguntas: como rodar o projeto (README), variáveis de ambiente, contrato da API (OpenAPI 3.0), padrões de código, fluxo Git e o detalhamento da Sprint 1.

Tudo aqui deriva dos blocos anteriores: requisitos (Bloco 2), arquitetura e ADRs (Bloco 3), UML (Bloco 4), banco de dados (Bloco 5), planejamento (Bloco 7) e qualidade (Bloco 9). Quando o texto cita RF, RNF, ADR, US ou R0x, a origem está nesses blocos.

# 2. Estrutura prevista do repositório:

vittacare/

├— README.md

├— .env.example

├— .gitignore

├— CODING_STANDARDS.md

├— GIT_WORKFLOW.md

├— SPRINT_1.md

├— docs/

│ ├— api/

```
│   │   └── openapi.yaml
│   ├── blocos/
│   │   ├── 01-concepcao-e-escopo.pdf
│   │   ├── 02-requisitos.pdf
│   │   ├── 03-arquitetura.pdf
│   │   ├── 04-diagramas-uml.pdf
│   │   ├── 05-banco-de-dados.pdf
│   │   ├── 07-planejamento-e-estimativas.pdf
│   │   ├── 08-documentacao-dev.pdf
│   │   ├── 09-qualidade-e-processo.pdf
│   │   └── VittaCare_Documentacao_Completa.pdf   (todos os blocos juntos)
│   ├── diagramas/
│   │   ├── vittacare-uml.drawio  (arquivo editável)
├── backend/    (criar no desenvolvimento)
├── frontend/   (criar no desenvolvimento)
```

## 2.1 O que é

VittaCare é uma plataforma web SaaS para gestão de clínicas e ambulatórios, focada em agendamento de consultas para pacientes, profissionais de saúde, recepcionistas e administradores. Back-end em Java com Spring Boot, front-end em React e banco PostgreSQL.

## 2.2 Pré-requisitos

- Git.
- Docker e Docker Compose (caminho recomendado).
- Para rodar sem Docker: Java 21 (LTS), Node.js (LTS) e PostgreSQL. As versões finais são confirmadas na tarefa S1-01.

## 2.3 Passo a passo

# 1. Clonar o repositório (substitua pela URL real)

git clone https://github.com/<organizacao>/vittacare.git

cd vittacare

# 2. Criar o arquivo de variáveis de ambiente

cp .env.example .env

# Edite o .env: troque POSTGRES_PASSWORD, JWT_SECRET e SEED_DEV_PASSWORD

# 3. Subir tudo (banco, API e front-end)

docker compose up --build

Na primeira execução, as migrations criam as tabelas e o seed de desenvolvimento cria uma clínica de teste e um usuário fictício para cada perfil, com a senha definida em SEED_DEV_PASSWORD.

## 2.4 Endereços locais

| SERVIÇO | ENDEREÇO |
|---|---|
| **FRONT-END** | http://localhost:3000 |
| **API** | http://localhost:8080/api/v1 |
| **SWAGGER UI** | http://localhost:8080/swagger-ui/index.html |
| **HEALTH CHECK** | http://localhost:8080/actuator/health |
| **POSTGRESQL** | localhost:5432 |

## 2.5 Rodar sem Docker

# Banco: crie o banco e o usuário conforme o .env (POSTGRES_HOST=localhost)

# Back-end (exemplo com Maven; use o comando equivalente se o projeto usar Gradle)

cd backend

./mvnw spring-boot:run

# Front-end

cd frontend

npm install

npm run dev

Os comandos exatos de build ficam registrados aqui assim que a ferramenta for definida na S1-01.

### 2.6 Testes

cd backend && ./mvnw test  # JUnit 5 + Testcontainers (exige Docker)

cd frontend && npm test  # Jest + React Testing Library

Metas de cobertura e estratégia completa: Bloco 9.

### 2.7 Problemas comuns

- **Porta ocupada (3000, 8080 ou 5432):** altere a porta no .env ou encerre o processo que a usa.
- **API não conecta ao banco no Docker:** dentro do Compose, POSTGRES_HOST deve ser o nome do serviço do banco (ex.: db), não localhost.
- **401 em todas as chamadas:** o token expirou ou o JWT_SECRET mudou; faça login novamente.

## 3. Variáveis de ambiente

O arquivo .env.example (na raiz) é o modelo versionado. O .env real nunca vai para o Git. O .gitignore contém .env, .env.* e a exceção !.env.example. Em staging e produção, os segredos vêm do gerenciador de segredos da AWS, nunca de arquivo.

| Variável | Descrição | Valor de exemplo (dev) |
|---|---|---|
| **SPRING_PROFILES_ACTIVE** | Perfil da aplicação (dev, staging, prod) | dev |
| **SERVER_PORT** | Porta da API | 8080 |

| | | |
|---|---|---|
| **APP_CORS_ALLOWED_ORIGINS** | Origem do front-end permitida | http://localhost:3000 |
| **POSTGRES_HOST / PORT / DB** | Endereço e nome do banco | localhost / 5432 / vittacare |
| **POSTGRES_USER / PASSWORD** | Credenciais do banco | vittacare / troque-esta-senha |
| **SPRING_DATASOURCE_URL / USERNAME / PASSWORD** | Conexão JDBC montada a partir das anteriores | jdbc:postgresql://... |
| **JWT_SECRET** | Chave de assinatura do JWT (mín. 32 caracteres, aleatória, por ambiente) | gerar com openssl rand -base64 48 |
| **JWT_EXPIRATION_MINUTES** | Validade do token de acesso | 60 |
| **PASSWORD_RESET_TOKEN_MINUTES** | Validade do token de redefinição de senha (RF002) | 30 |
| **SEED_DEV_PASSWORD** | Senha dos usuários fictícios do seed (somente dev) | troque-senha-de-teste |

| | | |
|---|---|---|
| **NOTIFICACAO_PROVIDER_URL / API_KEY** | Provedor externo de e-mail/SMS | https://api.provedor.example/v1 |
| **NOTIFICACAO_REMETENTE** | Remetente das notificações | nao-responda@vittacare.example |
| **VITE_API_URL** | URL base da API usada pelo front-end (o prefixo depende da ferramenta de build definida na S1-01) | http://localhost:8080/api/v1 |
| **FRONTEND_PORT** | Porta do front-end | 3000 |

# 4. Contrato de API (OpenAPI 3.0)

## 4.1 Especificação

| Item | Definição |
|---|---|
| **Nome / versão** | VittaCare API 1.0.0 |
| **Padrão** | OpenAPI 3.0.3 (arquivo docs/openapi.yaml, visualizado no Swagger UI) |
| **Arquitetura** | REST, JSON sobre HTTPS (ADR-005) |
| **Base** | /api/v1 |
| **Autenticação** | JWT Bearer (ADR-006); o token carrega perfil e clínica |
| **Identificadores e datas** | UUID (Bloco 5) e ISO-8601 com fuso (OffsetDateTime) |
| **Isolamento** | O clinicaId nunca prova autorização: vale a clínica do token (RF025, RNF014). Única exceção: autorregistro anônimo de paciente |

| | |
|---|---|
| **Listagens** | Parâmetros page (a partir de 0) e size |

## 4.2 Endpoints

| Método | Endpoint | Funcionalidade | Perfil | Requisito |
|---|---|---|---|---|
| **POST** | /auth/login | Autenticar | Público | RF001 |
| **POST** | /auth/senha/recuperacao | Solicitar redefinição de senha | Público | RF002 |
| **POST** | /auth/senha/redefinicao | Redefinir senha | Público | RF002 |
| **POST, GET** | /usuarios | Cadastrar e listar contas | Administrador | RF003 |
| **GET, PATCH** | /usuarios/me | Ver e atualizar os próprios dados | Autenticado | RF004 |
| **PATCH** | /usuarios/{id}/status | Ativar ou desativar conta | Administrador | RF003 |
| **PUT** | /usuarios/{id}/papeis | Definir papéis | Administrador | RF024 |
| **POST** | /pacientes | Cadastrar paciente (autorregistro ou recepção) | Público / Recepção / Admin | RF005 |
| **GET** | /pacientes | Pesquisar pacientes | Recepção, Admin | RF006 |
| **GET, PATCH** | /pacientes/{id} | Consultar e atualizar paciente | Paciente (próprio), Recepção, Admin | RF006 |
| **POST, GET** | /profissionais | Cadastrar e pesquisar por | Admin (POST), | RF007, RF011 |

| | | | | |
|---|---|---|---|---|
| | | nome ou especialidade | Autenticado (GET) | |
| **GET, PATCH** | /profissionais/{id} | Consultar, atualizar, desativar | Admin (PATCH) | RF007 |
| **PUT** | /profissionais/{id}/especialidades | Associar especialidades | Administrador | RF009 |
| **GET, POST** | /especialidades | Listar e cadastrar | Autenticado (GET), Admin (POST) | RF010, RF008 |
| **PATCH** | /especialidades/{id} | Atualizar ou desativar | Administrador | RF008 |
| **GET** | /disponibilidades | Consultar horários de um profissional | Autenticado | RF012 |
| **POST** | /disponibilidades | Cadastrar horário | Profissional | RF013 |
| **PATCH** | /disponibilidades/{id} | Atualizar horário | Profissional | RF013 |
| **POST** | /disponibilidades/bloqueios | Bloquear horário (409 se houver agendamento ativo) | Profissional | RF013, RF026 |
| **POST** | /agendamentos | Agendar consulta (409 em conflito) | Paciente, Recepção | RF014, RF015, RF018 |
| **GET** | /agendamentos | Minhas Consultas e histórico | Autenticado (escopo por perfil) | RF019 |
| **GET** | /agendamentos/{id} | Consultar agendamento | Autorizados | RF019 |

| | | | | |
|---|---|---|---|---|
| **PATCH** | /agendamentos/{id} | Remarcar consulta | Paciente, Recepção | RF017 |
| **PATCH** | /agendamentos/{id}/cancelamento | Cancelar consulta | Paciente, Recepção | RF016 |
| **PATCH** | /agendamentos/{id}/confirmacao | Confirmar presença (AGENDADO para CONFIRMADO) | Paciente, Recepção | Bloco 5 §5.1 |
| **POST** | /agendamentos/{id}/atendimento | Registrar comparecimento ou ausência | Profissional, Recepção | RF020 |
| **GET** | /relatorios/consultas | Relatório de agendamentos, cancelamentos e ausências | Administrador | RF023 |

Notificações (RF021, RF022) são internas e disparadas pelo Service de agendamento; não há endpoint público.

## 4.3 Exemplo: agendar consulta

POST /api/v1/agendamentos (paciente autenticado; o paciente é o do token)


{

"profissionalId": "9b2e4c7a-1d38-4f6b-8a52-6c1e0d7f3b44",

"disponibilidadeId": "c7a1e5d2-4b90-4e3a-a6f1-8d2b3c9e0f57"

}


Resposta 201 Created:


{

"id": "5d0e8a31-6b2c-4f79-b1a4-9e3c7d2f0a68",

"pacienteId": "3f1c2b9e-7a54-4d1e-9c0b-2f6a8e5d1a10",

"profissionalId": "9b2e4c7a-1d38-4f6b-8a52-6c1e0d7f3b44",

"disponibilidadeId": "c7a1e5d2-4b90-4e3a-a6f1-8d2b3c9e0f57",

"inicio": "2026-11-15T14:00:00-03:00",

"fim": "2026-11-15T14:30:00-03:00",

"status": "AGENDADO"

}

O recepcionista também envia pacienteId (agendamento em nome do paciente, RF015). Duas requisições concorrentes para o mesmo horário: a segunda recebe 409 (índice único parcial uq_agendamento_disponibilidade_ativa, RF018 e RNF008).

### 4.4 Erros

Todas as falhas usam o formato { timestamp, status, codigo, mensagem, detalhes[] }.

| HTTP | Quando | Exemplo de código |
|---|---|---|
| **400** | Entrada inválida | VALIDACAO |
| **401** | Credenciais inválidas, token ausente, expirado ou inválido | NAO_AUTENTICADO |
| **403** | Perfil sem permissão ou recurso de outra clínica | SEM_PERMISSAO |
| **404** | Recurso não existe na clínica do usuário | NAO_ENCONTRADO |
| **409** | Conflito de horário ou bloqueio com agendamento ativo | CONFLITO_AGENDAMENTO, BLOQUEIO_NAO_PERMITIDO |

Status válidos: usuário (ATIVO, INATIVO, BLOQUEADO), disponibilidade (DISPONIVEL, BLOQUEADO, AGENDADO), agendamento (AGENDADO, CONFIRMADO, CANCELADO, REALIZADO) e presença (PRESENTE, AUSENTE), iguais aos CHECKs do Bloco 5.

## 5. Guia de padrões de código

### 5.1 Tecnologias de referência

| Camada | Tecnologia | Origem |
|---|---|---|
| **Back-end** | Java + Spring Boot, monólito modular | ADR-001, ADR-003 |

| | | |
|---|---|---|
| **Persistência** | Spring Data JPA / Hibernate | ADR-009 |
| **Segurança** | Spring Security + JWT | ADR-006 |
| **Front-end web** | React (JavaScript/JSX), SPA | ADR-002 |
| **Banco** | PostgreSQL (Amazon RDS Multi-AZ) | ADR-004 |
| **API** | REST + JSON, OpenAPI 3.0 | ADR-005, ADR-010 |
| **Containers** | Docker | ADR-008 |
| **Testes** | JUnit 5, Mockito, Testcontainers, Jest, React Testing Library, Playwright | Bloco 9 |

## 5.2 Estrutura do back-end

Código organizado por módulo de domínio e, dentro dele, por camada. Um módulo nunca acessa o Repository de outro: chama o Service dele.

com.vittacare

├— autenticacao ├— usuarios ├— clinicas

├— pacientes ├— profissionais ├— especialidades

├— disponibilidade ├— agendamentos ├— atendimentos

├— notificacoes ├— relatorios └— compartilhado

## 5.3 Convenções de nomenclatura

| Elemento | Padrão | Exemplo |
|---|---|---|
| **Classes** | PascalCase | AgendamentoService |
| **Métodos e variáveis** | camelCase | agendarConsulta(), inicioEm |
| **Constantes** | UPPER_SNAKE_CASE | LIMITE_TENTATIVAS_LOGIN |
| **Pacotes** | minúsculas, sem acento | com.vittacare.agendamentos |
| **DTOs** | RequestDTO / ResponseDTO | AgendamentoRequestDTO |

| | | |
|---|---|---|
| **Exceções** | sufixo Exception | ConflitoAgendamentoException |
| **Tabelas e colunas** | snake_case, singular (Bloco 5) | agendamento.disponibilidade_id |
| **Endpoints** | substantivos no plural | /api/v1/agendamentos |
| **Componentes React** | PascalCase | AgendaProfissional.jsx |
| **Testes** | deve + Resultado + Quando + Condição | deveLancarConflitoQuandoHorarioJaReservado |

## 5.4 Camadas

- **Controller:** recebe a requisição, valida a entrada e chama o Service. Sem regra de negócio.
- **Service:** regras de negócio, transações, permissões e contexto da clínica.
- **Repository:** apenas acesso a dados; toda consulta filtra por clinica_id.
- **DTO:** a API nunca expõe entidades JPA nem senha_hash.

## 5.5 Regras obrigatórias do domínio

1. **Isolamento por clínica (RF025, RNF014):** o clinicaId vem sempre do token JWT, via ContextoClinica.
2. **Autorização no back-end (RNF010):** todo endpoint protegido valida assinatura, expiração, perfil e clínica. Esconder um botão no React não é segurança.
3. **Agendamento (RF018, RNF008):** criação em transação; a violação do índice único parcial vira ConflitoAgendamentoException (HTTP 409).
4. **Bloqueio de horário (RF026):** verificar agendamentos ativos e gravar o bloqueio na mesma transação; com agendamento ativo, HTTP 409.
5. **Senhas (RNF011):** apenas hash Argon2id ou bcrypt; nunca em texto puro nem em log.
6. **Segredos:** nunca no repositório; use variáveis de ambiente.
7. **LGPD (RNF012):** sem CPF, e-mail ou telefone em logs.
8. **Auditoria (RNF013):** operações administrativas críticas gravam registro_auditoria.

9. **Datas:** OffsetDateTime no Java, TIMESTAMPTZ no banco, ISO-8601 na API.

## 5.6 Exemplo: serviço de agendamento (alinhado ao C4 do Bloco 3)

```
@Service
public class AgendamentoService {

  private final AgendamentoRepository agendamentoRepository;
  private final DisponibilidadeRepository disponibilidadeRepository;
  private final NotificacaoService notificacaoService;

  public AgendamentoService(AgendamentoRepository agendamentoRepository,
                            DisponibilidadeRepository disponibilidadeRepository,
                            NotificacaoService notificacaoService) {
    this.agendamentoRepository = agendamentoRepository;
    this.disponibilidadeRepository = disponibilidadeRepository;
    this.notificacaoService = notificacaoService;
  }

  @Transactional
  public AgendamentoResponseDTO criar(AgendamentoRequestDTO req,
ContextoClinica ctx) {
    validarContexto(ctx);

    Disponibilidade disponibilidade = disponibilidadeRepository
        .findByClinicaIdAndId(ctx.clinicaId(), req.disponibilidadeId())
        .orElseThrow(() -> new RecursoNaoEncontradoException("Disponibilidade
não encontrada"));
```

```
    boolean ocupada = disponibilidade.getStatus() !=
StatusDisponibilidade.DISPONIVEL
            || agendamentoRepository.existeAtivoPorDisponibilidade(
                ctx.clinicaId(), disponibilidade.getId());
    if (ocupada) {
      throw new ConflitoAgendamentoException("Horário indisponível");
    }

    Agendamento agendamento = Agendamento.novo(req, disponibilidade, ctx);
    try {
      Agendamento salvo = agendamentoRepository.save(agendamento);
      disponibilidade.marcarComoAgendada();
      notificacaoService.enviarConfirmacao(salvo); // falha de envio não desfaz o
agendamento (R08)
      return AgendamentoResponseDTO.de(salvo);
    } catch (DataIntegrityViolationException e) {
      throw tratarConflito(e); // última barreira: índice único parcial
    }
  }
}
```

**5.7 Testes e cobertura (Bloco 9)**

- Back-end: linhas ≥ 80% e branches ≥ 70%. Módulos críticos (autenticação, autorização, agendamento, tenancy): ≥ 90% de linhas nas regras de domínio. Front-end: ≥ 70%.
- Obrigatórios: teste de acesso cruzado entre clínicas e teste de concorrência no agendamento.
- Todo defeito corrigido ganha teste de regressão. Não reduzir a meta só para o pipeline passar.

### 5.8 Boas práticas

Nomes claros; princípios SOLID; sem duplicação; injeção de dependência por construtor; validação de toda entrada (Bean Validation); métodos pequenos; migrations versionadas e nunca alteradas depois de aplicadas; contrato OpenAPI atualizado no mesmo PR que altera a API; no front-end, estados de carregamento, erro, vazio e sucesso, navegação por teclado e rótulos textuais (RNF017).

## 6. Git workflow

Git e GitHub (ADR-011).

### 6.1 Branches

| Branch | Finalidade | Origem | Destino |
|---|---|---|---|
| **main** | Versões estáveis (com tags) | — | — |
| **develop** | Integração das funcionalidades revisadas | main | main (release) |
| **feature/*** | Nova funcionalidade ou tarefa | develop | develop |
| **fix/*** | Correção de defeito | develop | develop |
| **docs/*** | Documentação técnica | develop | develop |
| **hotfix/*** | Correção urgente em produção | main | main e develop |

Nomes em minúsculas, com hífen e sem acento (ex.: feature/autenticacao-jwt, fix/validacao-horario, docs/contrato-api). main e develop são protegidas: sem push direto; só por Pull Request com revisão aprovada e checks do pipeline verdes.

### 6.2 Commits (Conventional Commits)

Formato: tipo(escopo): descrição no imperativo. Tipos: feat, fix, docs, refactor, test, chore, style.

feat(autenticacao): adicionar login com JWT

fix(agendamentos): corrigir conflito de horários concorrentes

docs(api): atualizar contrato OpenAPI

test(agendamentos): adicionar teste de concorrência

chore(ci): configurar pipeline inicial

## 6.3 Processo

1. Atualizar a develop (git checkout develop && git pull).
2. Criar a branch de trabalho (git checkout -b feature/nome-da-tarefa).
3. Desenvolver seguindo o guia de padrões, com commits pequenos.
4. Rodar testes, lint e análise estática localmente.
5. Publicar a branch e abrir Pull Request para a develop.
6. Corrigir a revisão e aguardar aprovação.
7. Integrar por squash merge e apagar a branch.

## 6.4 Pull Requests e revisão

Todo PR tem título objetivo, descrição, vínculo com a tarefa ou história (ex.: US002, RF001, S1-05), evidência de testes, impactos em API, banco ou variáveis de ambiente e pelo menos 1 revisor diferente do autor. Critérios de aprovação: checks verdes (build, lint, testes, cobertura, segurança), padrões respeitados, entrada validada e documentação atualizada. Exceção de cobertura só com justificativa no PR e tarefa para corrigir a lacuna.

O revisor avalia: legibilidade, conformidade com os padrões, duplicação, tratamento de exceções, segurança (autorização, isolamento por clinica_id, segredos e dados pessoais em log), testes (inclusive cenários negativos e de concorrência) e documentação.

## 6.5 Versionamento

Versionamento semântico. Ao fim de cada sprint estável, a develop é integrada à main por PR e recebe tag (ex.: v0.1.0 ao fim da Sprint 1). Hotfix parte da main, volta para a main com nova tag e é mesclado na develop.

## 6.6 Relação com a Definition of Done

Uma tarefa só é Done quando o PR foi revisado e integrado na develop e o pipeline passou. A branch principal de integração citada no DoD do Bloco 9 é a develop; a main recebe apenas versões estáveis.

# 7. Sprint 1: fundação do projeto e autenticação básica

Segue o Bloco 7 (Sprint 1: fundação do projeto, estrutura React/Spring Boot, banco, pipeline inicial e autenticação básica) e a decisão do Bloco 9 de configurar o pipeline nesta sprint.

## 7.1 Visão geral

| Item | Definição |
|---|---|
| **Sprint** | 01 de 08 |
| **Duração** | 2 semanas (10 dias úteis) |
| **Capacidade** | 20 Story Points (referência a validar após as Sprints 1 e 2) |
| **Equipe** | 2 devs back-end, 1 dev front-end, 1 QA; PO, UX/UI e DevOps em apoio parcial |
| **Objetivo** | Deixar o projeto executável em qualquer máquina e entregar o primeiro fluxo ponta a ponta: login com JWT |
| **Prioridade** | Alta |

## 7.2 Backlog da sprint

| ID | Item | Referência | SP |
|---|---|---|---|
| **S1-01** | Estrutura do repositório e dos projetos (Spring Boot + React) | ADR-001, ADR-002, ADR-003 | 3 |
| **S1-02** | Ambiente local com Docker Compose e .env.example | ADR-008 | 3 |
| **S1-03** | Migrations iniciais (identidade e acesso) e dados de teste | Bloco 5, RNF008, RNF014 | 3 |
| **S1-04** | Pipeline de CI inicial (build, lint, testes, cobertura, segurança) | Bloco 9 §3.3 e §10 | 3 |
| **S1-05** | Spring Security + JWT (filtro, claims de perfil e clínica, 401/403) | RF001, RNF009 a RNF011, ADR-006 | 3 |

| | | | |
|---|---|---|---|
| **S1-06** | US002: login com e-mail e senha (endpoint + tela React) | RF001, RNF015 a RNF017 | 3 |
| **S1-07** | Swagger/OpenAPI base e contrato inicial da API | ADR-010 | 2 |
| | **Total** | | **20** |

## 7.3 Critérios de aceite

**S1-01 Estrutura do repositório**

- Repositório com backend/, frontend/ e docs/ e pacotes por módulo conforme o guia de padrões.
- GET /actuator/health responde 200 com a API rodando.
- O front React abre a página inicial e fala com a API pela URL configurada em variável de ambiente.
- Versões de Java, Spring Boot e Node confirmadas e registradas no README.
- Branches main e develop criadas e protegidas, com modelo de PR configurado.

**S1-02 Ambiente local**

- docker compose up sobe PostgreSQL, API e front sem passos manuais além de copiar .env.example para .env.
- Nenhum segredo versionado; .env está no .gitignore.
- README descreve o caminho do git clone até o localhost funcionando.

**S1-03 Migrations iniciais**

- Migrations versionadas (ferramenta a registrar em ADR; proposta: Flyway) criam clinica, usuario, papel, permissao, usuario_papel e papel_permissao exatamente como o DDL do Bloco 5, incluindo FKs compostas por clinica_id.
- Seed de desenvolvimento com 1 clínica de teste, os 4 papéis e 1 usuário fictício por perfil, com senha vinda do .env, nunca do repositório.
- A migration roda do zero em banco vazio (teste com Testcontainers).

- Teste de integração confirma e-mail único por clínica e que a FK composta impede referência entre clínicas.

**S1-04 Pipeline de CI**

- A cada PR para a develop: lint, build, testes unitários e de integração, cobertura (JaCoCo e Jest) e verificação de dependências e segredos.
- PR com check vermelho não pode ser integrado; relatório de cobertura disponível no PR.

**S1-05 Spring Security + JWT**

- Given um token válido, When acessa endpoint protegido, Then a requisição passa com perfil e clinicaId do token em ContextoClinica.
- Given token ausente, expirado ou com assinatura inválida, When acessa endpoint protegido, Then resposta 401 no formato de erro do contrato.
- Given perfil sem permissão, When acessa, Then resposta 403.
- Chave e validade do token vêm de variáveis de ambiente; senhas verificadas com Argon2id ou bcrypt (RNF011).

**S1-06 US002: login**

Como paciente, quero entrar na plataforma utilizando meu e-mail e senha, para acessar minhas funcionalidades pessoais. (3 SP, Bloco 2)

- Given conta ATIVO, When informa credenciais válidas, Then 200 com token JWT e perfil, e a interface exibe as funcionalidades do perfil.
- Given senha incorreta, e-mail inexistente ou conta INATIVO ou BLOQUEADO, When tenta entrar, Then 401 com mensagem genérica, sem revelar se o e-mail existe.
- Tela de login com navegação por teclado e rótulos textuais (RNF017), compatível com Chrome, Firefox e Edge (RNF016).
- Testes: unitários do serviço de autenticação, integração do endpoint e um E2E do login.

**S1-07 OpenAPI base**

- Swagger UI acessível localmente exibindo ao menos POST /auth/login.
- docs/openapi.yaml versionado e igual ao publicado pela API.

## 7.4 Plano sugerido por dia

| Dias | Foco |
|---|---|
| 1 a 2 | S1-01 e S1-02 (estrutura e ambiente local) |
| 3 a 4 | S1-03 e S1-04 (banco e pipeline) |
| 5 a 6 | S1-05 (segurança e JWT) |
| 7 a 8 | S1-06 (login: back-end e tela) |
| 9 | S1-07 e correções |
| 10 | Revisão, Sprint Review e Retrospectiva |

## 7.5 DoR, DoD, riscos e escopo

- **DoR:** atendido para todos os itens (critérios verificáveis, vínculo com RF/RNF ou ADR, dependências identificadas, estimativa em SP, dados de teste não reais).
- **DoD:** critérios do Bloco 9 §6: critérios de aceite atendidos, código revisado e integrado na develop, testes e pipeline verdes, documentação, API e variáveis de ambiente atualizadas, sem segredos no repositório e demonstração ao PO.
- **Riscos:** R06 (velocidade menor que 20 SP: medir ao fim da sprint e replanejar com o PO), R05 (sobrecarga de pessoas-chave: revisão cruzada), R10 (defeitos em fluxos críticos: testes automatizados desde a primeira sprint).
- **Fora do escopo:** recuperação de senha e permissões completas (Sprint 2); cadastros (Sprint 3); pesquisa, disponibilidade e agendamento de consulta, UC01 (Sprints 4 a 6).
- **Demonstração:** git clone e docker compose up em máquina limpa; login com usuário de teste; tentativa com senha errada (401) e token expirado; Swagger UI com o contrato de login; pipeline rodando em um PR com relatório de cobertura.

# 8. Coerência com os demais blocos

| ITEM DO BLOCO 8 | ONDE SE APOIA |
|---|---|

| | |
|---|---|
| **STACK E TECNOLOGIAS DO GUIA DE CÓDIGO** | Bloco 3: stack, ADR-001 a ADR-010 |
| **EXEMPLO DE SERVIÇO DE AGENDAMENTO** | Bloco 3: C4 nível 4; Bloco 4: sequência de agendamento |
| **IDS UUID, STATUS E CHECKS DO CONTRATO** | Bloco 5: modelo físico |
| **ENDPOINTS E PERFIS** | Bloco 2: RF001 a RF026; Bloco 4: casos de uso |
| **RESPOSTAS 409 EM AGENDAMENTO E BLOQUEIO** | Bloco 2: RF018, RF026; Bloco 4: sequências 4.2 e 4.3 |
| **SPRINT 1 (20 SP, OBJETIVO E PIPELINE)** | Bloco 7: cronograma; Bloco 9: pipeline na Sprint 1 |
| **METAS DE COBERTURA, DOR E DOD** | Bloco 9 |
| **GIT E GITHUB** | ADR-011 |

## 9. Limites e pontos a validar

- **Login por e-mail:** o Bloco 5 permite o mesmo e-mail em clínicas diferentes (e-mail único por clínica), mas o RF001 e o diagrama de sequência autenticam apenas por e-mail e senha. O contrato prevê clinicaId opcional no login enquanto o grupo não fecha a regra.
- **Autorregistro de paciente:** é anônimo, então informa a clínica de destino; é a única exceção à regra de nunca confiar em clinicaId vindo do cliente.
- **Primeiro acesso de paciente cadastrado pela recepção (sem senha):** fluxo a confirmar com o PO.
- **Regras de cancelamento e remarcação:** a confirmar com o PO (Bloco 9 §10).
- **Ferramentas ainda a registrar na S1-01:** versões de Java, Spring Boot e Node, ferramenta de build do back-end e do front-end e ferramenta de migrations.
- **Endereços de exemplo:** repositório, domínios de staging e provedor de notificações são marcadores a substituir pelos reais.
- O documento propõe a documentação técnica para o desenvolvimento; os endpoints ainda não estão implementados.

# VittaCare

Qualidade e processo – Bloco 9

# Sumário

1. Objetivo ..... 2
2. Modelo de domínio e glossário ..... 3
3. Estratégia de testes ..... 4
3.1 Matriz mínima de cenários ..... 4
3.2 Testes não funcionais ..... 5
3.3 Ferramentas de teste ..... 6
4. Metas de cobertura de código ..... 6
5. Definition of Ready (DoR) ..... 6
6. Definition of Done (DoD) ..... 7
7. Metodologia de desenvolvimento ..... 7
8. Métricas e acompanhamento da qualidade ..... 8
9. Gestão de não conformidades ..... 8
10. Limites e pontos a validar ..... 9

## 1. Objetivo

Este documento define o vocabulário do domínio, a estratégia de testes, metas de cobertura, critérios para iniciar e concluir trabalho (DoR/DoD) e a metodologia de desenvolvimento do

VittaCare. O objetivo é tornar verificável a qualidade das entregas e permitir que a equipe trabalhe de forma consistente.

As definições respeitam o escopo da versão atual: gestão de clínicas, usuários, pacientes, profissionais, especialidades, disponibilidades, agendamentos, atendimentos, notificações e administração. Prontuário completo, prescrição, diagnóstico automatizado, teleconsulta, pagamentos e gestão financeira completa permanecem fora do escopo.

## 2. Modelo de domínio e glossário

| Termo | Definição no VittaCare |
|---|---|
| Clínica | Instituição contratante que delimita o conjunto de dados acessível no sistema. |
| Usuário | Identidade de acesso, com e-mail, senha protegida, situação da conta e vínculo com a clínica. |
| Paciente | Perfil de paciente associado a uma identidade de usuário. |
| Profissional de saúde | Perfil profissional responsável por disponibilidades e consultas. |
| Recepcionista | Perfil administrativo que cadastra/atualiza pacientes e realiza operações de agendamento na recepção. |
| Administrador | Perfil que gerencia contas, profissionais, especialidades e permissões. |
| Especialidade | Especialidade médica cadastrada para uma clínica. |
| Profissional_Especialidade | Associação entre profissionais e suas especialidades. |
| Disponibilidade | Intervalo de horário oferecido ou bloqueado por um profissional. |
| Agendamento | Reserva de uma disponibilidade para um paciente. |
| Atendimento | Registro de comparecimento ou ausência vinculado ao agendamento. |
| Notificação | Confirmação ou lembrete relacionado a um agendamento. |
| Papel | Agrupamento de permissões associado a um usuário. |
| Permissão | Ação autorizada a determinado papel/perfil. |
| Registro de auditoria | Histórico de operações administrativas e |
| | críticas, com usuário responsável e momento da operação. |

| | |
|---|---|
| clinica_id | Identificador usado para manter o isolamento lógico dos registros pertencentes a cada clínica. |

Regra transversal: o contexto da clínica deve ser obtido a partir da identidade autenticada no back-end. Um clinica_id enviado pelo front-end não constitui autorização. Consultas e operações devem respeitar o isolamento entre clínicas.

## 3. Estratégia de testes

A estratégia combina testes unitários, de integração e ponta a ponta (E2E). A pirâmide prioriza testes rápidos e numerosos nas regras de negócio, testes de integração para fronteiras entre camadas e banco/API, e testes E2E para os fluxos críticos percebidos pelos usuários.

| Nível | O que validar | Exemplos VittaCare |
|---|---|---|
| Unitário | Unidades isoladas, regras de negócio e validações, com dependências simuladas quando adequado. | Validação de disponibilidade; transição de status do agendamento; permissões; validação de dados; regras de cancelamento. |
| Integração | Comunicação real entre componentes, persistência, transações, segurança e API. | Repositories com PostgreSQL de teste; migrations; filtros por clinica_id; autenticação JWT; conflito concorrente no mesmo horário. |
| E2E | Fluxos completos pela interface e API, simulando o comportamento de um perfil. | Paciente autentica, pesquisa profissional, consulta horário e agenda; recepção agenda para paciente; profissional registra comparecimento. |

### 3.1 Matriz mínima de cenários

| Fluxo | Cenário positivo | Cenário negativo/limite |
|---|---|---|
| Autenticação | Credenciais válidas permitem acesso conforme perfil. | Senha incorreta, conta inativa e token ausente/expirado são rejeitados. |
| Pesquisa | Pesquisa por nome/especialidade retorna resultados da clínica. | Pesquisa sem resultados; usuário não acessa dados de outra clínica. |
| Disponibilidade | Horários livres são apresentados atualizados. | Horário bloqueado ou já reservado não pode ser escolhido. |

| | | |
|---|---|---|
| Agendamento | Reserva válida é registrada e disponibilidade atualizada. | Duas requisições concorrentes não podem confirmar a mesma vaga; falha não deixa registro incompleto. |
| Cancelamento/remarcação | Status é atualizado e vaga liberada conforme regra. | Usuário não autorizado não altera agendamento de terceiros. |
| Permissões | Administrador gerencia papéis/permissões. | Paciente ou profissional não executa operação administrativa. |
| Auditoria | Operação crítica registra ator e instante. | Falha ao auditar operação crítica deve ser tratada conforme política definida; não ocultar inconsistência. |

### 3.2 Testes não funcionais

| Requisito | Como verificar |
|---|---|
| RNF001 | Teste de carga/monitoramento: p95 de requisições comuns ≤ 300 ms em condições normais. |
| RNF002 | Medição de pesquisa de profissionais e consulta de horários: p95 < 2 s. |
| RNF003 | p95 ≤ 2 s. |
| RNF004 | Teste de carga até 10.000 usuários cadastrados e pico de 1.000 simultâneos, observando os limites de desempenho. |
| RNF005 | Verificar aumento de CPU/memória da aplicação sem alteração de código. |
| RNF006 | Monitorar disponibilidade mensal mínima de 99,9%, excluindo manutenção programada Medição do fluxo completo de agendamento: comunicada. |
| RNF007 | Exercício de restauração: RPO ≤ 15 min, RTO ≤ 4 h, retenção diária ≥ 7 dias e semanal ≥ 4 semanas. |
| RNF008 | Testes de integridade e concorrência para impedir agendamentos incompletos ou duplicados. |
| RNF009–RNF011 | Verificar HTTPS/TLS 1.2+, proteção de endpoints e hash de senha com Argon2id ou bcrypt. |
| RNF012–RNF014 | Revisar classificação de dados, trilha de auditoria e isolamento entre clínicas. |

| | |
|---|---|
| RNF015–RNF017 | Testes de usabilidade, compatibilidade com Chrome/Firefox/Edge e navegação por teclado. |

### 3.3 Ferramentas de teste

• Testes unitários do back-end: JUnit 5 e Mockito.
• Testes de integração: Spring Boot Test com Testcontainers (PostgreSQL real em contêiner).
• Testes do front-end: Jest e React Testing Library.
• Testes E2E dos fluxos críticos: Playwright.
• Cobertura de código: JaCoCo (back-end) e cobertura nativa do Jest (front-end), medidas no pipeline.
• Testes de carga (RNF001 a RNF004): k6.

## 4. Metas de cobertura de código

As metas abaixo são metas operacionais propostas para o projeto; os blocos anteriores não definem percentuais de cobertura. A cobertura deve ser medida por ferramenta automatizada no pipeline, sem substituir a avaliação da qualidade dos casos de teste.

| Escopo | Meta proposta | Critério complementar |
|---|---|---|
| Back-end — linhas | ≥ 80% | Cobrir regras de negócio e caminhos de erro relevantes. |
| Back-end — branches | ≥ 70% | Priorizar decisões de autorização, disponibilidade e transições de status. |
| Módulos críticos (autenticação, autorização, agendamento e tenancy) | ≥ 90% de linhas nas regras de domínio | Cobrir também cenários negativos e de concorrência. |
| Front-end — componentes e lógica | ≥ 70% | Cobrir estados de carregamento, erro, vazio e sucesso nos fluxos principais. |
| E2E | 100% dos fluxos críticos definidos na matriz mínima | Não se aplica como percentual de linhas; medir cobertura de jornadas. |

Exceções temporárias devem ser justificadas no Pull Request, acompanhadas de tarefa para corrigir a lacuna e aprovadas pela equipe. Não reduzir a meta apenas para fazer o pipeline passar.

## 5. Definition of Ready (DoR)

Uma história/tarefa está pronta para entrar em uma sprint quando todos os itens aplicáveis forem atendidos:

- Está descrita sob a perspectiva do usuário ou como tarefa técnica claramente delimitada.
- Possui vínculo com requisito RF/RNF ou necessidade técnica documentada.

- Tem critérios de aceitação objetivos e verificáveis; quando funcional, preferencialmente Given/When/Then.
- Está priorizada e estimada em Story Points pela equipe.
- Dependências, integrações, regras de autorização e impactos em dados/API foram identificados.
- Possui exemplos de entrada/saída e regras de erro relevantes.
- Está pequena o suficiente para ser concluída e revisada dentro da sprint.
- Há condições para testar a entrega, incluindo dados de teste não reais quando necessário.

## 6. Definition of Done (DoD)

Uma história só é considerada concluída quando todos os itens aplicáveis forem atendidos: •

Implementação atende a todos os critérios de aceitação e ao escopo aprovado.

- Código segue os padrões definidos, foi revisado por outra pessoa e está integrado à branch principal.
- Testes unitários e de integração relevantes foram criados/atualizados e passaram.
- Fluxos críticos afetados possuem cobertura E2E ou justificativa registrada para sua ausência.
- Checks de build, lint, segurança e testes do pipeline passaram.
- API, banco, variáveis de ambiente e documentação foram atualizados quando houve mudança.
- Migrations são versionadas e testadas; alterações respeitam integridade e isolamento por clinica_id.
- Não há segredos no repositório, dados pessoais em logs ou vulnerabilidades críticas conhecidas sem tratamento aprovado.
- Critérios de acessibilidade e compatibilidade aplicáveis foram verificados.
- A entrega foi demonstrada/validada pelo responsável pelo produto e está pronta para integração/implantação conforme o fluxo da equipe.

## 7. Metodologia de desenvolvimento

Metodologia escolhida: Scrum, com sprints de duas semanas. Essa escolha mantém a decisão registrada no Bloco 7, que prevê 16 semanas (oito sprints), equipe principal com dois desenvolvedores back-end, um front-end e um QA, além de apoio parcial de PO, UX/UI, DevOps e segurança. A velocidade inicial de referência é 20 Story Points por sprint, a validar após as duas primeiras sprints.

Justificativa: o projeto tem prazo e escopo definidos (160 Story Points em 16 semanas), prioridades já classificadas por MoSCoW e necessidade de validar incrementos com o Product Owner a cada duas semanas. As sprints de duração fixa permitem medir a velocidade real após as duas primeiras sprints e replanejar o escopo, por exemplo deslocando o RF023, antes que o prazo seja comprometido.

Alternativa descartada: o Kanban não oferece essa cadência de revisão nem a medição de velocidade por ciclo, embora seja adequado para fluxo contínuo de manutenção após a entrega.

**Eventos e artefatos**

| Elemento | Aplicação no projeto |
|---|---|
| Planejamento da Sprint | Selecionar histórias prontas, definir objetivo e capacidade; evitar exceder a referência de 20 |
| | SP. |
| Daily Scrum | Reunião curta diária para inspecionar progresso e impedimentos. |
| Refinamento do backlog | Detalhar histórias, critérios, dependências e estimativas antes de entrarem na sprint. |
| Sprint Review | Demonstrar incremento funcional e coletar feedback do PO/representantes. |
| Retrospectiva | Identificar melhorias no processo e acompanhar ações da sprint anterior. |
| Product Backlog | Lista priorizada de histórias, requisitos e débitos técnicos. |
| Sprint Backlog | Itens selecionados e plano de execução da sprint. |

## 8. Métricas e acompanhamento da qualidade

- Velocidade: Story Points concluídos e aceitos por sprint; comparar com a referência inicial de 20 SP.
- Previsibilidade: itens planejados versus itens concluídos.
- Defeitos: quantidade por severidade, origem e tempo de resolução.
- Cobertura: linhas/branches por módulo e tendência entre sprints.
- Desempenho: p95 das operações críticas, confrontado com RNF001 RNF003.
- Confiabilidade: disponibilidade, sucesso de restauração e cumprimento de RPO/RTO.
- Segurança: endpoints protegidos, falhas de autorização e vulnerabilidades identificadas.

## 9. Gestão de não conformidades

1. Registrar o defeito com passos para reprodução, ambiente, resultado esperado e resultado obtido.
2. Classificar severidade e impacto, especialmente quando envolver dados de saúde, acesso indevido, isolamento entre clínicas ou integridade de agendamentos.
3. Corrigir, adicionar teste de regressão e submeter a revisão.
4. Reexecutar os testes relacionados e os checks do pipeline.
5. Documentar causa raiz para falhas críticas e definir ação preventiva.

## 10. Limites e pontos a validar

• Os percentuais de cobertura são metas propostas neste Bloco 9, não métricas já aprovadas nos documentos anteriores.
• As ferramentas listadas na seção 3.3 são a proposta do projeto; a configuração do pipeline (build, testes, cobertura e E2E) será feita na Sprint 1.
• As regras detalhadas de cancelamento, remarcação, retenção de dados e classificação de severidade precisam ser confirmadas com o PO.
• O Bloco 9 não amplia o escopo funcional nem substitui os requisitos e RNFs do Bloco 2.

# VittaCare

Telas do sistema – Anexo A

# Sumário

1. Introdução . . . . . 1

2. Visão geral das telas . . . . . 1

3. Especificação das telas . . . . . 2

3.1 Tela 01 — Página inicial do site . . . . . 2

3.2 Tela 02 — Login e recuperação de senha . . . . . 2

3.3 Tela 03 — Cadastro e consulta de pacientes . . . . . 3

3.4 Tela 04 — Dashboard administrativo . . . . . 3

3.5 Tela 05 — Agendamento de consulta . . . . . 4

3.6 Tela 06 — Gerenciamento de agendamentos . . . . . 5

3.7 Tela 07 — Agenda do profissional . . . . . 5

3.8 Tela 08 — Registro de atendimento . . . . . 6

3.9 Tela 09 — Relatórios administrativos . . . . . 6

3.10 Tela 10 — Usuários, permissões, profissionais e especialidades . . . . . 7

3.11 Tela 11 — Meus agendamentos (nova) . . . . . 7

3.12 Tela 12 — Meus dados (nova) . . . . . 8

4. Requisitos transversais às telas . . . . . 8

5. Ajustes realizados em relação ao material original . . . . . 9

6. Limites e pontos a validar . . . . . 9

# 1. Introdução

Este anexo especifica as telas do sistema web do VittaCare: o site institucional e os painéis de gestão dos quatro perfis de usuário (Paciente, Profissional de saúde, Recepcionista e Administrador). O conteúdo parte do material “Dados de tela” e foi ajustado para ficar coerente com os Blocos 1 a 5, 7, 8 e 9: nomenclatura dos perfis, requisitos funcionais e não funcionais, regras de negócio, contrato da API (Bloco 8), modelo de dados (Bloco 5) e escopo da versão atual (Bloco 1).

As telas serão implementadas no front-end React (SPA) e consomem a API REST em **/api/v1**, com JWT. A interface não substitui a autorização do back-end: ocultar uma opção de menu não é mecanismo de segurança (Bloco 3, seção 1.5). Cada tela indica os perfis que a utilizam, os requisitos de origem e os endpoints do contrato do Bloco 8, além das regras que precisam ser respeitadas para manter a coerência com os demais blocos.

A prioridade de cada tela herda a classificação MoSCoW dos requisitos a que ela atende (Bloco 2). Funcionalidades fora da versão atual (pagamentos, teleconsulta, prontuário, prescrição e diagnóstico automatizado) não aparecem em nenhuma tela.

# 2. Visão geral das telas

Foram mantidas as dez telas do material original, com ajustes, e acrescentadas duas telas necessárias para cobrir requisitos e histórias de usuário que não tinham tela correspondente (telas 11 e 12).

| Nº | Tela | Perfis | Requisitos | Prioridade |
|---|---|---|---|---|
| 01 | Página inicial do site | Público geral | RF001, RF005 (acesso) | Institucional (sem RF próprio) |
| 02 | Login e recuperação de senha | Todos os perfis | RF001, RF002, RNF009–RNF011 | Must |
| 03 | Cadastro e consulta de pacientes | Paciente, Recepcionista | RF005, RF006 | Must |
| 04 | Dashboard administrativo | Administrador | RF023, RF025 | Could |
| 05 | Agendamento de consulta | Paciente, Recepcionista | RF010–RF012, RF014, RF015, RF018, RF021 | Must |
| 06 | Gerenciamento de agendamentos | Recepcionista (operação), Administrador (acompanhamento) | RF015–RF019, RF025 | Must |
| 07 | Agenda do profissional | Profissional de saúde | RF013, RF019, RF026 | Must |
| 08 | Registro de atendimento | Profissional de saúde, Recepcionista | RF020 | Should |
| 09 | Relatórios administrativos | Administrador | RF023 | Could |
| 10 | Usuários, permissões, profissionais e especialidades | Administrador | RF003, RF007–RF009, RF024 | Must |
| 11 | Meus agendamentos (nova) | Paciente | RF016, RF017, RF019 | Must / Should |
| 12 | Meus dados (nova) | Todos os perfis autenticados | RF004 | Should |

# 3. Especificação das telas

## 3.1 Tela 01 — Página inicial do site

| | |
|---|---|
| **Perfil** | Público geral |
| **Requisitos** | RF001 e RF005 (pontos de entrada); página institucional, sem requisito funcional próprio |
| **API (Bloco 8)** | Nenhum endpoint de dados; os botões levam às telas 02 e 03 |

Página de apresentação do VittaCare, que divulga a proposta da plataforma e direciona os usuários para login, cadastro e agendamento.

**Funcionalidades principais:**

- Apresentar a proposta de valor e os benefícios da plataforma de gestão de consultas (Bloco 1, seção 1.8).
- Descrever as funcionalidades disponíveis para cada perfil.
- Oferecer os botões **Entrar**, **Cadastrar-se** (paciente) e **Agendar consulta**; este último direciona ao login quando o usuário não está autenticado.
- Disponibilizar informações institucionais e de contato.

**Regras e coerência com os blocos:**

- A lista de especialidades **não é pública**: o endpoint GET /especialidades exige autenticação e os dados pertencem a uma clínica (RF010, RF025, RNF010). A consulta de especialidades ocorre na tela 05, após o login.
- O site apresenta o produto (plano único de assinatura, Bloco 1); não oferece contratação nem pagamento, que estão fora do escopo (RF-W01).
- Interface responsiva e compatível com Chrome, Firefox e Edge (RNF016).

## 3.2 Tela 02 — Login e recuperação de senha

| | |
|---|---|
| **Perfil** | Todos os perfis |
| **Requisitos** | RF001, RF002, RF024, RNF009, RNF010, RNF011 |
| **API (Bloco 8)** | POST /auth/login; POST /auth/senha/recuperacao; POST /auth/senha/redefinicao |

Autentica os usuários que acessam o sistema pelo navegador e direciona cada perfil para a área autorizada.

**Funcionalidades principais:**

- Receber e-mail e senha e enviar a autenticação ao back-end por HTTPS.
- Em caso de sucesso, receber o token JWT (com perfil e clínica) e redirecionar para o painel do perfil: Paciente para a tela 05, Profissional de saúde para a tela 07, Recepcionista para a tela 06 e Administrador para a tela 04.
- Em caso de credenciais inválidas ou conta inativa, exibir mensagem de erro (resposta HTTP 401), conforme o diagrama de sequência de autenticação (Bloco 4, seção 4.1).
- Permitir a recuperação de senha: solicitar a redefinição por e-mail cadastrado e definir a nova senha por meio de token temporário.
- Tratar token ausente, expirado ou inválido encaminhando o usuário novamente ao login.

**Regras e coerência com os blocos:**

- Senhas protegidas com Argon2id ou bcrypt e comunicação com TLS 1.2 ou superior (RNF009, RNF011).
- O diagrama de classes prevê conta com status ATIVO, INATIVO ou BLOQUEADO; apenas contas ativas autenticam.
- O contrato prevê o campo opcional **clinicaId** no login enquanto a regra de e-mail único por clínica não é fechada (Bloco 8, seção 9); a tela deve estar preparada para exibir ou ocultar esse campo.
- A proteção contra tentativas indevidas (por exemplo, bloqueio após falhas) não está definida nos requisitos; o item permanece como ponto a validar (seção 6).

## 3.3 Tela 03 — Cadastro e consulta de pacientes

| | |
|---|---|
| **Perfil** | Paciente (autorregistro) e Recepcionista |
| **Requisitos** | RF005, RF006; história de usuário de cadastro do paciente e da recepcionista |
| **API (Bloco 8)** | POST /pacientes; GET /pacientes; GET e PATCH /pacientes/{id} |

Permite cadastrar pacientes. O paciente cria a própria conta; a recepcionista registra pacientes durante o atendimento presencial e consulta ou atualiza os cadastros existentes.

**Funcionalidades principais:**

- Cadastrar os dados do paciente: nome, e-mail, telefone, CPF, data de nascimento e observações cadastrais (atributos do modelo de classes e da tabela de pacientes, Blocos 4 e 5).
- Validar CPF, e-mail e campos obrigatórios antes do envio.
- Identificar cadastros já existentes: o CPF é único por clínica e o e-mail é único por clínica; o conflito é informado ao usuário.
- **Recepcionista:** pesquisar pacientes, consultar e atualizar cadastros.
- **Paciente:** o autorregistro é anônimo e informa a clínica de destino.

**Regras e coerência com os blocos:**

- Os dados pessoais seguem o dicionário de dados e a LGPD (RNF012); CPF, e-mail e telefone não aparecem em logs.
- O autorregistro é a única exceção à regra de nunca confiar em clinicaId enviado pelo cliente; a forma de escolher a clínica (endereço específico ou convite) está pendente (Bloco 5, seção 8.1).
- O primeiro acesso de paciente cadastrado pela recepção, sem senha, é um fluxo a confirmar com o Product Owner (Bloco 8, seção 9).
- A recepcionista só enxerga pacientes da própria clínica (RF025, RNF014).

## 3.4 Tela 04 — Dashboard administrativo

| | |
|---|---|
| **Perfil** | Administrador |
| **Requisitos** | RF023 (Could), RF025 |
| **API (Bloco 8)** | GET /relatorios/consultas (indicadores de agendamentos) |

Apresenta uma visão geral das operações da clínica, com indicadores de agendamentos e atalhos para os módulos administrativos.

**Funcionalidades principais:**

- Exibir a quantidade de pacientes cadastrados e de profissionais ativos.
- Mostrar consultas agendadas, canceladas e realizadas.
- Apresentar indicadores de comparecimento e ausência, obtidos dos registros de atendimento.
- Exibir gráfico de agendamentos por período.
- Disponibilizar atalhos para as telas 06, 09 e 10.

**Regras e coerência com os blocos:**

- Todos os indicadores se limitam à clínica do token (RF025, RNF014).
- Por depender de RF023, a tela tem prioridade Could e pode ser deslocada para uma versão posterior sem comprometer os fluxos essenciais de agendamento (Bloco 7, seção 4).
- O contrato do Bloco 8 não traz endpoint específico para a contagem de pacientes e profissionais ativos; o item deve ser definido na implementação (seção 6).

## 3.5 Tela 05 — Agendamento de consulta

| | |
|---|---|
| **Perfil** | Paciente e Recepcionista |
| **Requisitos** | RF010, RF011, RF012, RF014, RF015, RF018, RF021, RNF003, RNF015 |
| **API (Bloco 8)** | GET /especialidades; GET /profissionais; GET /disponibilidades; POST /agendamentos |

Permite agendar consultas pelo navegador, seguindo a sequência de escolha de especialidade, profissional, data e horário.

**Funcionalidades principais:**

- Selecionar a especialidade entre as cadastradas e ativas da clínica.
- Pesquisar e escolher o profissional por nome ou especialidade.
- Consultar as datas e os horários livres do profissional, sempre atualizados.
- **Recepcionista:** selecionar o paciente em nome de quem o agendamento é feito; para o paciente, o agendamento usa a identidade do token.
- Confirmar a reserva; o agendamento é criado com status AGENDADO e a confirmação é enviada ao paciente por e-mail ou SMS.

**Regras e coerência com os blocos:**

- A verificação de conflito acontece no back-end, com transação e índice único parcial; se o horário deixou de estar disponível, a resposta é HTTP 409 e a tela informa a indisponibilidade e devolve o usuário à escolha de horário (RF018, RNF008; Bloco 4, seção 4.2).
- A operação do paciente deve ser concluída em no máximo 5 etapas (RNF015): especialidade, profissional, horário e confirmação; na recepção entra a escolha do paciente.
- Tempo de resposta de pesquisa e horários inferior a 2 s e de agendamento de até 2 s, no percentil 95 (RNF002, RNF003).
- A confirmação por notificação é interna ao serviço de agendamento (RF021); a tela apenas exibe o resultado.

## 3.6 Tela 06 — Gerenciamento de agendamentos

| | |
|---|---|
| **Perfil** | Recepcionista (operação) e Administrador (acompanhamento) |
| **Requisitos** | RF015, RF016, RF017, RF018, RF019, RF025, RNF013 |
| **API (Bloco 8)** | GET /agendamentos; GET /agendamentos/{id}; PATCH /agendamentos/{id}; PATCH /agendamentos/{id}/cancelamento; PATCH /agendamentos/{id}/confirmacao |

Centraliza as consultas da clínica para acompanhamento e organização da agenda.

**Funcionalidades principais:**

- Listar consultas por data, paciente e profissional, com pesquisa.
- Filtrar por status: AGENDADO, CONFIRMADO, CANCELADO e REALIZADO.
- Remarcar consulta escolhendo outro horário disponível; o horário anterior é liberado e não pode surgir consulta duplicada.
- Cancelar agendamentos nos status AGENDADO ou CONFIRMADO, liberando a disponibilidade.
- Confirmar a presença (de AGENDADO para CONFIRMADO) quando solicitado pelo paciente ou pela recepção.
- Consultar os detalhes de cada consulta e acompanhar as alterações realizadas (auditoria).

**Regras e coerência com os blocos:**

- No contrato do Bloco 8 apenas Paciente e Recepção alteram agendamentos; por isso o Administrador usa a tela para consulta e acompanhamento. Se o Administrador também deve remarcar ou cancelar, o contrato precisa ser revisto (seção 6).
- Usuário não autorizado não altera agendamento de terceiros (Bloco 9, matriz de cenários).
- As regras detalhadas de cancelamento e remarcação precisam ser confirmadas com o Product Owner (Bloco 9, seção 10).
- Alterações em agendamentos geram registro de auditoria com usuário e momento (RNF013).

## 3.7 Tela 07 — Agenda do profissional

| | |
|---|---|
| **Perfil** | Profissional de saúde |
| **Requisitos** | RF013, RF019, RF026; histórias de usuário do profissional |
| **API (Bloco 8)** | GET /agendamentos (escopo do profissional); POST e PATCH /disponibilidades; POST /disponibilidades/bloqueios |

Permite que cada profissional acompanhe seus compromissos e gerencie os períodos em que atende.

**Funcionalidades principais:**

- Visualizar as consultas do dia e da semana e consultar o histórico de agendamentos.
- Consultar horários e pacientes agendados.
- Cadastrar e atualizar períodos de disponibilidade.
- Bloquear horários e informar indisponibilidades.
- Visualizar alterações e cancelamentos.
- Consultar detalhes administrativos dos atendimentos.

**Regras e coerência com os blocos:**

- Antes de bloquear, o sistema verifica agendamentos com status AGENDADO ou CONFIRMADO no período; se existir, o bloqueio é impedido (HTTP 409, BLOQUEIO_NAO_PERMITIDO) e a tela informa o agendamento associado (RF026; Bloco 4, seção 4.3).
- A verificação e a gravação do bloqueio ocorrem na mesma transação, para evitar novo agendamento entre as duas etapas (RNF008).
- A visualização é diária e semanal, conforme a história de usuário; a visão mensal do material original foi retirada por não constar nos requisitos.
- Não há prontuário, diagnóstico nem prescrição: são funcionalidades fora da versão atual (RF-W03 a RF-W05).

## 3.8 Tela 08 — Registro de atendimento

| | |
|---|---|
| **Perfil** | Profissional de saúde e Recepcionista (autorizados) |
| **Requisitos** | RF020; histórias de usuário do profissional e da recepcionista |
| **API (Bloco 8)** | POST /agendamentos/{id}/atendimento |

Registra o comparecimento ou a ausência do paciente na consulta agendada, apoiando o fluxo ambulatorial.

**Funcionalidades principais:**

- Identificar o paciente agendado e consultar data, horário e profissional.
- Registrar **comparecimento** (presença PRESENTE) ou **ausência** (presença AUSENTE).
- Atualizar o status do agendamento para REALIZADO, conforme o ciclo de vida do Bloco 5 (seção 5.1).
- Manter o registro de quem realizou a operação e de quando ela ocorreu.

**Regras e coerência com os blocos:**

- O atendimento está ligado ao agendamento por composição, no máximo um por agendamento (Bloco 4, diagrama de classes).
- A tela é administrativa: não registra prontuário nem dados clínicos (escopo, Bloco 1).
- A operação é registrada na auditoria (RNF013).

## 3.9 Tela 09 — Relatórios administrativos

| | |
|---|---|
| **Perfil** | Administrador |
| **Requisitos** | RF023 (Could), RF025 |
| **API (Bloco 8)** | GET /relatorios/consultas |

Permite consultar indicadores operacionais sobre agendamentos, cancelamentos e ausências.

**Funcionalidades principais:**

- Selecionar o período e gerar o relatório de consultas.
- Consultar os quantitativos de agendamentos.
- Apresentar cancelamentos e ausências.
- Filtrar a análise por profissional ou especialidade.

- Exibir os resultados em tabelas e gráficos.

**Regras e coerência com os blocos:**

- A exportação de relatórios, que constava no material original, foi retirada: não existe no RF023 nem no contrato da API. Pode entrar como evolução futura, se for aprovada (seção 6).
- Prioridade Could: pode ser deslocada para versão posterior (Bloco 7).
- Dados restritos à clínica do token (RF025, RNF014).

## 3.10 Tela 10 — Usuários, permissões, profissionais e especialidades

| | |
|---|---|
| **Perfil** | Administrador |
| **Requisitos** | RF003, RF007, RF008, RF009, RF024, RNF013 |
| **API (Bloco 8)** | GET e POST /usuarios; PATCH /usuarios/{id}/status; PUT /usuarios/{id}/papeis; GET e POST /profissionais; GET e PATCH /profissionais/{id}; PUT /profissionais/{id}/especialidades; GET e POST /especialidades; PATCH /especialidades/{id} |

Administra as contas do sistema, os profissionais e as especialidades, e define o que cada perfil pode fazer.

**Funcionalidades principais:**

- Cadastrar e consultar contas de usuários.
- Ativar e desativar contas.
- Definir papéis e permissões de acesso.
- Cadastrar, atualizar e desativar profissionais (nome, registro profissional, e-mail e especialidades).
- Cadastrar, atualizar e desativar especialidades.
- Associar profissionais a uma ou mais especialidades.
- Registrar na auditoria as alterações administrativas relevantes.

**Regras e coerência com os blocos:**

- No modelo físico, Recepcionista e Administrador são papéis e permissões (tabela usuario_papel), enquanto Paciente e Profissional têm tabelas de perfil associadas ao usuário (Blocos 4 e 5); a tela deve refletir essa estrutura.
- Os registros de auditoria guardam usuário responsável, clínica, ação, entidade afetada e data e hora (RNF013).
- O profissional é sempre vinculado à clínica do administrador autenticado (diagrama de atividades do Bloco 4, seção 7.2).

## 3.11 Tela 11 — Meus agendamentos (nova)

| | |
|---|---|
| **Perfil** | Paciente |
| **Requisitos** | RF016, RF017, RF019; histórias de usuário do paciente |
| **API (Bloco 8)** | GET /agendamentos (escopo do paciente); PATCH /agendamentos/{id}; PATCH /agendamentos/{id}/cancelamento; PATCH /agendamentos/{id}/confirmacao |

Tela acrescentada porque o paciente precisa acompanhar, cancelar e remarcar as próprias consultas, e o material original não trazia tela para isso.

**Funcionalidades principais:**

- Listar consultas futuras e anteriores do paciente autenticado.
- Consultar os detalhes de cada consulta.
- Cancelar uma consulta agendada.
- Remarcar para outro horário disponível, sem gerar consultas duplicadas.
- Confirmar a presença quando solicitado, passando de AGENDADO para CONFIRMADO.

**Regras e coerência com os blocos:**

- Somente os agendamentos associados à conta do paciente são exibidos (história de usuário de histórico).
- Os lembretes (RF022) são notificações enviadas por e-mail ou SMS; não há tela nem endpoint para eles.

## 3.12 Tela 12 — Meus dados (nova)

| | |
|---|---|
| **Perfil** | Todos os perfis autenticados |
| **Requisitos** | RF004; história de usuário “visualizar e atualizar minhas informações cadastrais” |
| **API (Bloco 8)** | GET e PATCH /usuarios/me |

Tela acrescentada para que qualquer usuário autorizado visualize e atualize as próprias informações cadastrais.

**Funcionalidades principais:**

- Visualizar os dados cadastrais.
- Alterar as informações permitidas ao perfil.
- Validar os dados e salvar, exibindo as informações atualizadas.

**Regras e coerência com os blocos:**

- Cada usuário só acessa os próprios dados; campos de perfil e de clínica não são editáveis pela interface.
- Dados pessoais seguem a LGPD e não aparecem em logs (RNF012).

# 4. Requisitos transversais às telas

- **Autenticação e autorização (RNF010, ADR-006):** todas as telas, exceto 01, 02 e o autorregistro da tela 03, exigem JWT. O back-end valida assinatura, expiração, perfil e clínica em cada requisição; a interface trata 401 e 403 com redirecionamento ao login ou mensagem de falta de permissão.
- **Isolamento por clínica (RF025, RNF014, ADR-012):** os dados exibidos pertencem sempre à clínica do token.
- **Desempenho (RNF001 a RNF003):** consulta e navegação comuns em até 300 ms; pesquisa de profissionais e horários abaixo de 2 s; agendamento em até 2 s, no percentil 95.
- **Usabilidade (RNF015):** as principais operações do paciente em no máximo 5 etapas.
- **Compatibilidade e acessibilidade (RNF016, RNF017):** Chrome, Firefox e Edge; navegação por teclado e identificação textual dos elementos interativos.

- **Proteção de dados (RNF009, RNF012, LGPD):** HTTPS com TLS 1.2 ou superior; dados pessoais tratados conforme o dicionário de dados.
- **Auditoria (RNF013):** operações administrativas críticas registram usuário responsável e momento.
- **Erros:** as falhas da API seguem o formato único { timestamp, status, codigo, mensagem, detalhes[] } (Bloco 8, seção 4.4), e a interface traduz os códigos 400, 401, 403 e 409 em mensagens compreensíveis.

# 5. Ajustes realizados em relação ao material original

A tabela resume o que foi alterado nos dados de tela para manter a coerência com os demais blocos.

| Item | Material original | Ajuste | Fundamento |
|---|---|---|---|
| Perfis | Médico | Profissional de saúde | Blocos 1 a 4 |
| Quantidade de telas | 10 telas | 12 telas (acrescentadas 11 e 12) | RF004, RF016, RF017, RF019 |
| Tela 01 | Exibe especialidades disponíveis ao público | Especialidades somente após login | RF010, RF025, RNF010, Bloco 8 |
| Tela 02 | Proteger o acesso contra tentativas indevidas | Mensagem de erro 401 e controles de RNF009 a RNF011; bloqueio por tentativas fica pendente | Bloco 4 (seq. 4.1), Bloco 8 |
| Tela 03 | Validar CPF e identificar cadastros existentes | CPF e e-mail únicos por clínica; autorregistro informa a clínica | Blocos 5 e 8 |
| Telas 04 e 09 | Dashboard e relatórios sem prioridade; exportação de relatórios | Prioridade Could (RF023); exportação retirada | Blocos 2 e 7 |
| Tela 06 | Recepcionista e administrador remarcam e cancelam | Operação pela recepção; administrador acompanha | Bloco 8, seção 4.2 |
| Tela 07 | Consultas do dia, semana ou mês; bloqueios | Dia e semana; bloqueio impedido com agendamento AGENDADO ou CONFIRMADO | RF026, Bloco 4 (seq. 4.3) |
| Tela 08 | Atualizar “status administrativo” | Presença PRESENTE ou AUSENTE e status REALIZADO | Bloco 5, seção 5.1 |
| Tela 10 | Vincular contas aos perfis de paciente, médico, recepcionista ou administrador | Papéis para Recepcionista e Administrador; perfis para Paciente e Profissional | Blocos 4 e 5 |

# 6. Limites e pontos a validar

- A página inicial institucional não tem requisito funcional nem pacote próprio na WBS do Bloco 7; sua prioridade deve ser definida com o Product Owner.
- Regra de login por e-mail e uso do campo clinicaId (Bloco 8, seção 9) e política de bloqueio por tentativas inválidas.
- Forma de escolha da clínica no autorregistro e fluxo de primeiro acesso do paciente cadastrado pela recepção.

- Se o Administrador deve poder remarcar ou cancelar agendamentos; hoje o contrato da API restringe a Paciente e Recepção.
- Endpoint para as contagens de pacientes e profissionais ativos do dashboard.
- Inclusão ou não da exportação de relatórios em versões futuras.
- Regras detalhadas de cancelamento e remarcação (Bloco 9, seção 10).
- As telas 11 e 12 são propostas deste anexo e ainda não constam nas estimativas do Bloco 7, mas se apoiam em requisitos e histórias já estimados.

Este anexo não amplia o escopo funcional definido no Bloco 2 e não substitui os requisitos, o contrato da API nem o modelo de dados.